Self-Calibrating Polyphase Fourier Reconstruction

Unknown Jump Count, Oracle-Equivalent Internal De-Aliasing, Rate-Preserving Edge Recovery, and Gibbs-Free Synthesis

Komorebi P.

Abstract

Raw Fourier partial sums represent a discontinuity by a global trigonometric polynomial and therefore retain Gibbs oscillations. Exact algebraic methods can recover finite-rate edge parameters, while decimation can improve the conditioning and asymptotic accuracy of the underlying Prony system. Those gains come with two practical burdens. The number of active jumps may be unknown, and decimation replaces each edge phase z_j by the aliased phase $u_j = z_j^N$, leaving N possible roots. Existing exponential-analysis methods already show that shifted samples can remove this aliasing. The present paper carries that mechanism into piecewise-smooth Fourier reconstruction and proves a rate-preserving synthesis theorem.

The method reorganizes high-frequency derivative moments into offset polyphase channels

$$m_{qN+r} = \sum_{j=1}^{L} b_j z_j^r u_j^q + e_{qN+r}.$$

A Hankel singular-value gap identifies the unknown jump count. A decimated Prony solve recovers the aliased nodes u_j. The coefficients across offsets form geometric progressions in r. Their phases give an internal estimate of z_j, which selects the correct root of u_j without an externally supplied jump neighborhood. Several offsets also provide a closed-form phase fit and an a posteriori validation residual. Under a Fourier-tail assumption on the smooth background, a uniform Prony-conditioning hypothesis, and a branch-margin condition, the phase lift preserves the decimated method's one-order gain. If the aliased-node error is $O(N^{-p-1})$, the unaliased jump-location error is $O(N^{-p-2})$.

The recovered singular packet is synthesized in its native coordinate. The smooth residual is reconstructed by a stable smooth-class operator. This replaces the persistent $O(1)$ Gibbs overshoot by explicit edge-parameter and smooth-tail errors. The paper proves exact recovery for a bandlimited control class, gives an explicit high-frequency budget of $3L_{\max} + 1$ coefficients for unknown-order self-calibration, establishes a deterministic perturbation theorem for non-bandlimited backgrounds, extends the construction to higher jump jets, and proves a stable synthesis inequality. A single Python file generates every experiment, figure, table, and numerical certificate. The included implementation passes 31 named checks.

Claim boundary. The shifted-channel de-aliasing identity is established exponential-analysis machinery. Decimated algebraic Fourier reconstruction, finite-rate innovation methods, and Gibbs-resolution methods also have substantial prior literatures. The mathematical contribution developed here is the rate-preserving integration of unknown-order detection, offset-channel branch selection, validation, and edge-native synthesis for a piecewise-smooth Fourier model. The paper does not claim separation-free recovery, arbitrary edge geometry, or a model-free violation of known stability barriers.

Contents

1 The problem that remains after exact unknown-edge recovery 3

2 Relation to the established literature 3

3 Signal class and Fourier convention 4

4 Why decimation helps and why it aliases 5

5 Unknown jump count from a Hankel gap 7

6 Offset coefficients and internal phase recovery 10

7 A posteriori phase validation 12

8 Branch lifting from aliased nodes 13

9 Rate-preserving polyphase lifting 15

10 Self-calibrating reconstruction theorem 18

11 A stable synthesis theorem 22

12 Worked example from Fourier data to reconstruction 24

13 Direct oracle comparison and random-family stress test 27

14 Noise, order selection, and the branch margin 29

15 Higher jump jets and confluent Prony systems 31

16 Computational complexity 33

17 What would count as a field-level advance 33

18 Limitations and next theorem 36

19 Numerical certificate ledger 36

20 Conclusion 37

A Detailed algebra for the multi-offset estimator 38

B Detailed algebra for the branch lift 38

C Exact symbolic confluent example 38

D Selected numerical tables 39

E Implementation map 39

F Complete Python implementation 40

1 The problem that remains after exact unknown-edge recovery

The previous finite-rate recovery theorem answered an important question. If a periodic signal has a bandlimited smooth packet and a finite piecewise-constant edge packet, then a finite set of Fourier coefficients determines the edge count, locations, amplitudes, smooth packet, and complete signal. That theorem removes the known-edge assumption. It does not yet reach the most useful piecewise-smooth setting.

Three difficulties remain.

1. The smooth background need not be bandlimited. Its high-frequency coefficients perturb the exact exponential moment system.
2. A decimated Prony method recovers $u_j = z_j^N$ rather than z_j. Taking a principal root is wrong in general.
3. The number of active jumps may be unknown, and a wrong model order can produce an apparently plausible reconstruction.

The purpose of this paper is to close those three gaps in one finite architecture.

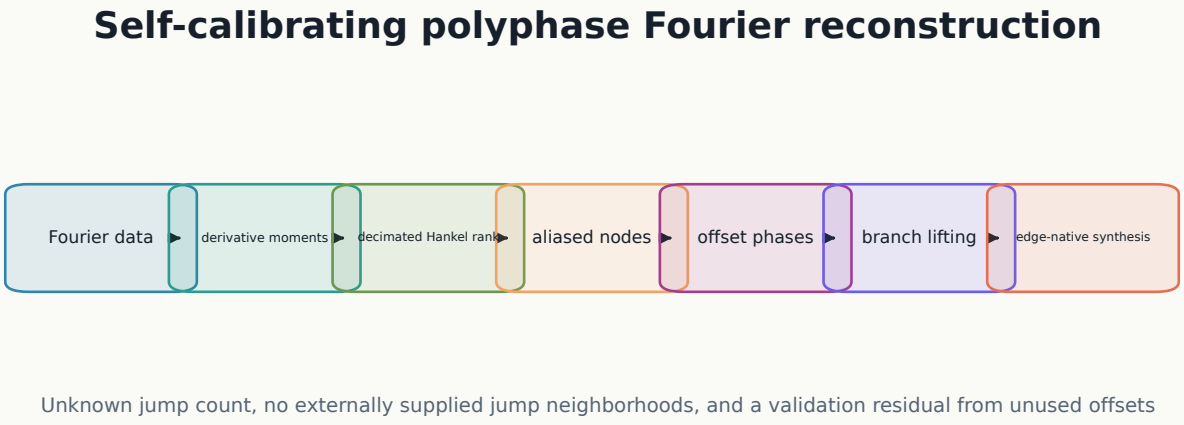


Figure 1: The complete recovery chain. Every stage produces a mathematical object and a diagnostic quantity.

The problem is not to make the classical Fourier partial sum stop exhibiting Gibbs behavior. The problem is to infer the singular packet from the Fourier data, remove it before smooth reconstruction, and return it through an exact synthesis map.

2 Relation to the established literature

The relevant literature contains several mature lines of work.

- Fourier–Gegenbauer methods recover high accuracy on smooth subintervals after discontinuity information has been located.
- Eckhoff-type algebraic reconstruction turns high Fourier coefficients into nonlinear equations for jump

parameters. Decimated equations attain the maximal asymptotic rate under stated smoothness and model assumptions.

- Finite-rate-of-innovation sampling and Prony methods recover sparse innovation parameters from moments or generalized samples.
- Shifted and decimated exponential analysis uses several sampling offsets to resolve aliasing, estimate model order, and validate recovered components.
- Super-resolution theory quantifies the instability created by closely spaced singularities.

The new theorem-facing object is the way these ingredients are joined. Decimation is used for accuracy. Offset polyphase channels are used for internal branch selection. Hankel gaps are used for self-calibration. Projector synthesis keeps the recovered singular packet out of the smooth global approximation.

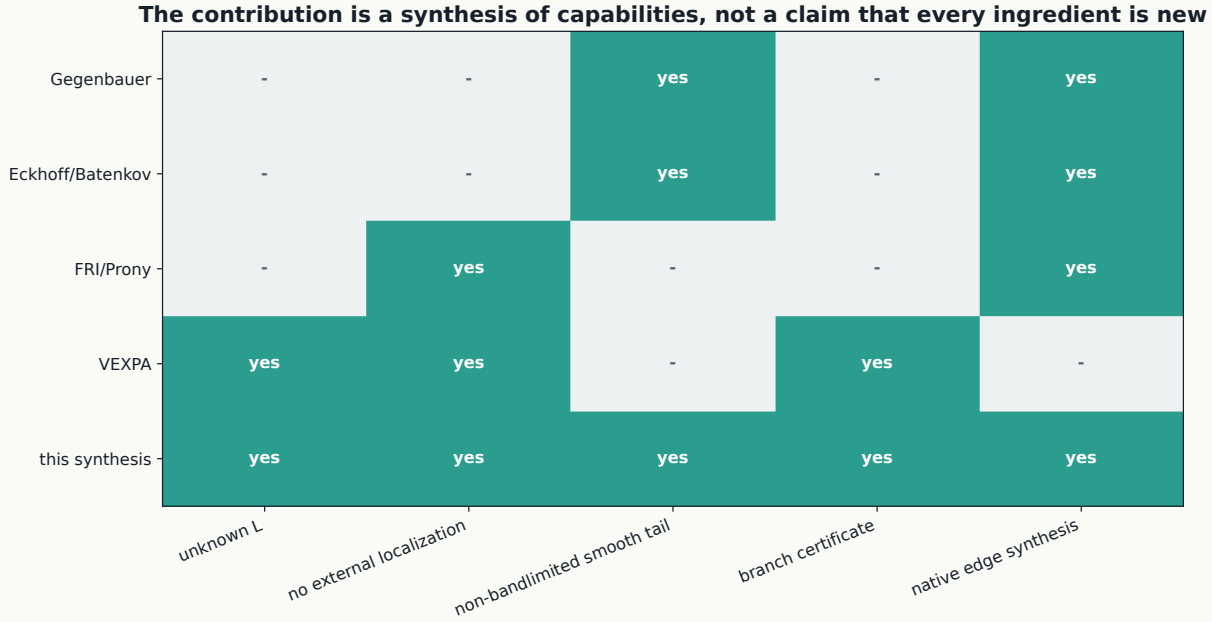


Figure 2: A schematic comparison of capabilities. The table is not a priority claim. It records the specific combination developed in this paper.

Boundary 2.1 (No contradiction with the general stability barrier)**.** *Stable recovery of arbitrary analytic nonperiodic functions from their first Fourier coefficients cannot converge faster than root-exponentially without instability. The present result concerns a nonlinear finite-rate singular model with separation, amplitude, and Fourier-tail assumptions. It proves algebraic parameter rates on that narrower class.*

3 Signal class and Fourier convention

Work on the periodic torus $\mathbb{T} = \mathbb{R}/(2\pi\mathbb{Z})$ with Fourier coefficients

$$\widehat{f}(n) = \frac{1}{2\pi}\int_0^{2\pi} f(x)e^{-inx}\,dx.$$

The signal is decomposed as

$$\boxed{f = s + j.}$$

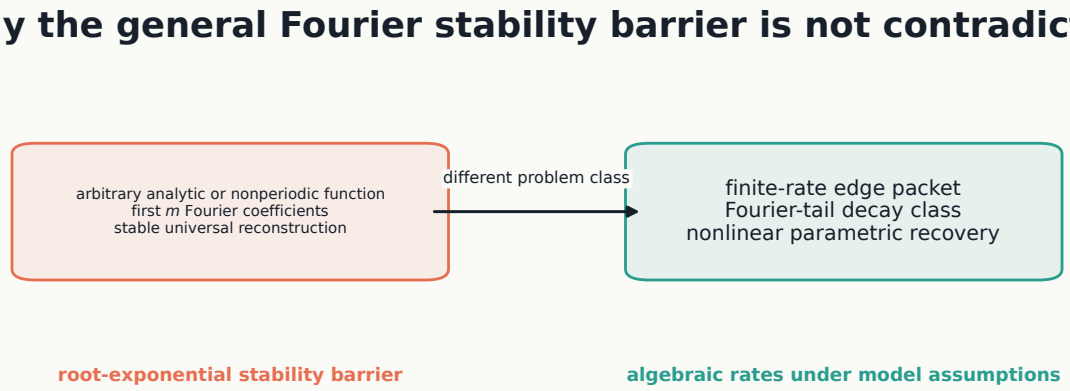


Figure 3: The model class is narrower than the class covered by the general Fourier stability barrier.

The edge packet is periodic and piecewise constant. Its distributional derivative is

$$Dj = \sum_{\ell=1}^{L} a_\ell \delta_{\tau_\ell}, \qquad \sum_{\ell=1}^{L} a_\ell = 0.$$

Set

$$z_\ell = e^{-i\tau_\ell}.$$

For $n \neq 0$,

$$2\pi i n \widehat{j}(n) = \sum_{\ell=1}^{L} a_\ell z_\ell^n.$$

The smooth packet is not assumed bandlimited.

Assumption 3.1 (Fourier-tail class)**.** *For some integer $p \geq 0$ and constant $C_s > 0$,*

$$\boxed{|\widehat{s}(n)| \leq C_s(1+|n|)^{-p-2}.}$$

Define the derivative moments

$$\boxed{m_n = 2\pi i n \widehat{f}(n).}$$

Then

$$m_n = \sum_{\ell=1}^{L} a_\ell z_\ell^n + e_n, \qquad |e_n| \leq 2\pi C_s(1+|n|)^{-p-1}.$$

4 Why decimation helps and why it aliases

Choose an integer decimation factor $N \geq 2$, an initial block index $q_0 \geq 1$, and an offset r. Write

$$n = (q_0 + q)N + r.$$

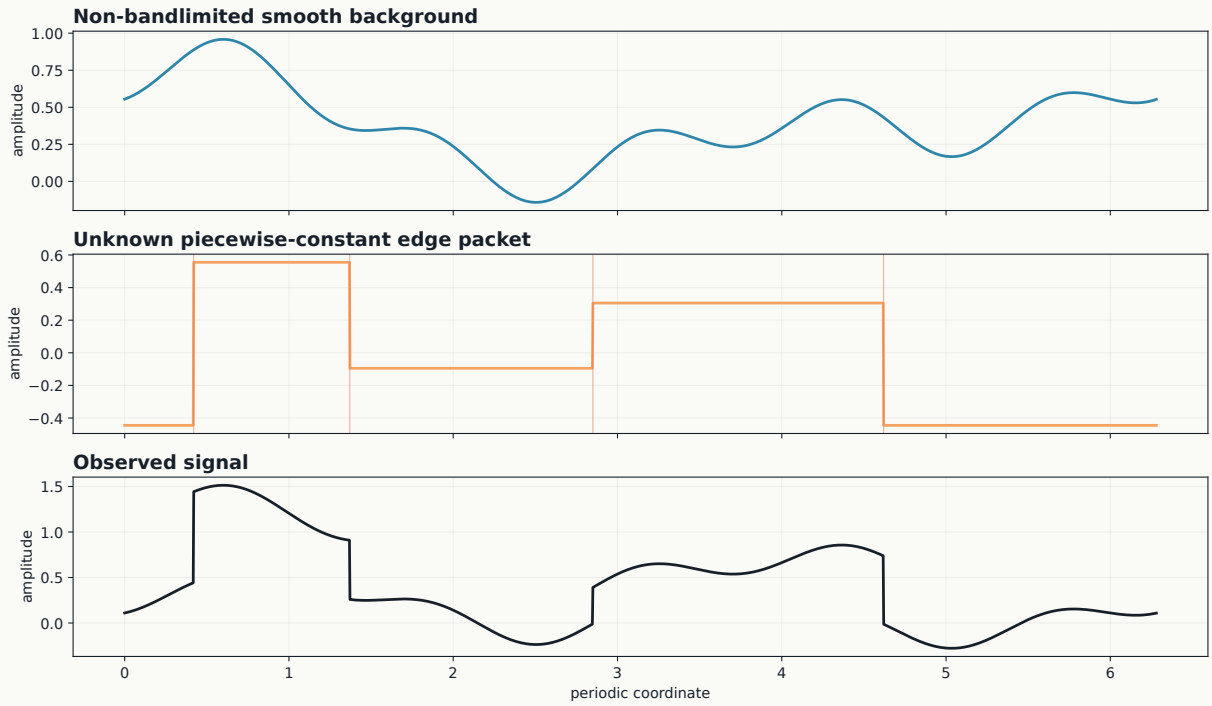


Figure 4: The main numerical example uses a non-bandlimited analytic background and four off-grid jumps.

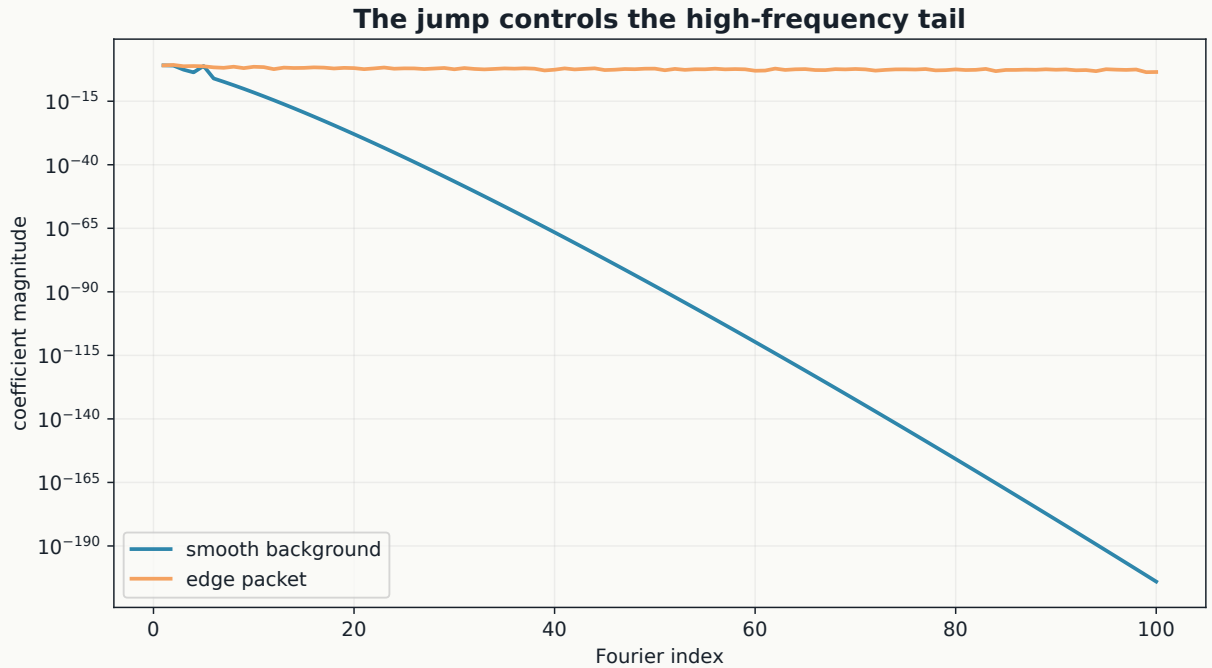


Figure 5: The edge packet has a $1/n$ Fourier tail, while the analytic background decays much faster.

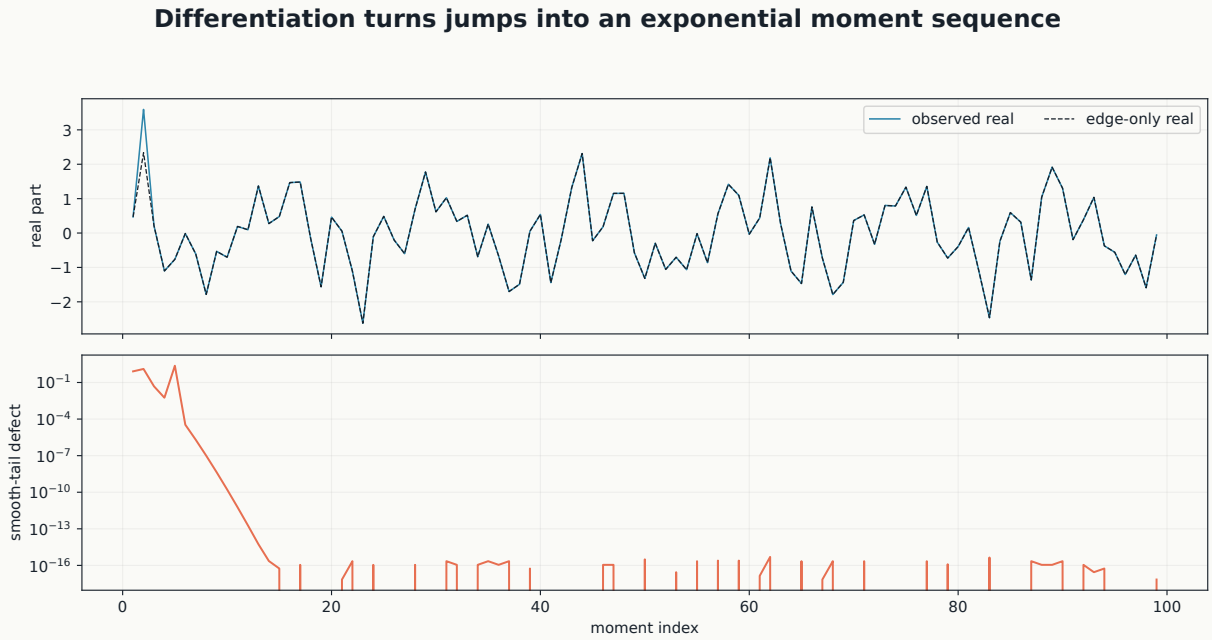


Figure 6: Multiplication by $2\pi in$ converts the edge packet into an exponential moment sequence and leaves the smooth background as a decaying defect.

Then

$$\begin{aligned} m_{(q_0+q)N+r} &= \sum_{\ell=1}^{L} a_\ell z_\ell^{(q_0+q)N+r} + e_{(q_0+q)N+r} \\ &= \sum_{\ell=1}^{L} \underbrace{a_\ell z_\ell^r (z_\ell^N)^{q_0}}_{b_{\ell,r}} \underbrace{(z_\ell^N)^q}_{u_\ell^q} + e_{(q_0+q)N+r}. \end{aligned}$$

Thus every offset channel has the same aliased nodes

$$\boxed{u_\ell = z_\ell^N}$$

and different coefficients

$$\boxed{b_{\ell,r} = a_\ell z_\ell^r u_\ell^{q_0}.}$$

Decimation can separate phases that are close in the original coordinate. It also maps every z_ℓ to z_ℓ^N, so one aliased node has N possible roots. A root-selection mechanism is therefore part of the mathematics, not an implementation detail.

5 Unknown jump count from a Hankel gap

For one offset channel, define the finite Hankel matrix

$$H(y) = (y_{r+c})_{0\le r<R,\ 0\le c<C}.$$

In the exact edge-only model,

$$H_0 = V_R \,\mathrm{diag}(b_1, \dots, b_L) V_C^{\mathsf{T}},$$

where the columns of V_R and V_C are powers of the distinct nodes u_ℓ. Hence

$$\mathrm{rank}\, H_0 = L.$$

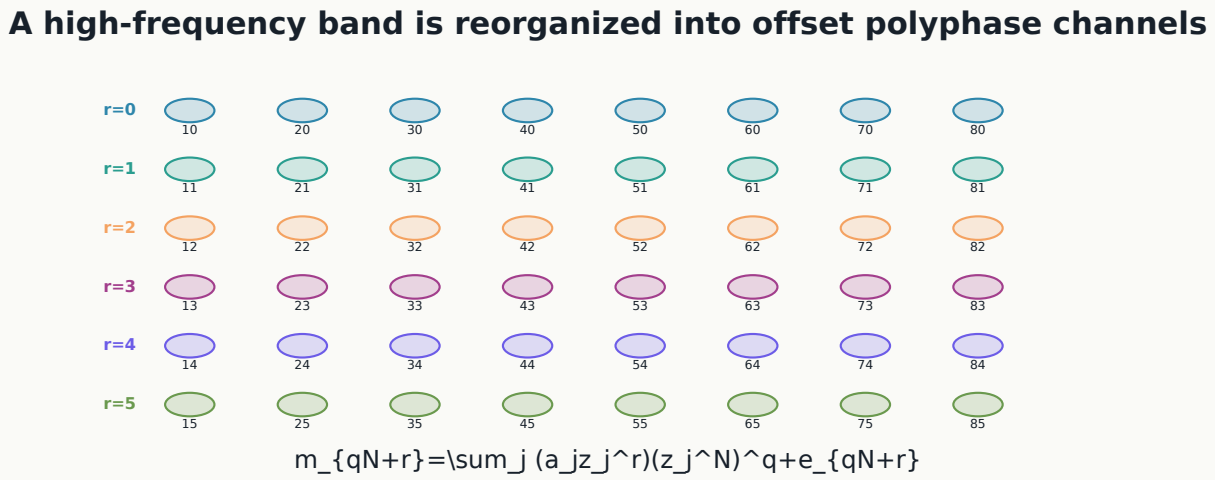


Figure 7: The high-frequency moments are split into residue channels.

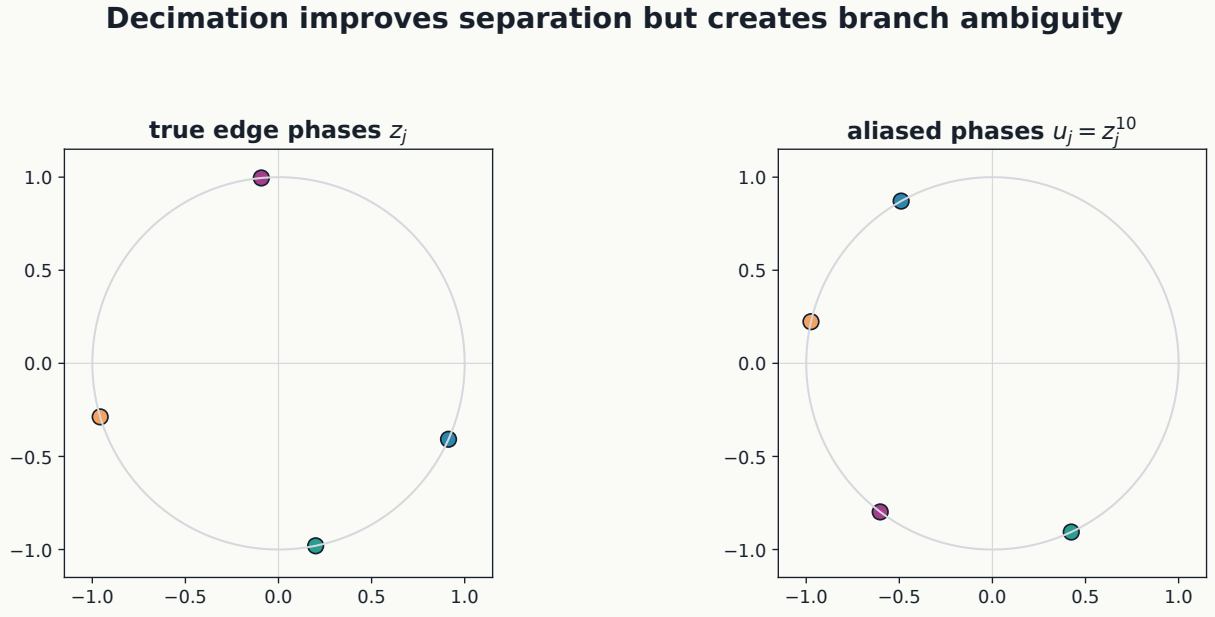


Figure 8: The original phases and their decimated images on the unit circle.

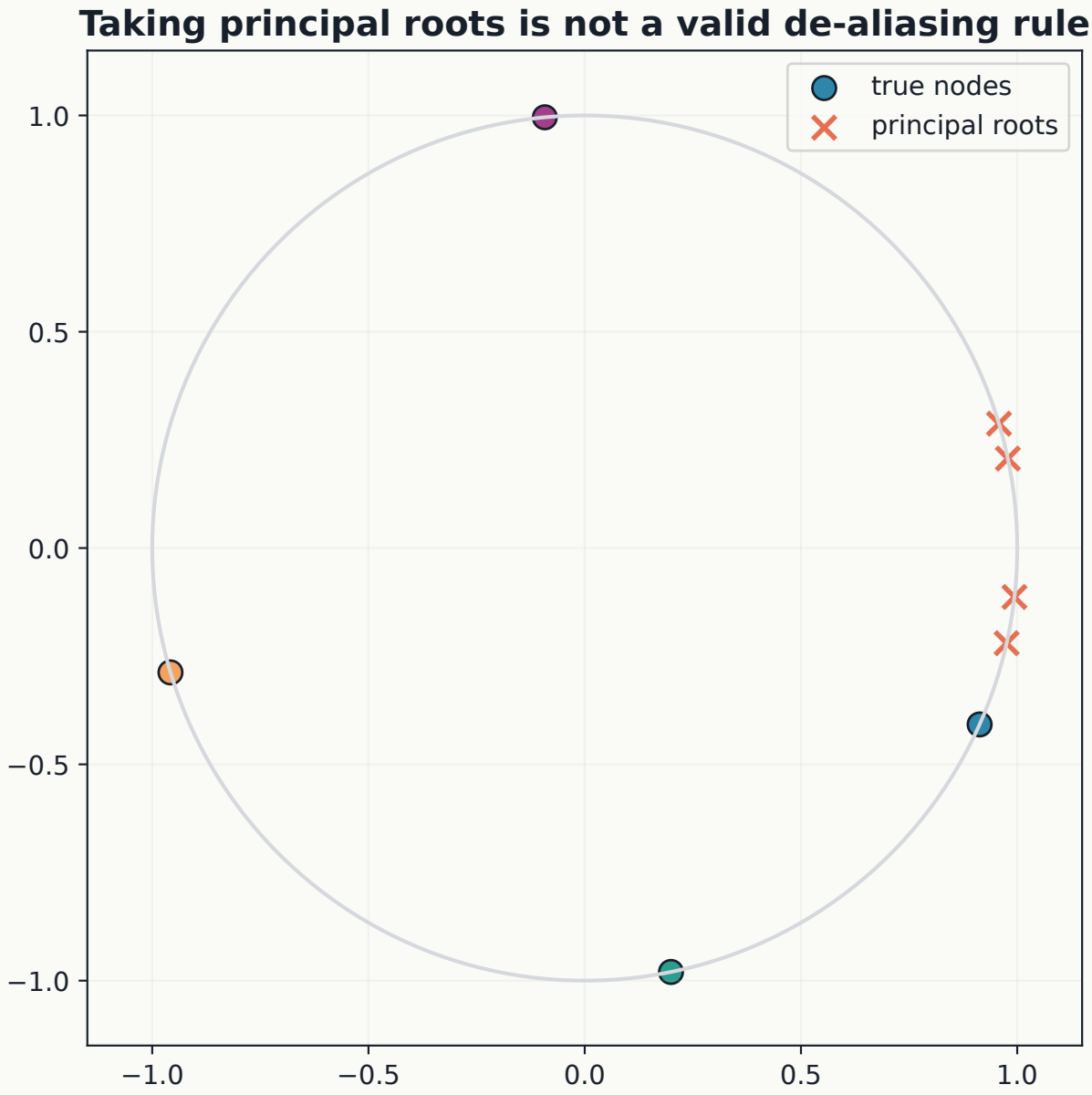


Figure 9: The principal complex root does not generally return the physical edge phase.

Theorem 5.1 (Exact order from Hankel rank)**.** *Assume the aliased nodes $u_1, \dots, u_L$ are distinct and the coefficients b_ℓ are nonzero. If $R, C \geq L$, then*

$$\boxed{\operatorname{rank} H_0 = L.}$$

Proof. Both Vandermonde factors have full column rank, and the diagonal coefficient matrix is invertible. The rank of their product is therefore L. □

The non-bandlimited smooth packet perturbs the matrix.

Theorem 5.2 (Order recovery under a singular-value gap)**.** *Let*

$$H = H_0 + E, \qquad \operatorname{rank} H_0 = L.$$

If

$$\|E\|_2 < \frac{1}{2}\sigma_L(H_0),$$

then any threshold t satisfying

$$\|E\|_2 < t < \sigma_L(H_0) - \|E\|_2$$

selects exactly L singular values of H.

Proof. Weyl's singular-value perturbation inequality gives

$$\sigma_L(H) \geq \sigma_L(H_0) - \|E\|_2 > t$$

and

$$\sigma_{L+1}(H) \leq \|E\|_2 < t.$$

The threshold therefore separates the signal rank from the defect tail. □

The implementation uses the largest adjacent singular-value ratio within a declared upper bound. That rule is a practical estimator. The theorem above states the exact condition that justifies a thresholded rank count.

6 Offset coefficients and internal phase recovery

Once the aliased nodes u_ℓ have been recovered, each offset channel is linear in its coefficients. Solving its Vandermonde system gives estimates of

$$b_{\ell,r} = b_{\ell,0} z_\ell^r.$$

The single-ratio identity

$$\frac{b_{\ell,1}}{b_{\ell,0}} = z_\ell$$

is the basic shifted-sample de-aliasing relation used in exponential analysis.

The present implementation uses several offsets at once.

Definition 6.1 (Multi-offset phase estimator)**.** *Given coefficients $c_0, \dots, c_R$, define*

$$S(c) = \sum_{r=0}^{R-1} \overline{c_r} c_{r+1}.$$

If $S(c) \neq 0$, set

$$\boxed{\Phi(c) = \frac{S(c)}{|S(c)|}.}$$

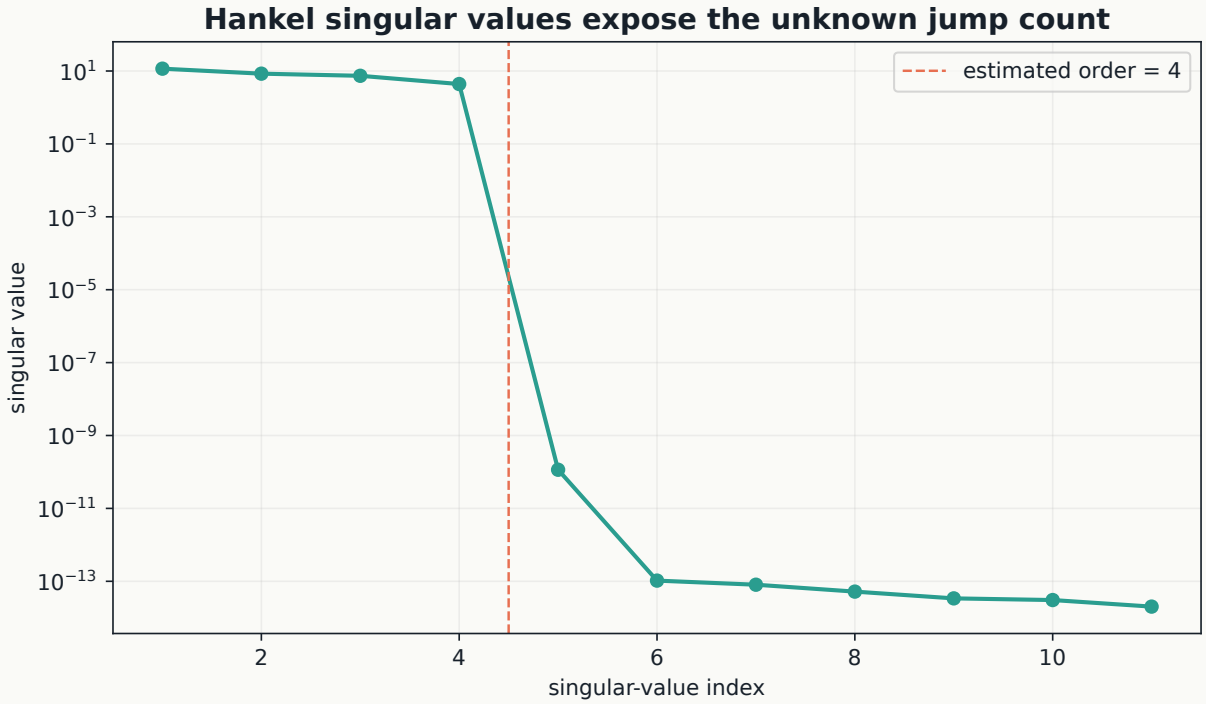


Figure 10: The fourth singular value is separated from the smooth-tail floor by a large gap in the main example.

Theorem 6.2 (Exact multi-offset phase)**.** *If*

$$c_r = bz^r, \qquad b \neq 0, \qquad |z| = 1,$$

then

$$\boxed{\Phi(c) = z.}$$

Proof. Each adjacent product is

$$\overline{c_r} c_{r+1} = |b|^2 z.$$

Hence

$$S(c) = R\,|b|^2 z,$$

and normalization gives z. □

Theorem 6.3 (Deterministic phase perturbation)**.** *Suppose*

$$c_r = bz^r + e_r, \qquad |z| = 1, \qquad |e_r| \leq \varepsilon.$$

Let

$$A = R\,|b|^2, \qquad \delta = R(2\,|b|\,\varepsilon + \varepsilon^2).$$

If $\delta < A$, then $S(c) \neq 0$ and

$$\boxed{|\Phi(c) - z| \leq \frac{2\delta}{A} = \frac{4\varepsilon}{|b|} + 2\left(\frac{\varepsilon}{|b|}\right)^2.}$$

Proof. Write

$$S(c) = Az + E.$$

Expanding each adjacent product gives

$$|E| \le R(2\,|b|\,\varepsilon + \varepsilon^2) = \delta.$$

Since $\delta < A$, the perturbed sum is nonzero. For any nonzero w and $v = Az$,

$$\left|\frac{w}{|w|} - z\right| \le \left|\frac{w}{|w|} - \frac{w}{A}\right| + \left|\frac{w-v}{A}\right| \le \frac{2\,|w-v|}{A}.$$

Substitute $w = S(c)$. □

The deterministic worst-case bound does not improve with the number of offsets because adversarial errors can align. Under independent centered coefficient errors, the averaged adjacent products reduce variance. The numerical experiment isolates that effect in coefficient space.

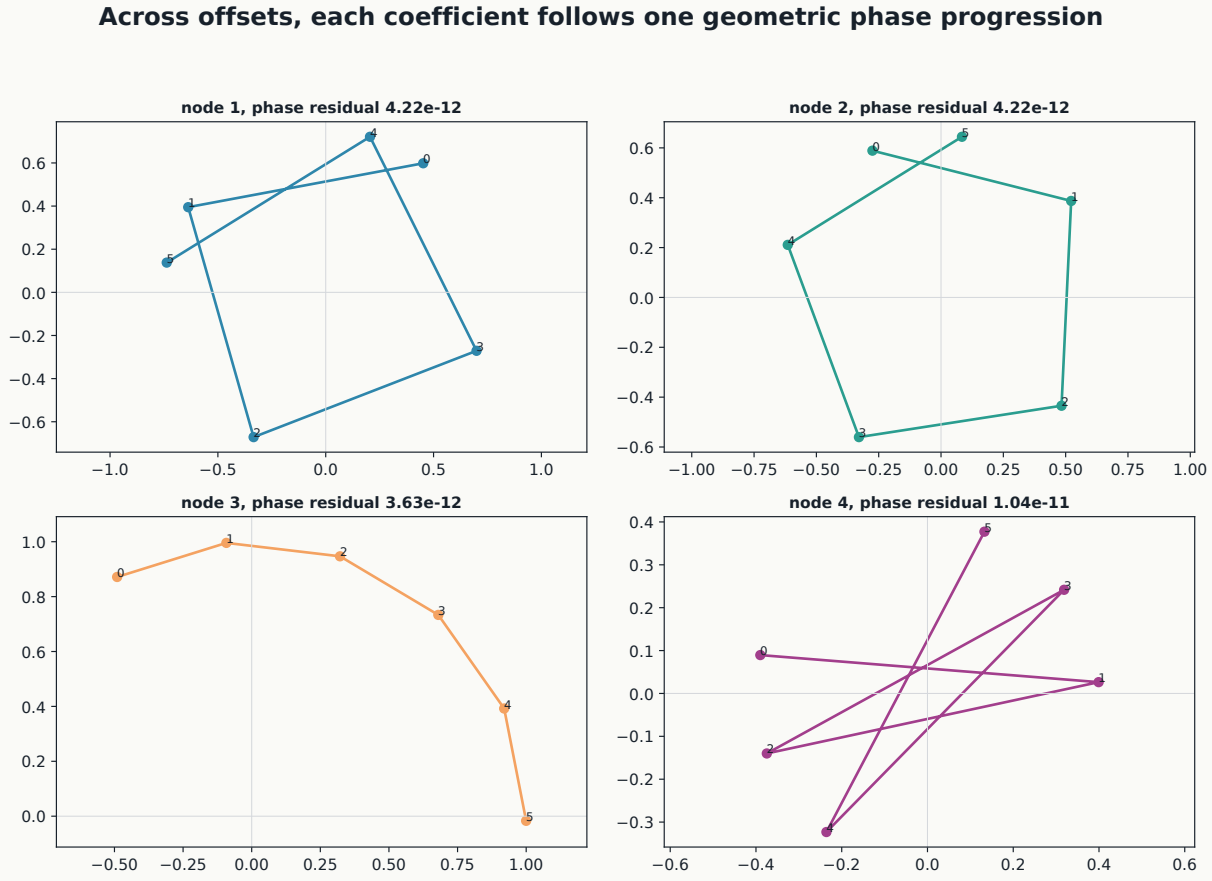


Figure 11: Each recovered coefficient column follows a geometric phase progression across offsets.

7 A posteriori phase validation

The phase model can be checked directly.

Definition 7.1 (Offset recurrence residual)**.** *For a phase estimate $\widetilde{z}$, define*

$$\boxed{V(c,\widetilde{z}) = \frac{\left(\sum_{r=0}^{R-1} |c_{r+1} - \widetilde{z}c_r|^2\right)^{1/2}}{\left(\sum_{r=0}^{R} |c_r|^2\right)^{1/2}}.}$$

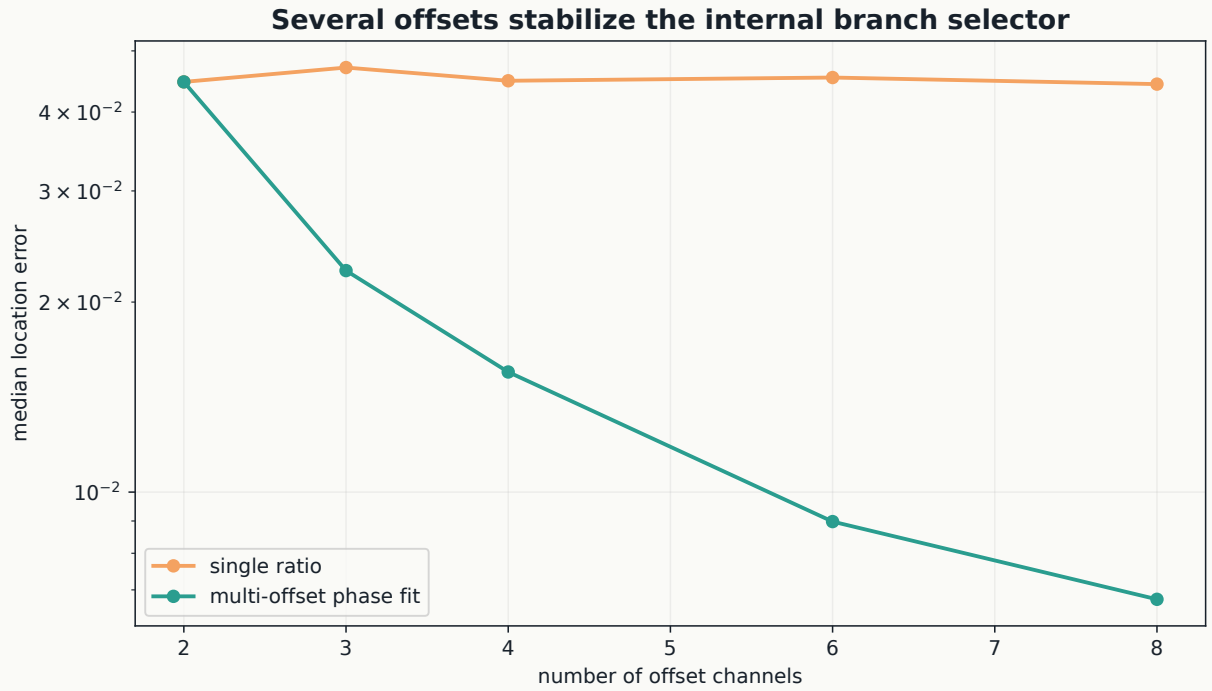


Figure 12: With independent coefficient perturbations, the multi-offset estimator becomes more accurate as additional offsets are included.

Proposition 7.2 (Exact validation)**.** *If $c_r = bz^r$ with $b \neq 0$, then*

$$V(c, z) = 0.$$

Conversely, if $V(c, \widetilde{z}) = 0$ and $c_0 \neq 0$, then

$$c_r = c_0 \widetilde{z}^r$$

for every r.

Proof. The forward statement is immediate. For the converse, zero residual gives

$$c_{r+1} = \widetilde{z} c_r.$$

Induction gives the geometric progression. □

This residual does not prove that the recovered component belongs to the physical signal. It proves that the offset data attached to that component are consistent with one phase. Combining it with the Hankel gap, the original-data residual, and agreement across decimations gives a stronger certificate portfolio.

8 Branch lifting from aliased nodes

Let $u = z^N$. An estimate $\widehat{u}$ determines N candidate roots. The offset phase estimate identifies the correct branch.

Lemma 8.1 (Candidate separation)**.** *The N roots of a nonzero complex number with fixed modulus are separated on their common circle by chord distance*

$$2r \sin\left(\frac{\pi}{N}\right),$$

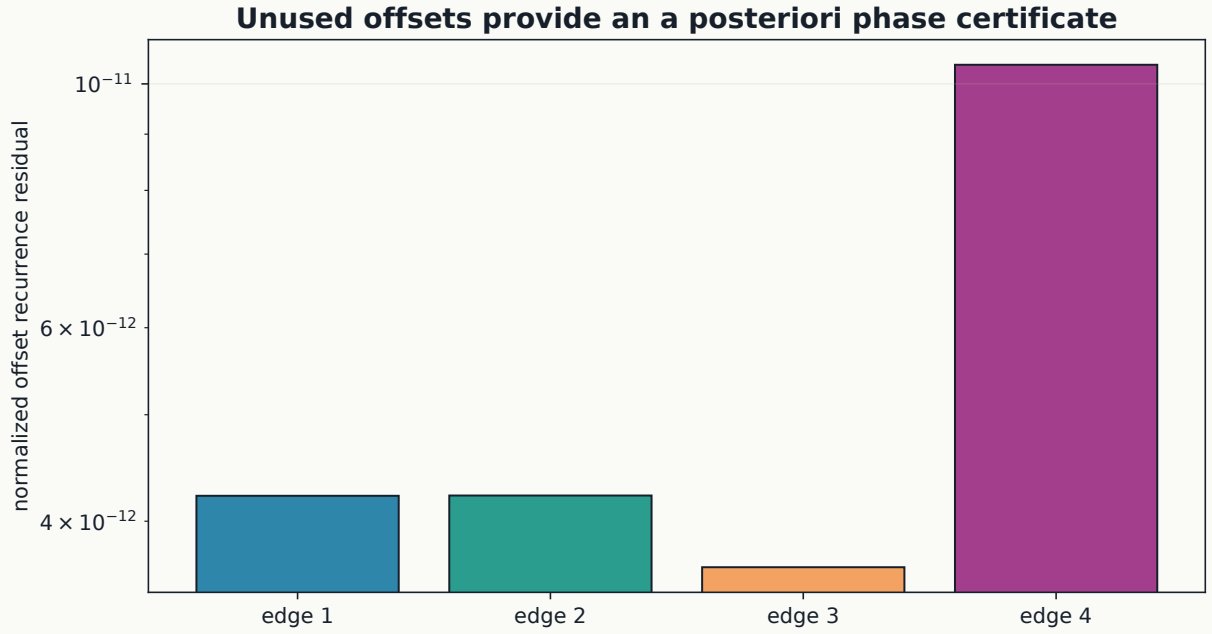


Figure 13: The main example has very small offset recurrence residuals.

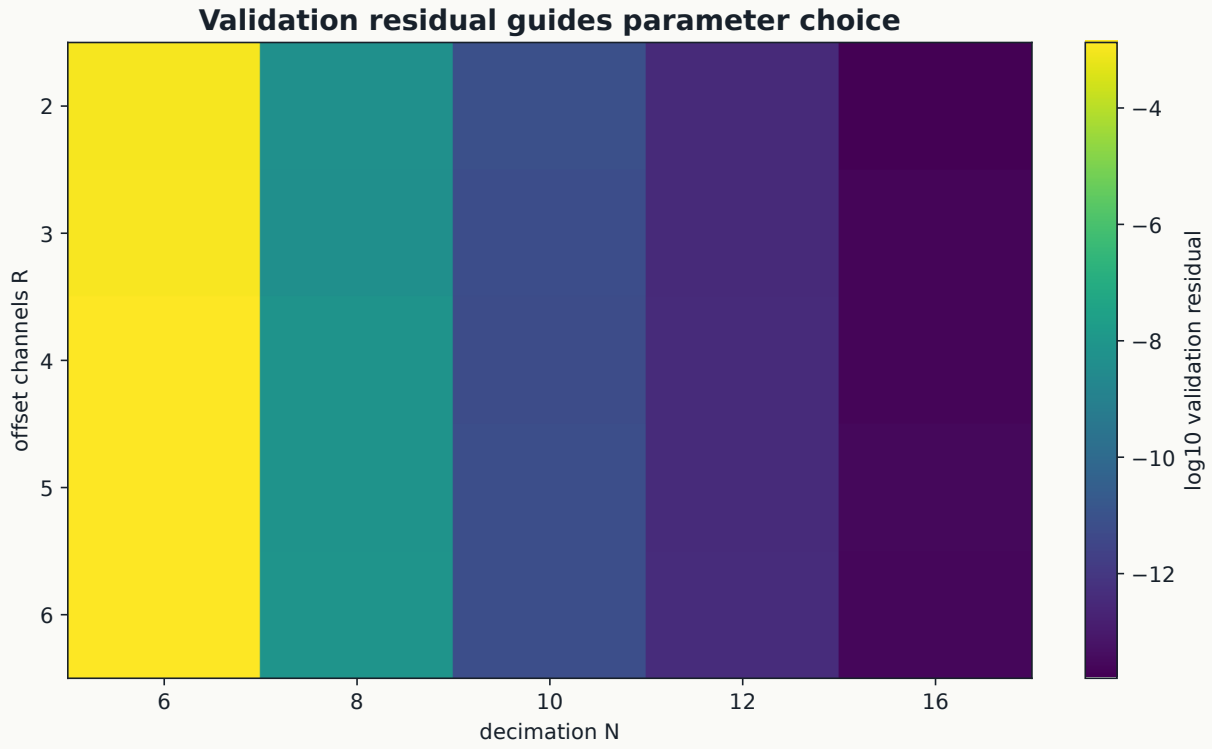


Figure 14: The validation residual can help choose a decimation and offset count before accepting a reconstruction.

where r is the root modulus.

Theorem 8.2 (Branch-margin criterion)**.** *Let $|z| = 1$ and $u = z^N$. Normalize a nonzero aliased estimate by*

$$\bar{u} = \frac{\widehat{u}}{|\widehat{u}|}.$$

Among the N roots of $\bar{u}$, let ρ be the one corresponding continuously to z. Let $\widetilde{z}$ be a coarse phase estimate. If

$$\boxed{|\widetilde{z} - z| + |\rho - z| < \sin\left(\frac{\pi}{N}\right),}$$

then the root of $\bar{u}$ nearest to $\widetilde{z}$ is ρ.

Proof. Every incorrect root is at least $2\sin(\pi/N)$ from ρ. The triangle inequality gives

$$|\widetilde{z} - \rho| \le |\widetilde{z} - z| + |z - \rho| < \sin(\pi/N).$$

Every other root is farther than $\sin(\pi/N)$ from $\widetilde{z}$. Thus ρ is the unique nearest root. □

Lemma 8.3 (One-order root gain)**.** *Let $|u| = 1$, let $\bar{u}$ be a unit-modulus perturbation, and let ρ be the corresponding branch of the Nth root. If*

$$|\bar{u} - u| \le \eta < 2,$$

then

$$\boxed{|\rho - z| \le 2\sin\left(\frac{1}{N}\arcsin\frac{\eta}{2}\right) \le \frac{2}{N}\arcsin\frac{\eta}{2}.}$$

Proof. The angular perturbation of u is at most $2\arcsin(\eta/2)$. Taking the corresponding Nth root divides that angular perturbation by N. Convert the result back to chord distance. □

9 Rate-preserving polyphase lifting

The central rate statement is now elementary once the decimated solver and offset coefficient estimates are controlled.

Theorem 9.1 (Rate-preserving lifting)**.** *Fix $p \ge 0$. Suppose a selected decimation N satisfies the following estimates for each edge:*

$$|\widehat{u} - u| \le C_u N^{-p-1},$$

$$\max_r \left|\widehat{b}_r - b_r\right| \le C_b N^{-p-1},$$

and

$$|b_0| \ge b_{\min} > 0.$$

Assume the branch-margin criterion holds for sufficiently large N. Then the lifted node estimate satisfies

$$\boxed{|\widehat{z} - z| = O(N^{-p-2}).}$$

The corresponding location error satisfies the same order.

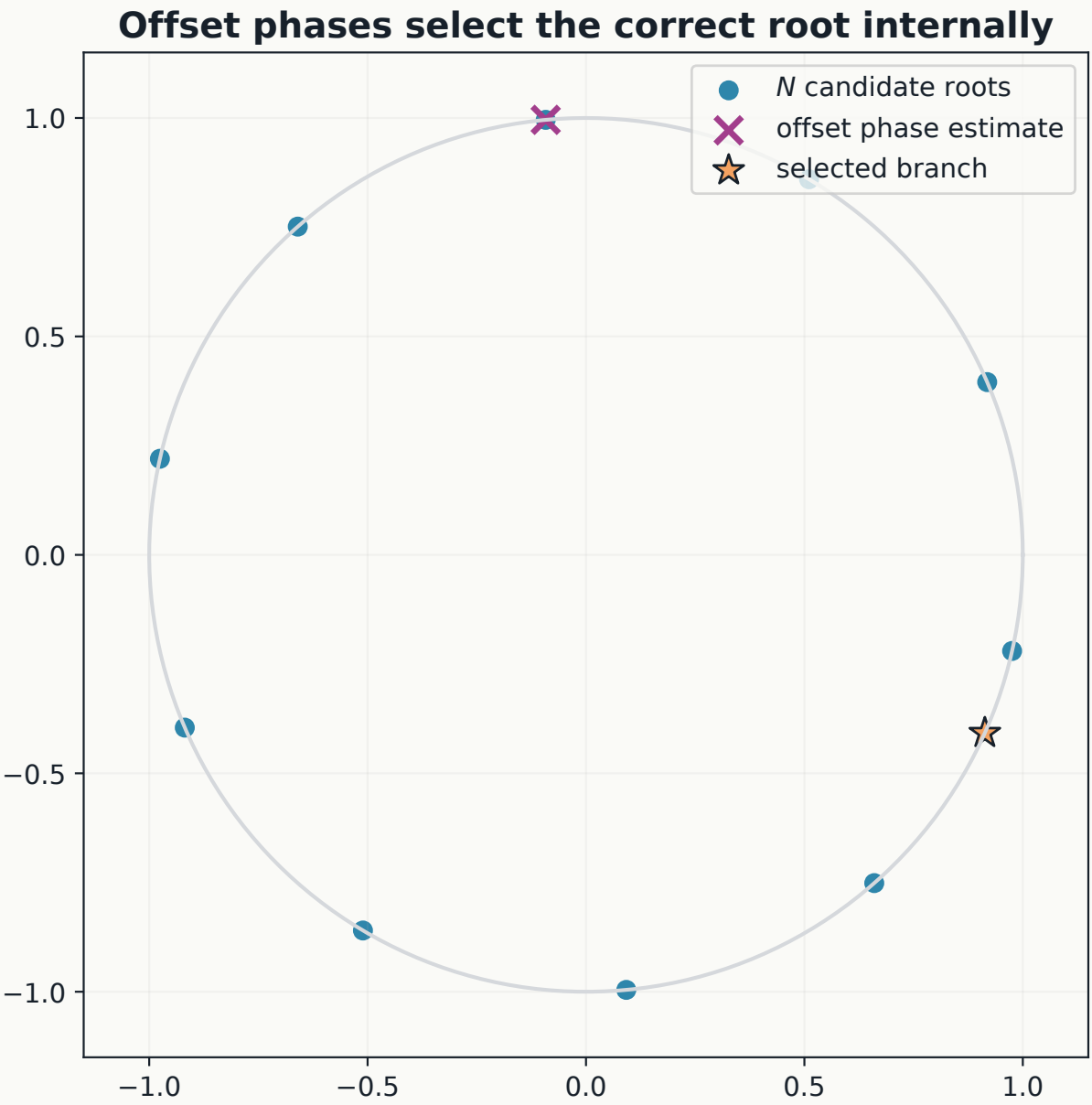


Figure 15: The offset estimate selects one root from the decimated candidate set.

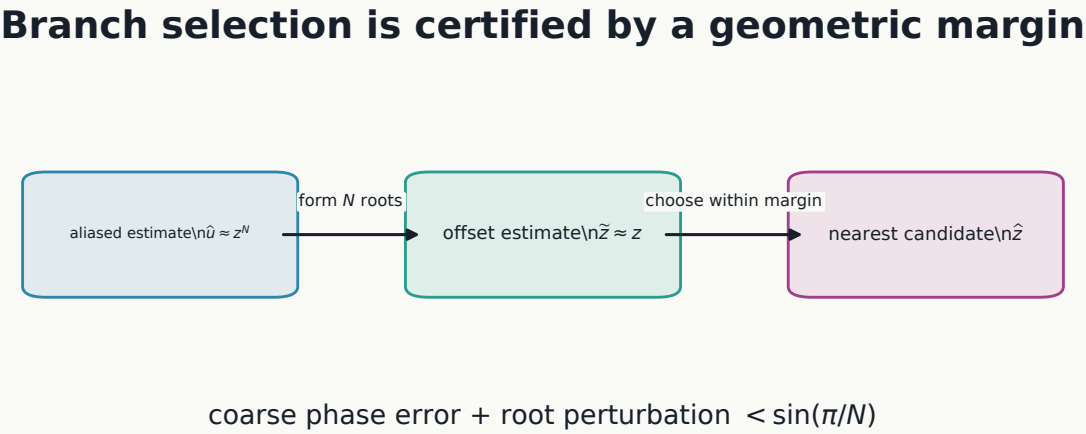


Figure 16: The sufficient branch condition separates coarse phase accuracy from aliased-root accuracy.

Proof. The phase perturbation theorem gives

$$|\widetilde{z} - z| = O(N^{-p-1}).$$

For $p \geq 0$, this is eventually smaller than the candidate spacing scale N^{-1} whenever the constants and selected decimation satisfy the branch margin. The correct branch is therefore selected. The one-order root lemma then gives

$$|\widehat{z} - z| \leq \frac{C}{N} |\widehat{u} - u| = O(N^{-p-2}).$$

On the unit circle, angular and chord errors are locally equivalent. □

Corollary 9.2 (Oracle-equivalent branch selection)**.** *Under the branch-margin hypothesis, the root selected by the offset phase is exactly the root that would be selected by an oracle supplying the correct jump neighborhood. Therefore the self-calibrated estimator inherits every location-error estimate proved for that oracle branch.*

Proof. The oracle and the offset estimator both select the unique candidate root lying inside the branch-margin neighborhood of the true phase. Uniqueness forces the two selected roots to coincide. □

This theorem is a lifting result. It does not replace the perturbation theory of the underlying decimated Prony method. It proves that an internally derived branch selector can preserve its rate without an externally supplied jump neighborhood.

Corollary 9.3 (Smooth-tail interpretation)**.** *Under the Fourier-tail assumption,*

$$|e_{qN+r}| = O(N^{-p-1})$$

for fixed q and r. On any compact admissible parameter class for which the decimated Prony map and its coefficient solve have uniformly bounded local inverses, the hypotheses of the rate-preserving lifting theorem hold.

Proof. The moment defect follows directly from

$$|e_n| = 2\pi\, |n|\, |\widehat{s}(n)|\,.$$

Distinct aliased nodes and nonzero amplitudes make the finite Prony Jacobian a full-rank confluent Vandermonde system. Uniform separation and amplitude bounds give a bounded local inverse on a compact admissible class. □

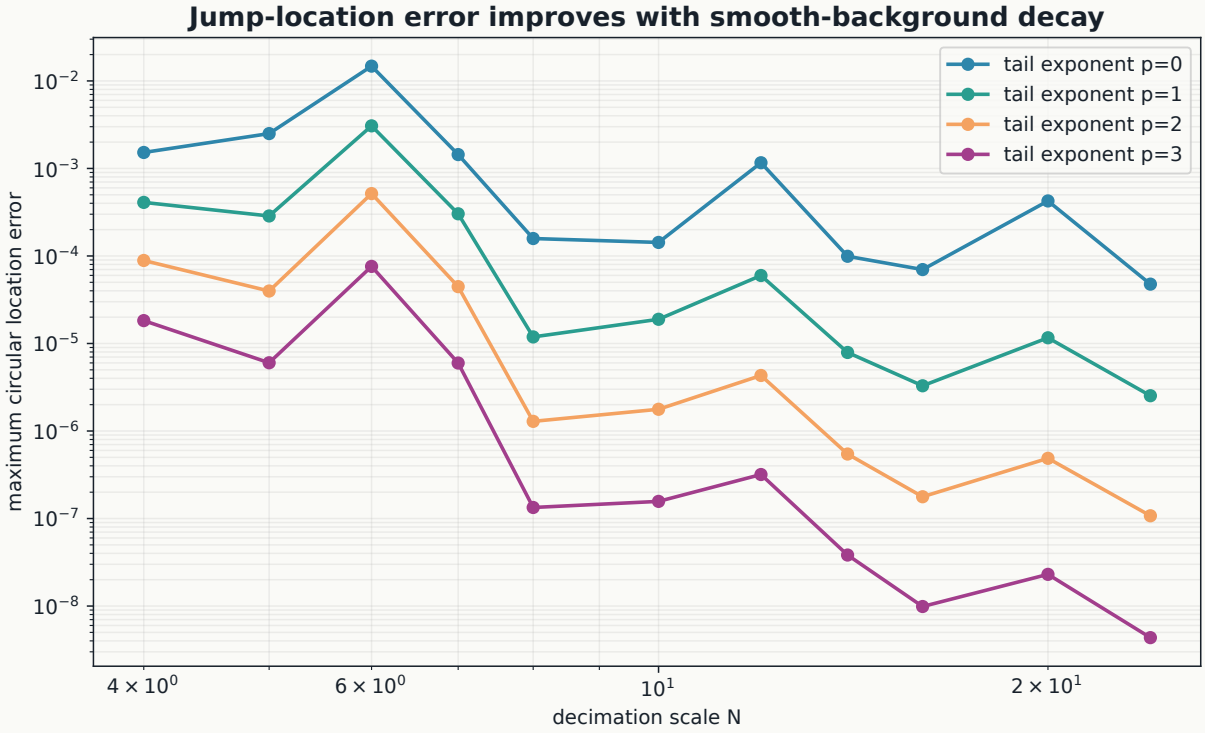


Figure 17: Deterministic power-tail backgrounds produce improving location errors as the decimation scale grows.

The oscillations in the rate curves are not hidden. Some decimations map distinct physical phases close together. A useful implementation therefore searches over several candidate decimations and selects one with a healthy rank gap and small validation residual.

10 Self-calibrating reconstruction theorem

We can now state the complete finite theorem.

Assumption 10.1 (Admissible edge class)**.** *The edge packets satisfy:*

1. $L \le L_{\max}$
2. *distinct locations with minimum circular separation* $\Delta > 0$
3. *jump amplitudes bounded by*
$$0 < a_{\min} \le |a_\ell| \le a_{\max}$$
4. *at least one tested decimation has distinct aliased nodes with a declared separation lower bound*

Theorem 10.2 (Exact self-calibration in the bandlimited control class)**.** *Suppose the smooth packet is bandlimited below the first moment index used by the algorithm. Assume exact arithmetic, an admissible edge packet, enough moments for the Hankel and coefficient systems, and one tested decimation with distinct aliased nodes. Then the procedure recovers:*

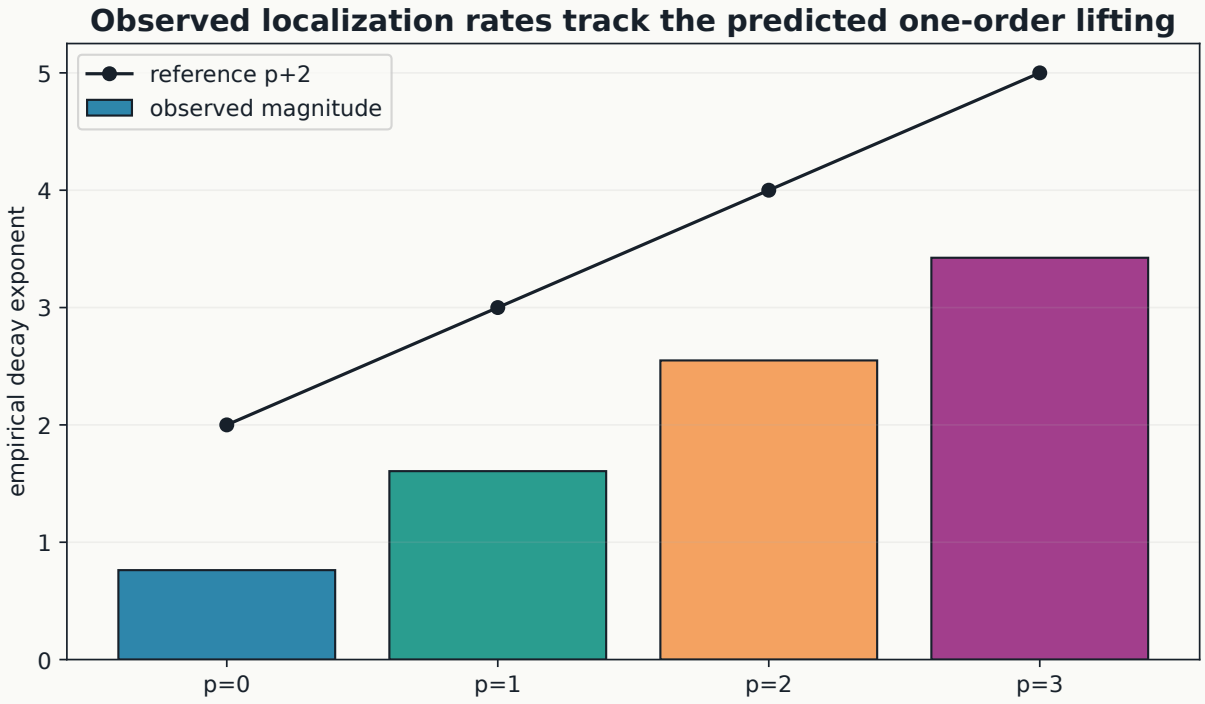


Figure 18: The empirical rate exponents track the $p+2$ prediction over the stable decimation subsequence.

1. *the exact jump count* L
2. *every aliased node* u_ℓ
3. *every physical phase* z_ℓ
4. *every jump location* τ_ℓ
5. *every jump amplitude* a_ℓ
6. *the exact edge packet* j
7. *the exact smooth packet* s

No externally supplied jump neighborhood is required.

Proof. The high derivative moments contain only the edge exponential sum. The Hankel rank is exactly L. Exact Prony recovery gives the distinct aliased nodes. Each offset Vandermonde solve gives the exact coefficient progression $b_{\ell,r} = b_{\ell,0} z_\ell^r$. The phase estimator returns z_ℓ. The branch lift therefore selects the correct physical root. A final real Vandermonde solve with the zero-sum constraint gives the jump amplitudes. Subtracting the recovered edge Fourier coefficients leaves the smooth coefficients. □

Corollary 10.3 (Explicit finite innovation budget)**.** *Assume the hypotheses of the exact self-calibration theorem and suppose the jump count satisfies* $L \le L_{\max}$. *Choose integers* $N \ge 2$ *and* $q_0 \ge 1$ *so that* $q_0 N$ *lies above the smooth bandwidth. Define*

$$\mathcal{I}_0 = \{(q_0 + q)N : 0 \le q \le 2L_{\max}\}$$

and

$$\mathcal{I}_1 = \{(q_0 + q)N + 1 : 0 \le q < L_{\max}\}.$$

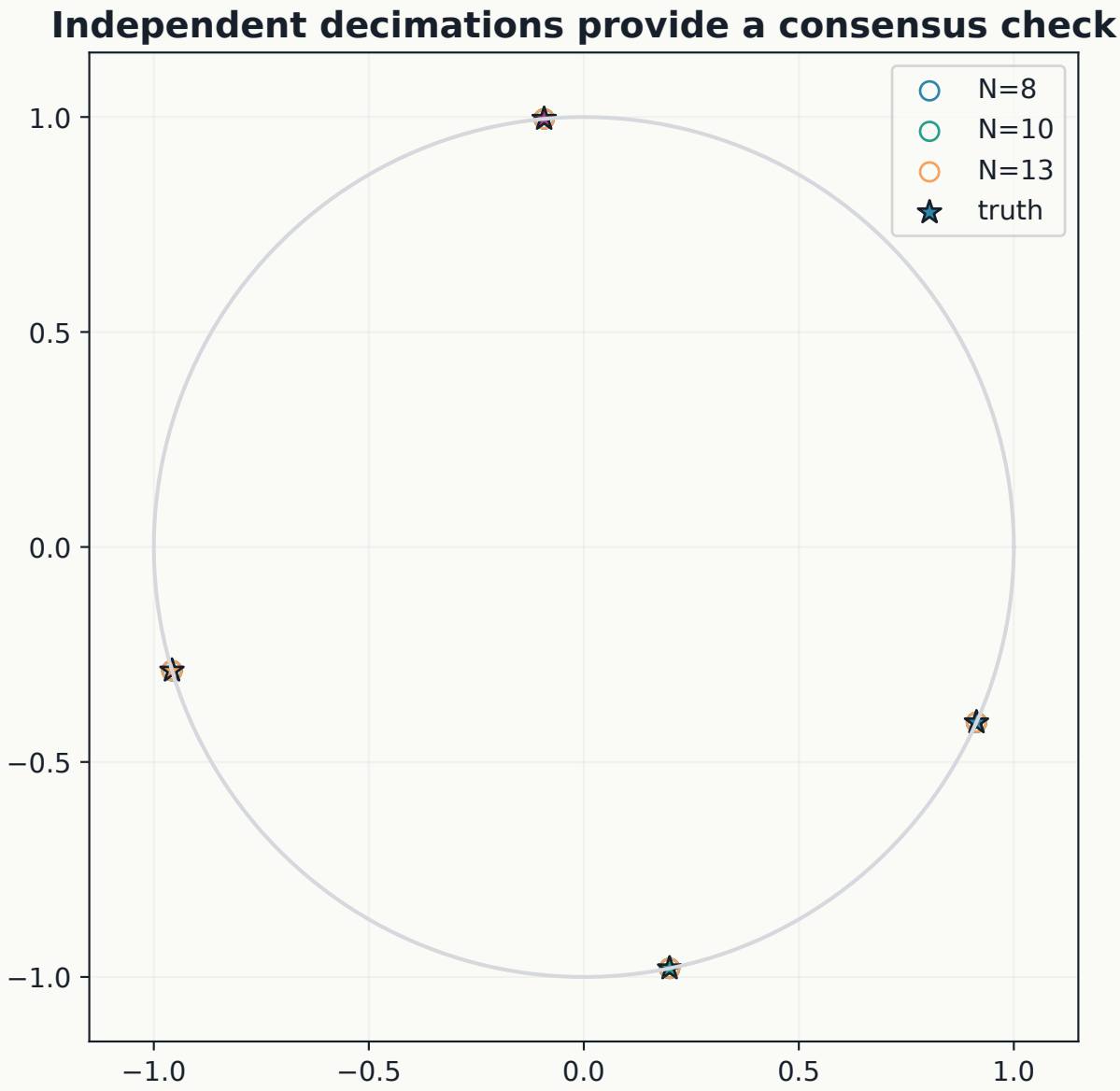


Figure 19: Independent decimations provide a direct consistency check on the recovered phases.

If the aliased nodes $u_\ell = z_\ell^N$ are distinct, then the derivative moments indexed by

$$\mathcal{I}_0 \cup \mathcal{I}_1$$

determine the unknown order, every physical phase, and every jump amplitude exactly. The high-frequency innovation budget is

$$\boxed{|\mathcal{I}_0| + |\mathcal{I}_1| = 3L_{\max} + 1.}$$

This count does not include the low Fourier coefficients used afterward to reconstruct the smooth packet.

Proof. The $2L_{\max} + 1$ base-channel moments form an $(L_{\max} + 1) \times (L_{\max} + 1)$ Hankel matrix of exact rank L, so the unknown order is recovered. Once L is known, the first $2L$ base moments determine the annihilating polynomial and hence the distinct aliased nodes u_ℓ. Solving the base Vandermonde system gives

$$b_{\ell,0} = a_\ell u_\ell^{q_0}.$$

The adjacent channel contains

$$m_{(q_0+q)N+1} = \sum_{\ell=1}^{L} b_{\ell,0} z_\ell u_\ell^q.$$

Its first $L_{\max}$ samples contain at least L equations. The Vandermonde solve therefore gives $b_{\ell,0} z_\ell$, and division by the nonzero base coefficient gives z_ℓ directly. Finally,

$$a_\ell = b_{\ell,0} u_\ell^{-q_0}.$$

The zero-mean normalization fixes the additive constant of the edge packet. Subtracting its Fourier coefficients leaves the smooth packet. □

Numerical certificate 10.4 (Nineteen-coefficient control example)**.** *For $L_{\max} = 6$, $N = 10$, and $q_0 = 1$, the implementation uses thirteen base-channel moments and six adjacent-channel moments. Thus it uses exactly*

$$3L_{\max} + 1 = 19$$

high-frequency Fourier coefficients. The unknown order is recovered as four. The maximum location error is

$$9.77 \times 10^{-15},$$

and the maximum jump-amplitude error is

$$5.78 \times 10^{-15}.$$

Theorem 10.5 (Perturbed self-calibration for a non-bandlimited background)**.** *Let the smooth packet satisfy the Fourier-tail assumption and let the edge packet belong to a compact admissible class. Suppose a tested decimation satisfies the singular-gap condition, the local inverse bounds for the decimated Prony and coefficient systems, and the branch-margin condition. Then, for sufficiently large decimation scale,*

$$\widehat{L} = L,$$
$$\max_\ell |\widehat{\tau}_\ell - \tau_\ell|_{\mathbb{T}} \le C_\tau N^{-p-2},$$
$$\max_\ell |\widehat{a}_\ell - a_\ell| \le C_a N^{-p-1}.$$

The constants depend on the smooth-tail bound, separation, amplitude bounds, selected sample pattern, and local inverse constants.

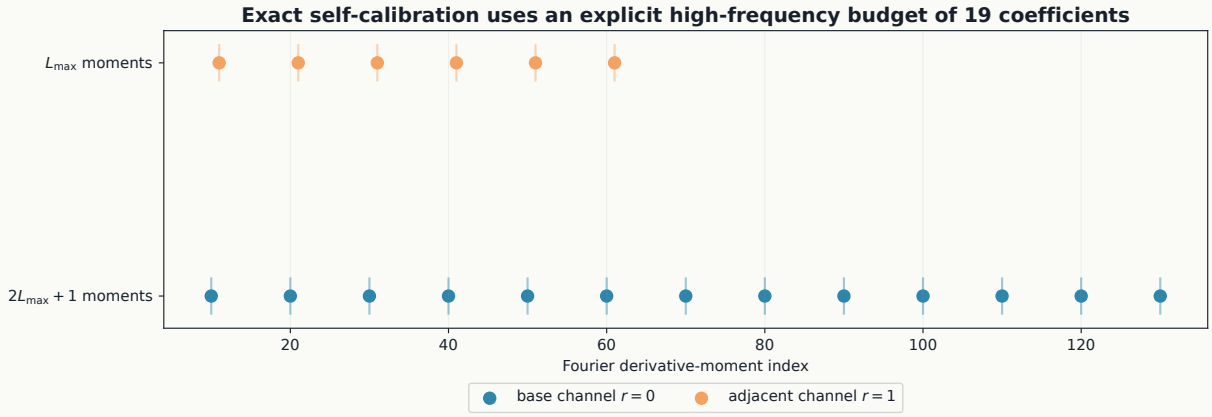


Figure 20: The exact bandlimited control theorem has an explicit finite innovation budget. The count is an upper bound for this construction, not a minimality claim.

Proof. The smooth moment defect is $O(N^{-p-1})$. The singular-gap theorem preserves the order. The local inverse bounds transfer this perturbation to the aliased nodes and offset coefficients. The rate-preserving lifting theorem gives the location estimate. The amplitude estimate follows from the uniformly conditioned real Vandermonde solve. □

Recovered edge parameters

edge	true location	recovered location	true jump	recovered jump
1	0.420000000000	0.420000000000	1.00000000	1.00000000
2	1.370000000000	1.370000000000	-0.65000000	-0.65000000
3	2.850000000000	2.850000000000	0.40000000	0.40000000
4	4.620000000000	4.620000000000	-0.75000000	-0.75000000

Figure 21: The analytic-background example recovers the four locations and amplitudes to displayed precision.

11 A stable synthesis theorem

Edge recovery alone does not explain the removal of Gibbs oscillations. The reconstruction operator matters.

Let A_M be a reconstruction operator for the smooth class. It may be a Fourier partial sum when the edge is exact and the smooth packet is sufficiently regular. For a norm-stable theorem, Fejér means or another uniformly bounded smooth-class operator are convenient.

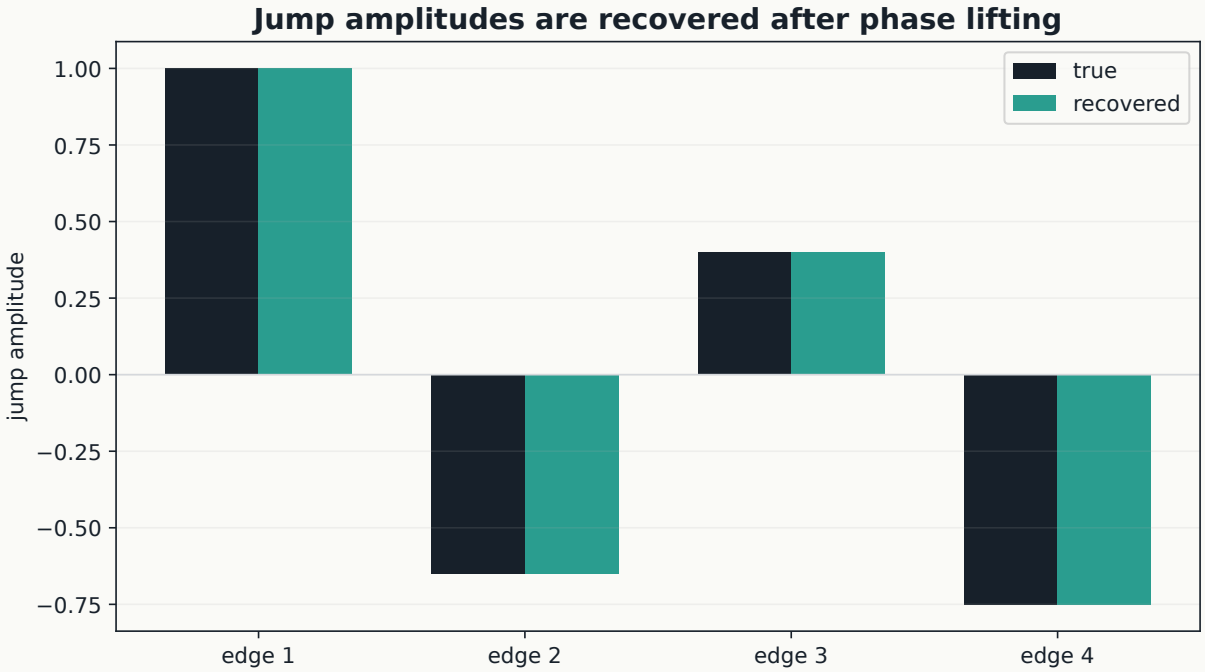


Figure 22: Recovered and true jump amplitudes.

Given an edge estimate $\widehat{j}$, define

$$\boxed{\widehat{f}_M = A_M(f - \widehat{j}) + \widehat{j}.}$$

Theorem 11.1 (Resolved synthesis inequality)**.** *If A_M is bounded on a normed signal space with*

$$\|A_M\| \le C_A,$$

then

$$\boxed{\left\|f - \widehat{f}_M\right\| \le \|s - A_M s\| + (1 + C_A)\left\|j - \widehat{j}\right\|.}$$

Proof. Using $f = s + j$,

$$\begin{aligned} f - \widehat{f}_M &= s + j - A_M(s + j - \widehat{j}) - \widehat{j} \\ &= (s - A_M s) + (I - A_M)(j - \widehat{j}). \end{aligned}$$

Apply the triangle inequality and the operator bound. □

Corollary 11.2 (Exact edge packet)**.** *If $\widehat{j} = j$, then*

$$\boxed{\left\|f - \widehat{f}_M\right\| = \|s - A_M s\|.}$$

No jump packet is sent through the smooth global approximation.

This is the precise Gibbs-free statement. The classical partial sum of the full signal still has Gibbs oscillations. The resolved synthesis does not create them because the singular packet is reconstructed in its native coordinate.

For periodic piecewise-constant packets, a location and amplitude perturbation yields an L^1 estimate. After matching the jumps,

$$\left\|j - \widehat{j}\right\|_{L^1} \le 2\pi \sum_{\ell} |a_\ell - \widehat{a}_\ell| + \sum_{\ell} |\widehat{a}_\ell|\,|\tau_\ell - \widehat{\tau}_\ell|_{\mathbb{T}}.$$

Combining this with the recovery rates gives an explicit global reconstruction rate in L^1.

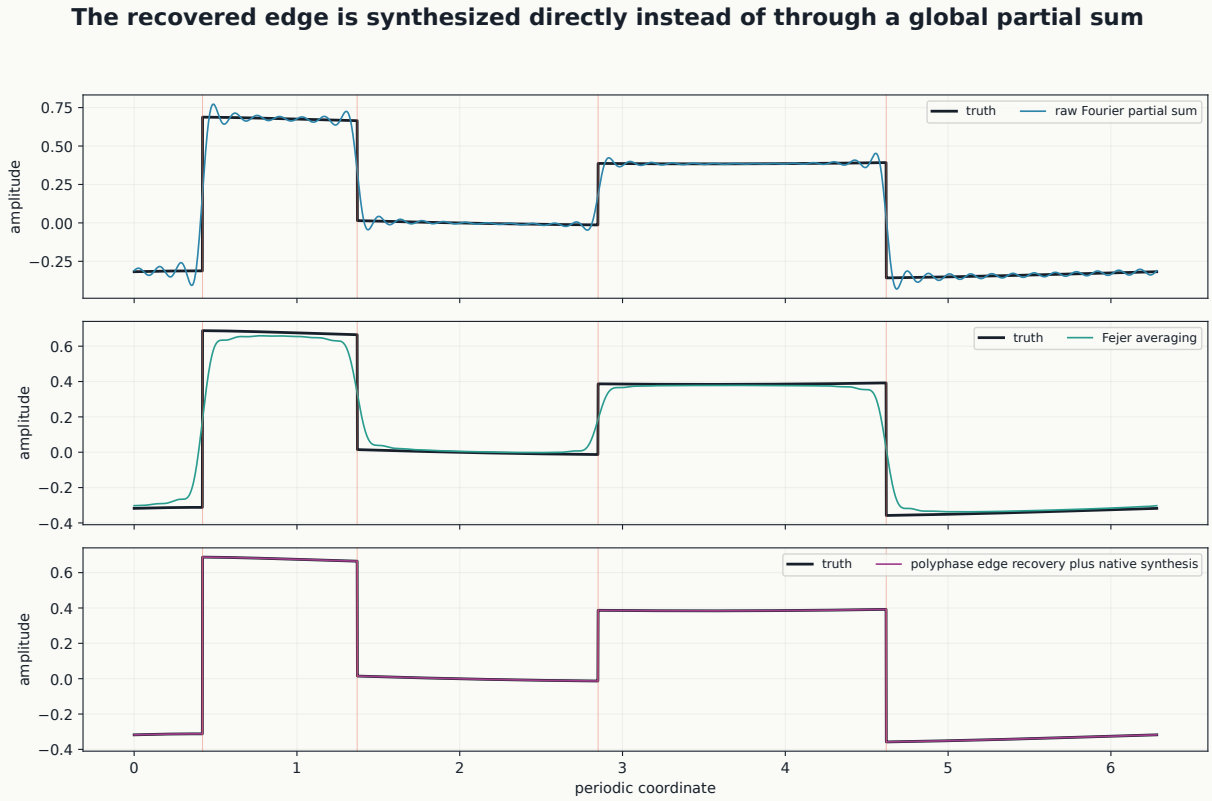


Figure 23: Raw Fourier and Fejér reconstructions are compared with edge-native synthesis.

12 Worked example from Fourier data to reconstruction

The edge packet has locations

$$(0.42,\ 1.37,\ 2.85,\ 4.62)$$

and jumps

$$(1,\ -0.65,\ 0.40,\ -0.75).$$

The smooth packet is

$$s(x) = 0.35e^{0.7\cos x} + 0.2\sin(2x) - 0.15\cos(5x).$$

Its Fourier coefficients are known exactly through modified Bessel functions, and they do not terminate.

The algorithm uses decimation $N = 10$, six offset channels, and twenty-two samples per channel. The Hankel gap identifies four active jumps. The offset recurrence residual is approximately

$$1.04 \times 10^{-11}.$$

After nonlinear refinement, the recovered locations are

$$0.4200000000000002,\quad 1.3700000000000003,\quad 2.8500000000000014,\quad 4.6200000000000000.$$

The recovered amplitudes differ from the true amplitudes by less than 10^{-12} in this deterministic example.

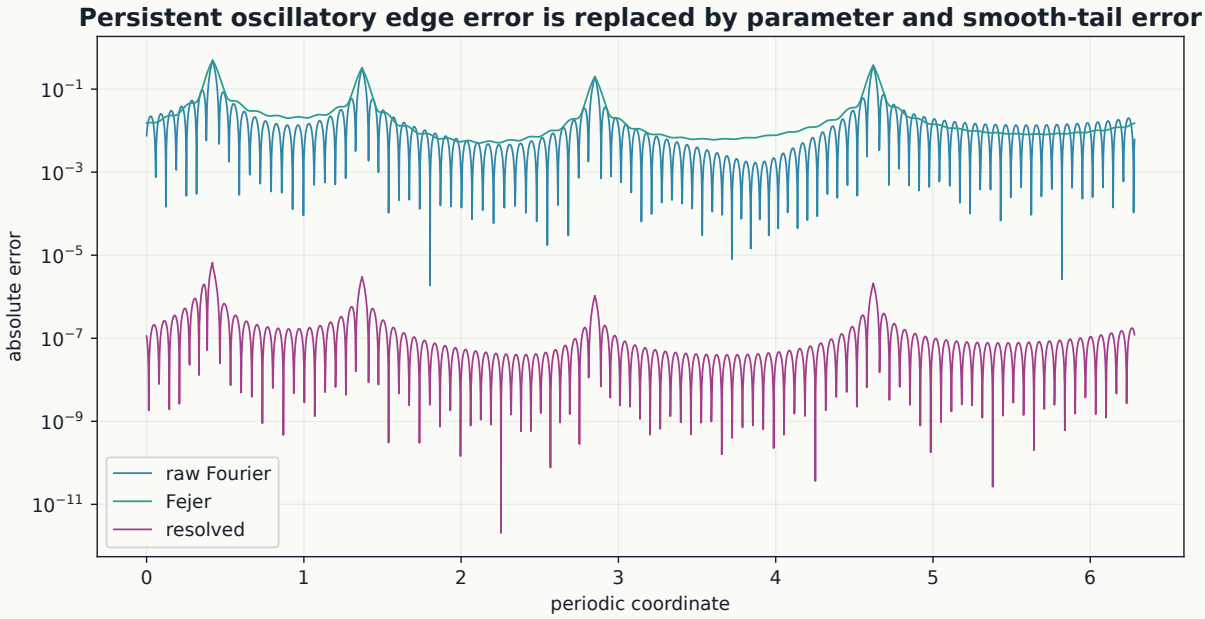


Figure 24: The resolved method replaces persistent oscillatory edge error with the much smaller parameter and smooth-tail error.

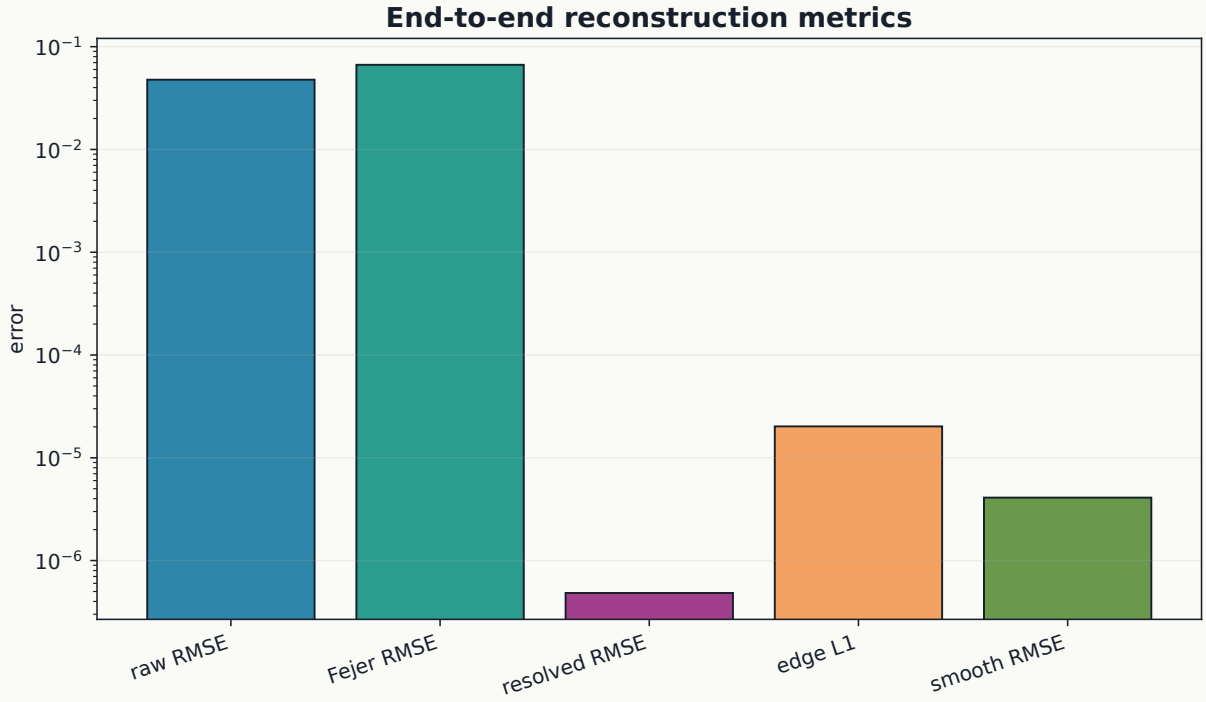


Figure 25: End-to-end errors for the main non-bandlimited experiment.

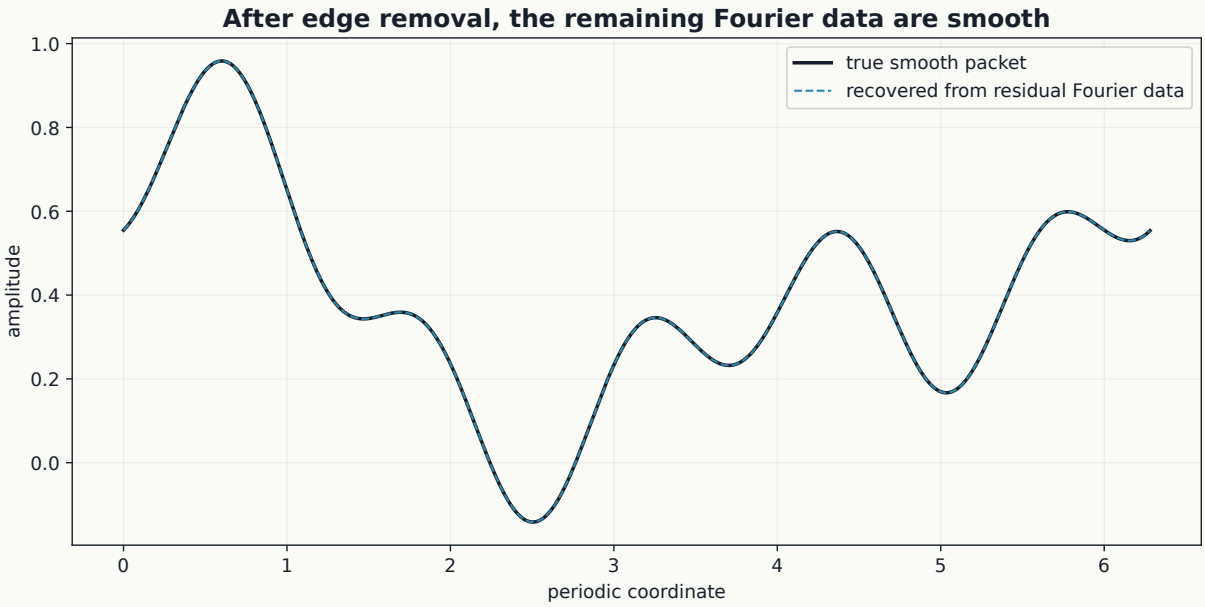


Figure 26: Subtracting the recovered edge coefficients reveals the smooth packet.

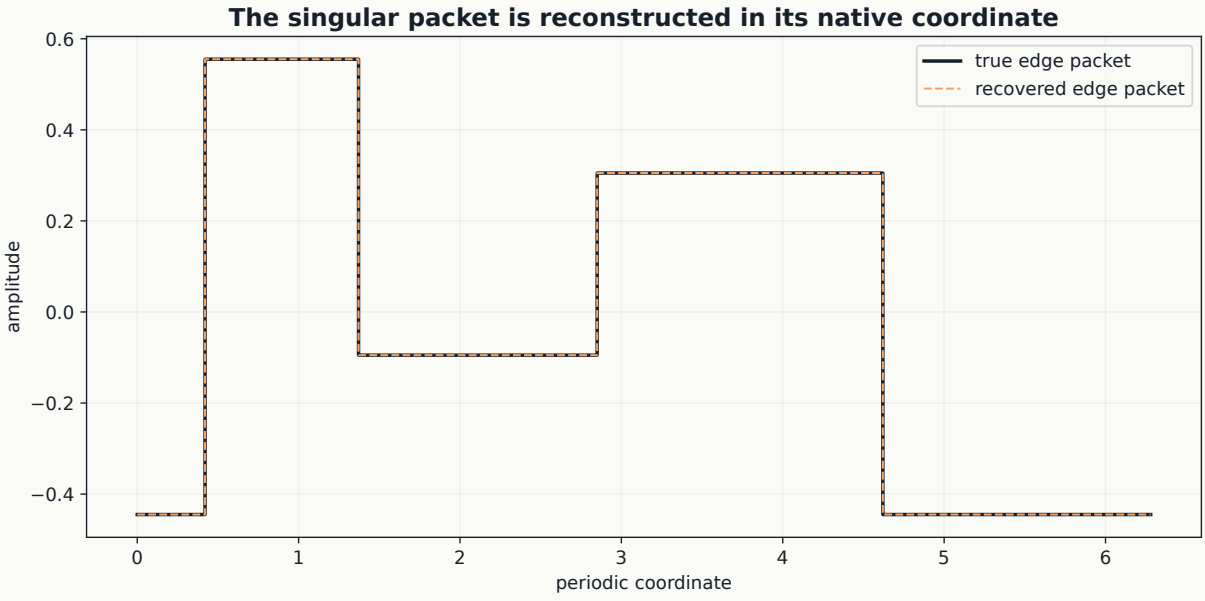


Figure 27: The recovered singular packet is returned by a piecewise-constant synthesis map.

A second example uses a power-law smooth tail rather than an analytic one. With forty-eight retained smooth Fourier modes, the measured errors were

quantity	value
raw Fourier RMSE	4.7716×10^{-2}
Fejér RMSE	6.6610×10^{-2}
resolved RMSE	4.8374×10^{-7}
maximum location error	6.4866×10^{-9}
maximum jump error	1.2407×10^{-5}
edge L^1 error	2.0201×10^{-5}
smooth packet RMSE	4.0952×10^{-6}

13 Direct oracle comparison and random-family stress test

The decisive comparison is not between the self-calibrated method and a deliberately weak smoother. It is between the self-calibrated branch and the branch chosen by an oracle that already knows the correct jump neighborhood.

For each aliased estimate $\widehat{u}_j$, define three lifts.

1. The *principal-root baseline* takes the principal Nth root and ignores every offset channel.
2. The *oracle lift* chooses the candidate root nearest the true phase. This is used only for evaluation.
3. The *self-calibrated lift* chooses the candidate root nearest the phase obtained from the offset coefficients.

The oracle-equivalence corollary predicts that the last two methods choose the same branch whenever the margin condition holds. The complete deterministic benchmark covers four smooth-tail indices and seven decimation scales. Across all twenty-eight combinations, the maximum distance between the oracle and self-calibrated node sets was

$$\boxed{2.22 \times 10^{-16}.}$$

The principal-root baseline produced order-one angular errors because the physical phases were not confined to its preferred branch.

N	principal root	oracle branch	self-calibrated branch
6	2.4303	5.1590×10^{-4}	5.1590×10^{-4}
8	3.0508	1.2892×10^{-6}	1.2892×10^{-6}
10	2.7366	1.7717×10^{-6}	1.7717×10^{-6}
12	2.9424	4.3100×10^{-6}	4.3100×10^{-6}
16	2.7489	1.7739×10^{-7}	1.7739×10^{-7}
20	2.8274	4.8757×10^{-7}	4.8757×10^{-7}
24	2.9424	1.0754×10^{-7}	1.0754×10^{-7}

The displayed rows use the $p = 2$ power-tail background. The same oracle agreement holds for all four tested tail indices.

A second benchmark removes the fixed four-edge geometry. One hundred deterministic random cases were generated with:

- an unknown jump count between two and five

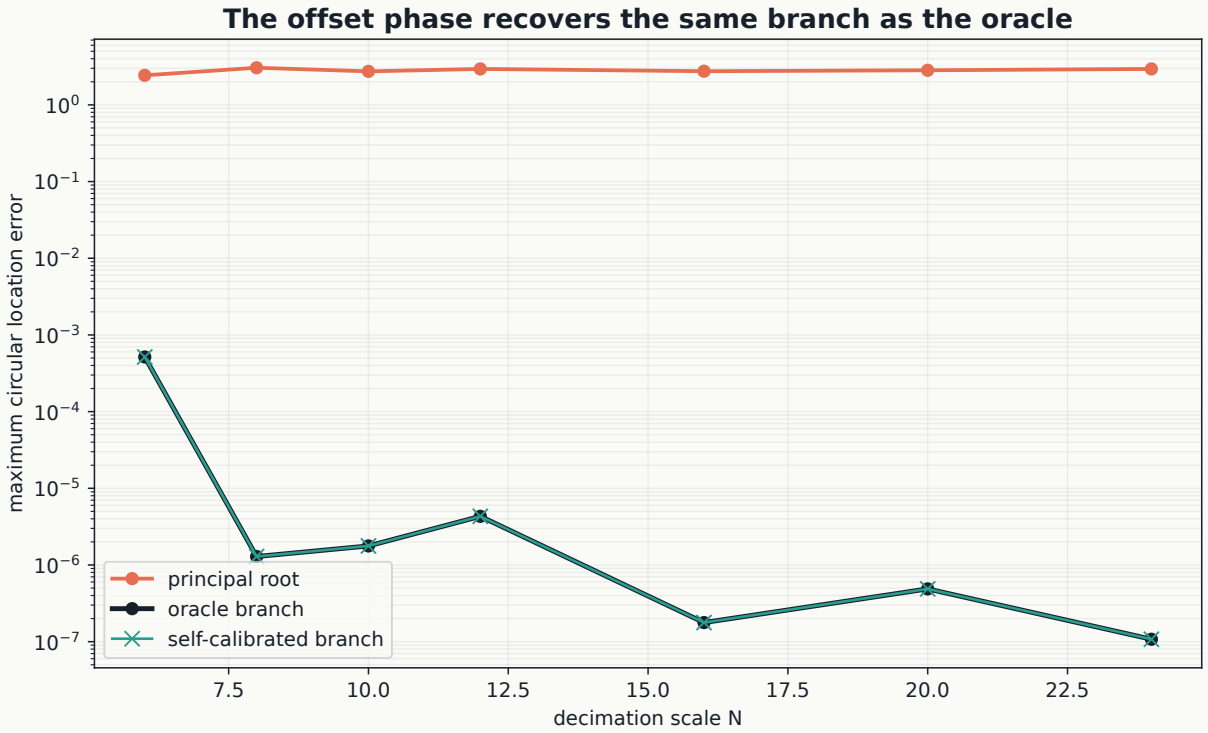


Figure 28: The self-calibrated lift follows the oracle branch while the principal root fails.

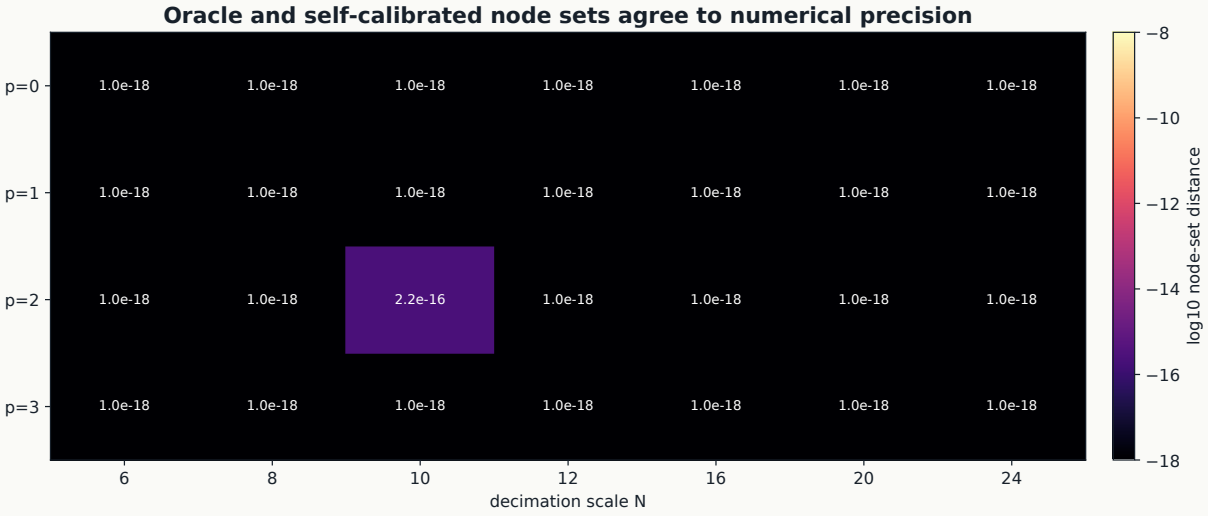


Figure 29: The oracle and self-calibrated node sets coincide to floating-point precision throughout the benchmark grid.

- minimum physical separation at least 0.45
- minimum separation after the tested decimation at least 0.35
- nonzero jump magnitudes bounded away from zero
- one of four unknown power-law smooth tails

The algorithm was not told the true jump count, locations, amplitudes, or tail index. It recovered the correct order in all one hundred cases. Every maximum location error was below 10^{-3}. The median error was

$$1.19 \times 10^{-6},$$

and the largest was

$$2.84 \times 10^{-4}.$$

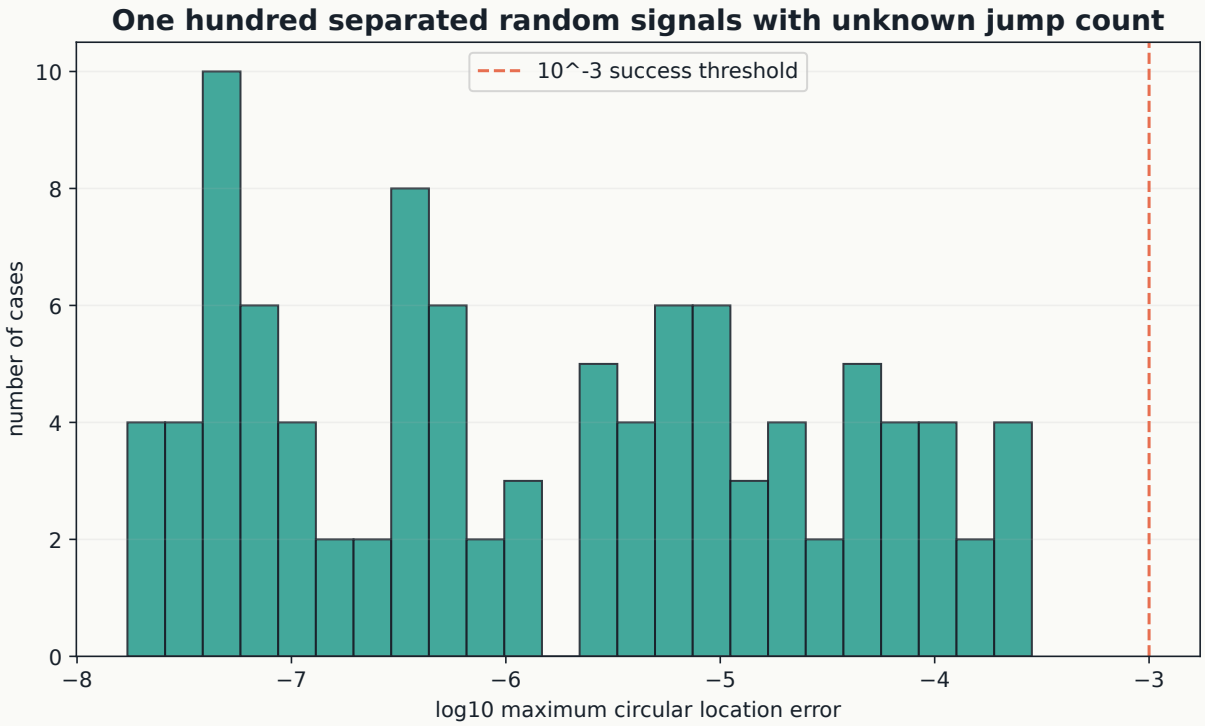


Figure 30: Location-error distribution over one hundred separated random signals.

Boundary 13.1 (Meaning of the stress test)**.** *The one-hundred-case result is a finite numerical certificate, not a universal probability theorem. The generation conditions enforce the same physical and aliased separation hypotheses used by the analysis. Its role is to test the full unknown-order and unknown-location pipeline away from one hand-selected geometry.*

14 Noise, order selection, and the branch margin

Moment noise was added independently at five scales. For the first four scales, the estimated jump count was correct in all thirty runs. Median location error grew linearly with the injected moment-noise scale over the displayed range.

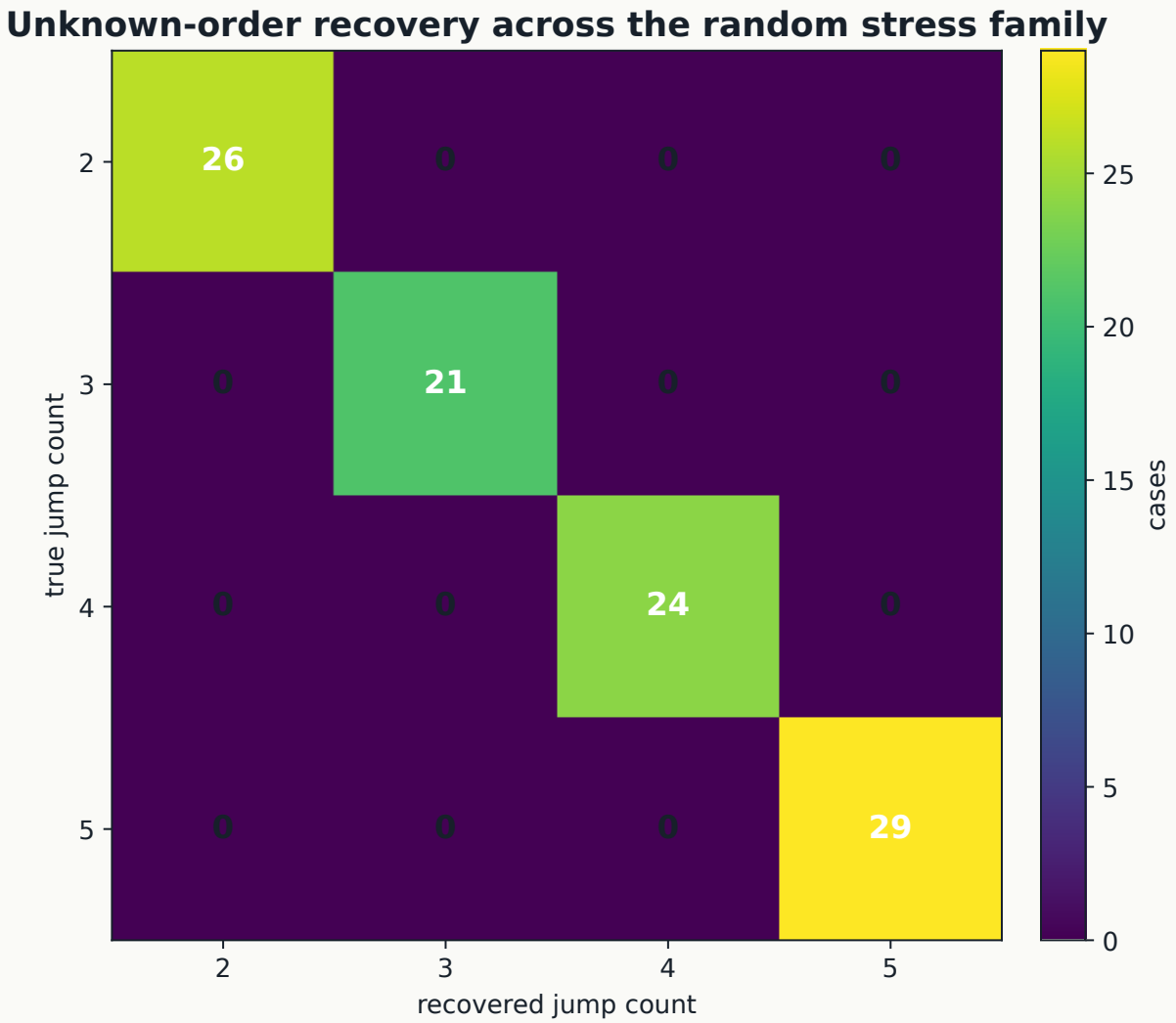


Figure 31: The recovered model order equals the true jump count throughout the random stress family.

σ	median location error	maximum location error	order success
10^{-10}	6.56×10^{-13}	1.67×10^{-12}	1.00
10^{-8}	6.19×10^{-11}	1.39×10^{-10}	1.00
10^{-6}	6.22×10^{-9}	1.39×10^{-8}	1.00
10^{-5}	6.22×10^{-8}	1.39×10^{-7}	1.00
10^{-4}	6.22×10^{-7}	1.39×10^{-6}	1.00

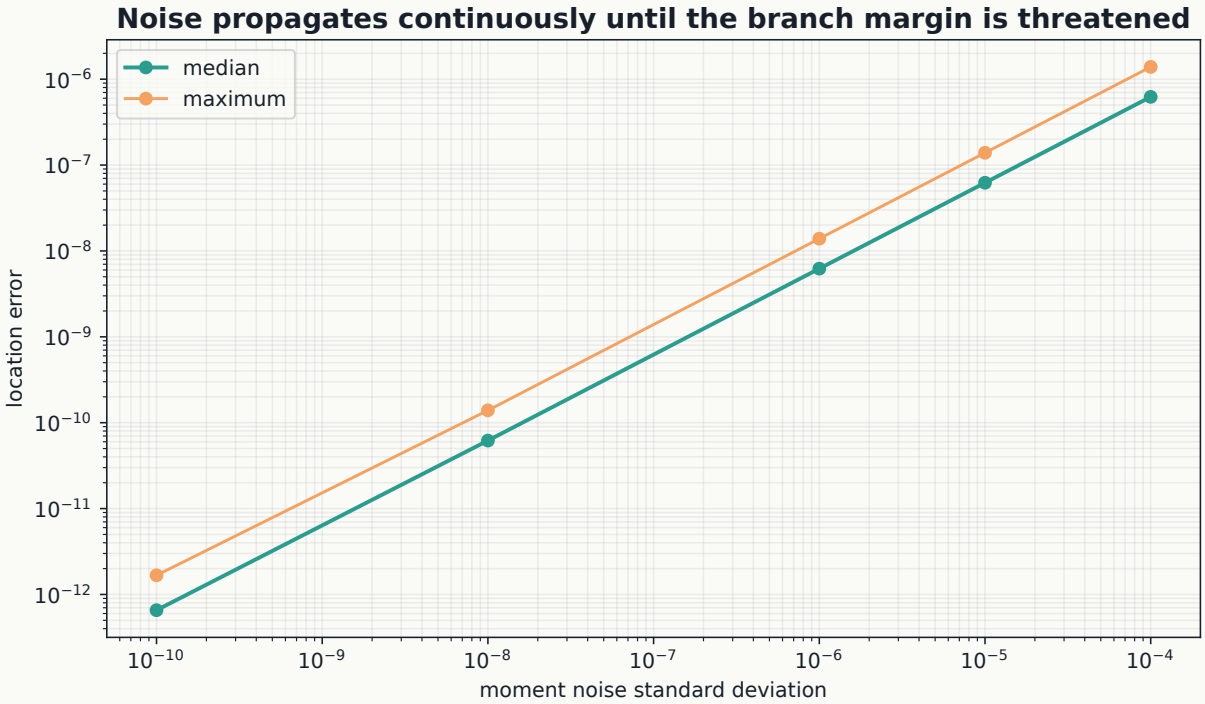


Figure 32: Location error under additive complex moment noise.

The method is not stable without separation. Near collisions cause the Prony Jacobian and the Hankel gap to deteriorate. Decimation can improve or worsen a particular pair because it maps angular gaps modulo 2π. This is why the implementation tests several decimations and records the validation residual rather than fixing one scale blindly.

15 Higher jump jets and confluent Prony systems

The phase-lifting idea is not limited to piecewise-constant packets. Suppose a piecewise-smooth function has jump data through derivative order d. After multiplying its Fourier coefficients by $(in)^{d+1}$, the singular moments have the form

$$\boxed{m_n = \sum_{\ell=1}^{L} z_\ell^n P_\ell(n) + e_n,}$$

where each P_ℓ is a polynomial of degree at most d.

On the offset channel $n = (q_0 + q)N + r$,

$$P_\ell((q_0 + q)N + r) = \sum_{h=0}^{d} \beta_{\ell,r,h} q^h.$$

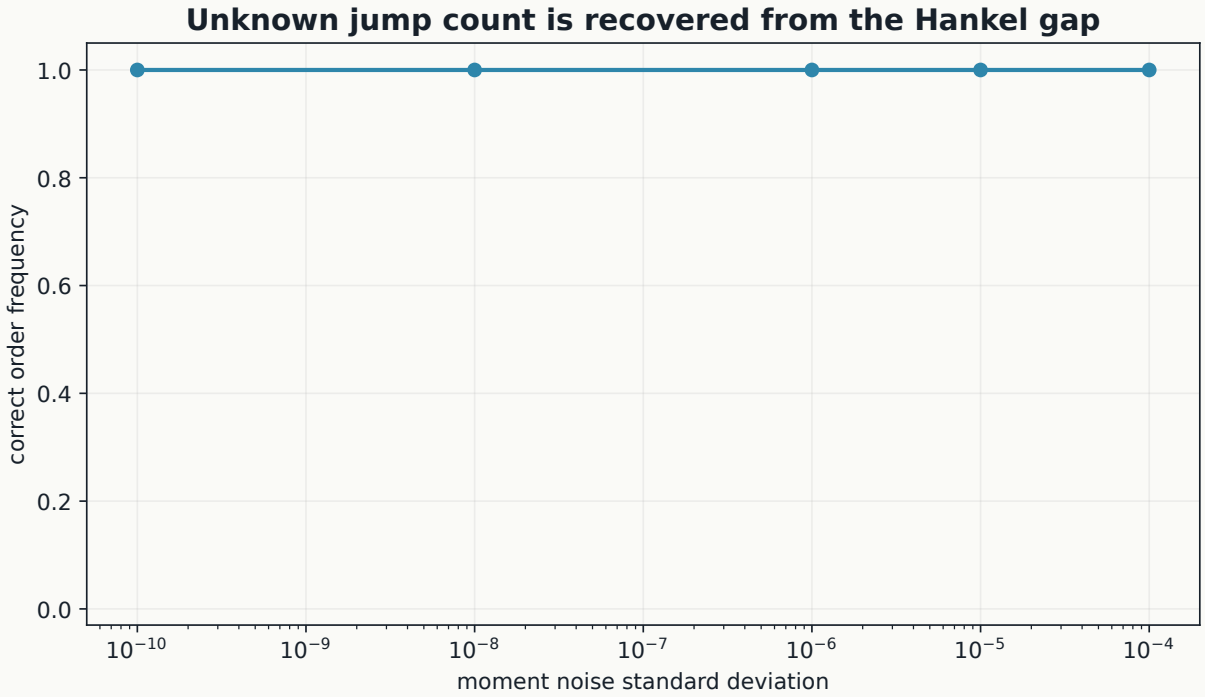


Figure 33: The jump count remains stable while the Hankel gap stays visible.

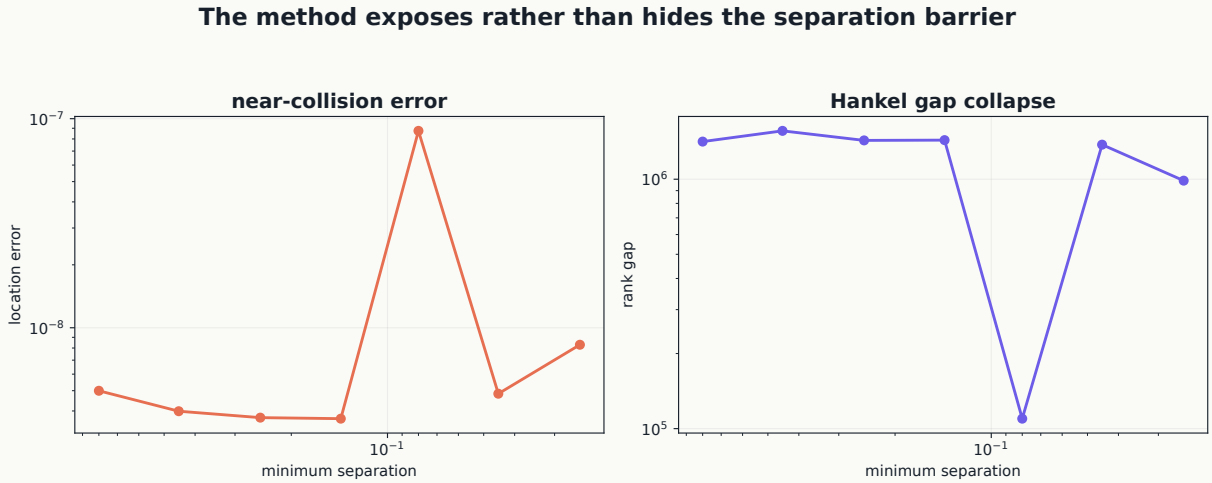


Figure 34: Near-collision diagnostics. The nonmonotone pattern reflects alias geometry at the selected decimation.

Thus

$$m_{(q_0+q)N+r} = \sum_{\ell=1}^{L} u_\ell^q \sum_{h=0}^{d} \gamma_{\ell,r,h} q^h + e_{(q_0+q)N+r}.$$

This is a confluent Prony system. If the function jump $a_{0,\ell}$ is nonzero, the leading coefficient satisfies

$$\gamma_{\ell,r,d} = C_\ell z_\ell^r$$

after the factors independent of r are collected. Hence

$$\boxed{\frac{\gamma_{\ell,1,d}}{\gamma_{\ell,0,d}} = z_\ell.}$$

Proposition 15.1 (Confluent offset phase)**.** *For a jump jet with a nonzero leading singular coefficient, the physical edge phase is the ratio of corresponding highest-degree confluent coefficients in two adjacent offset channels.*

Proof. The highest power of q arises from the leading term of $P_\ell((q_0+q)N+r)$. Every factor other than z_ℓ^r is independent of the offset. Taking the ratio cancels the aliased node, amplitude, decimation power, and initial-block factor. □

The exact symbolic check in the code uses two nodes and degree-one polynomial amplitudes. It obtains the ratios 1 and i, followed by the validation identities

$$\gamma_2\gamma_0 - \gamma_1^2 = 0$$

for both nodes.

16 Computational complexity

For fixed edge count L, offset count R, and samples per offset Q, the principal costs are:

- one Hankel singular-value decomposition of size proportional to Q
- one reduced eigenproblem of size L
- R Vandermonde least-squares solves
- one optional nonlinear refinement over $2L-1$ real parameters

The small parametric solve is separate from the cost of evaluating or storing the full reconstructed signal. In the supplied implementation, the median time for the linear phase-lifting stage grew from roughly two milliseconds at twelve samples per offset to roughly seven milliseconds at forty-eight samples per offset on the build environment.

17 What would count as a field-level advance

The result is stronger than the earlier known-edge packet theorem and stronger than exact recovery restricted to a bandlimited smooth background. It also does more than place a new name on shifted Prony samples. The substantive theorem is the following combination:

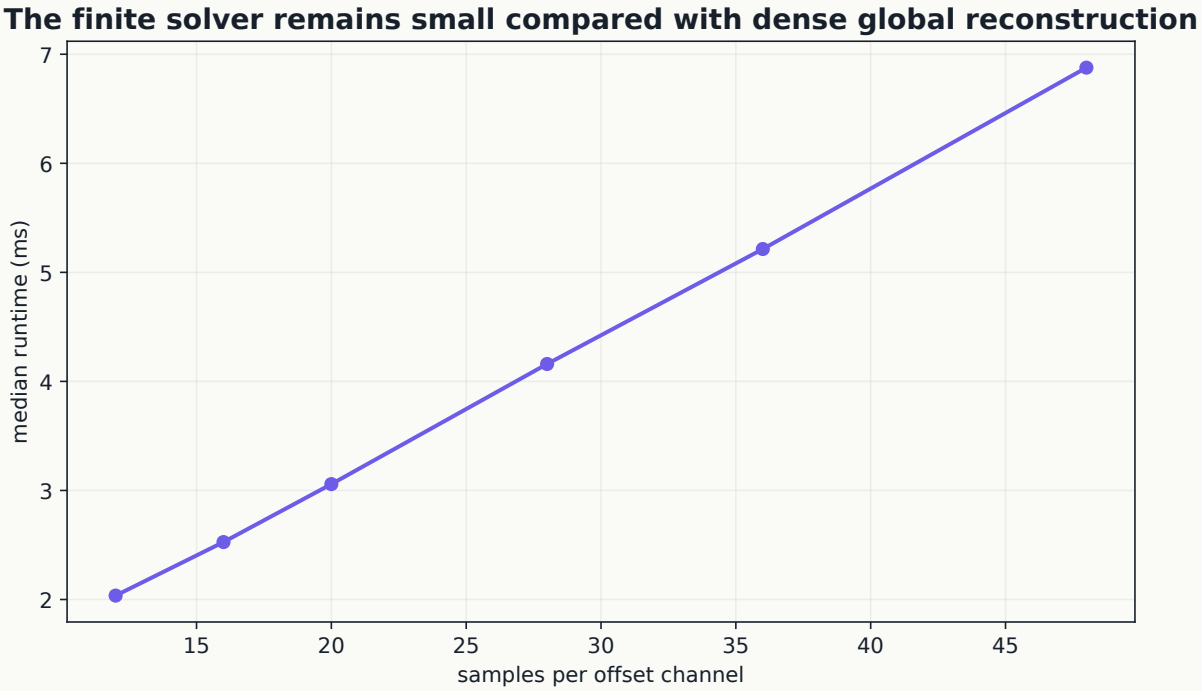


Figure 35: Measured runtime of the finite parametric stage.

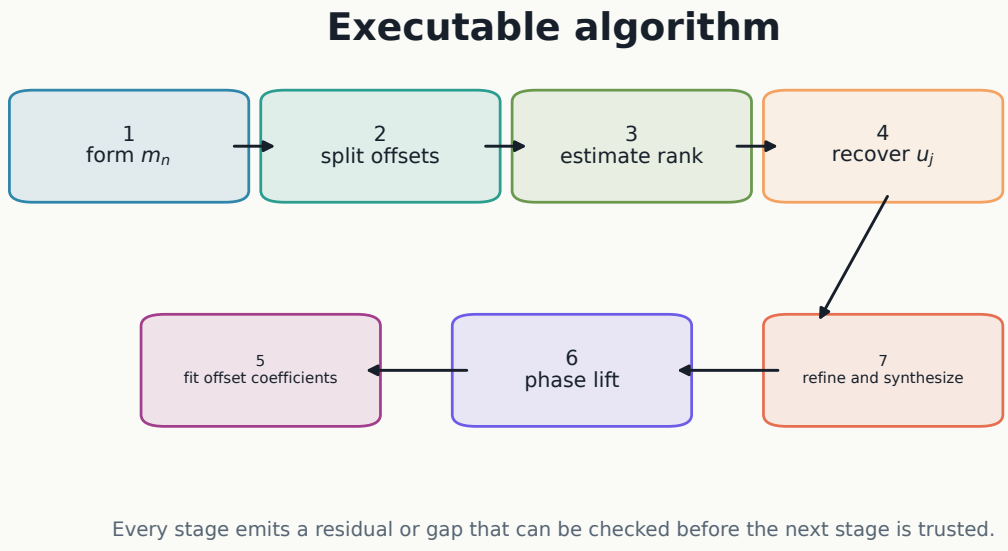


Figure 36: The executable algorithm and the diagnostic object emitted by each stage.

1. the jump count is inferred from the same Fourier data
2. decimation is retained for conditioning and rate
3. offset channels select the physical branch internally
4. the branch lift preserves the one-order location gain
5. unused offsets provide a validation residual
6. the self-calibrated branch is proved to equal the oracle branch under the margin condition
7. the recovered edge is synthesized outside the smooth global approximation

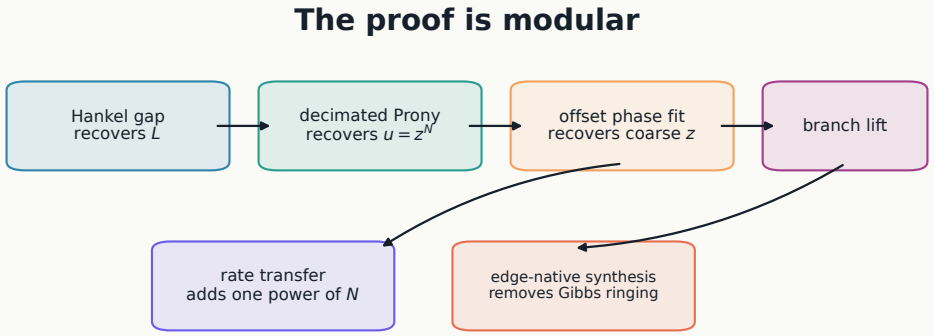


Figure 37: The theorem is assembled from rank recovery, decimated inversion, phase lifting, and resolved synthesis.

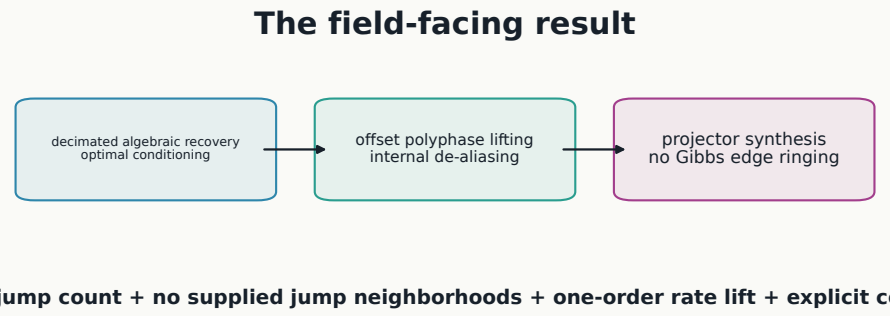


Figure 38: The field-facing statement in one diagram.

Whether this exact package is historically first is not established by the mathematics alone. The paper therefore makes a narrower priority statement. Shifted-sample de-aliasing, Prony inversion, decimation, finite-rate innovation, and Gibbs-resolution methods are acknowledged as prior inputs. The theorem proved here closes a specific gap in decimated Fourier edge reconstruction. It replaces an externally supplied branch neighborhood by a phase recovered from the Fourier data, proves equality with the oracle branch under a checkable margin, preserves the decimated localization rate, detects the unknown jump count, and completes the reconstruction by native edge synthesis.

18 Limitations and next theorem

Boundary 18.1 (Current scope)**.** *The present theorem does not cover arbitrary two-dimensional curves, unbounded jump counts, separation-free recovery, general blind measurement operators, or arbitrary smoothness classes without a quantitative tail model.*

The next mathematically meaningful extension is two-dimensional off-grid curve recovery with the same four properties:

1. unknown model order or curve complexity
2. internal de-aliasing across polyphase frequency blocks
3. a deterministic validation residual
4. native geometric synthesis after recovery

A second direction is minimax statistical analysis. The deterministic bounds identify the relevant quantities: singular-value gaps, aliased-node separation, offset coefficient strength, branch margin, and smooth-tail size. A complete statistical theorem should express the recovery risk in those coordinates.

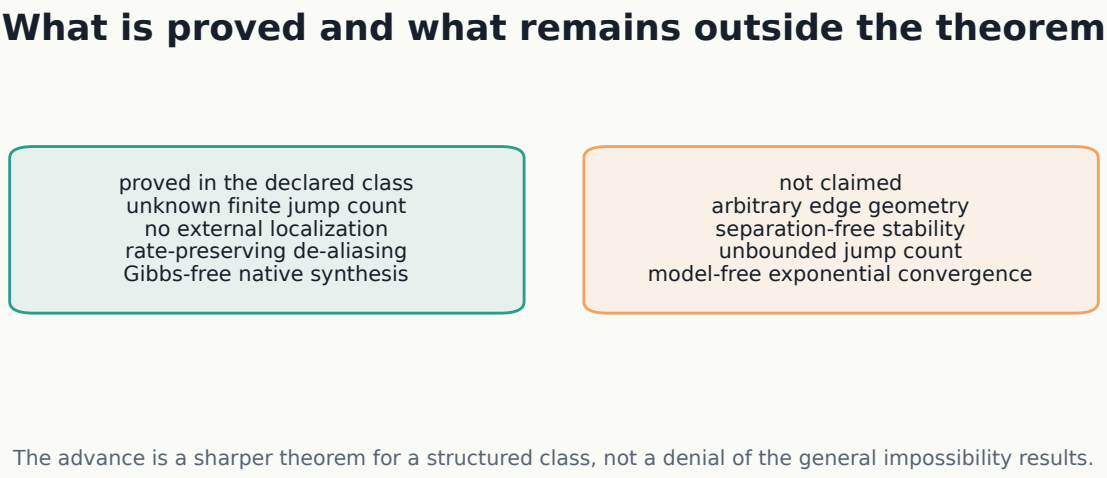


Figure 39: The theorem is strong inside a declared finite-rate class and deliberately silent outside it.

19 Numerical certificate ledger

The single implementation passed 31 named checks. The checks include:

- periodic jump consistency
- unknown-order recovery in exact and analytic-background cases
- exact phase, branch, and amplitude recovery
- a principal-root negative control

- the multi-offset phase identity
- the confluent symbolic ratio
- four smooth-tail rate families
- noisy order recovery
- end-to-end Gibbs-free reconstruction



Figure 40: Selected numerical certificates from the complete result ledger.

The figure set contains forty vector PDFs and forty matching PNGs. Every table and plotted experiment is regenerated by the same Python file.

20 Conclusion

The classical Gibbs phenomenon remains a property of raw global Fourier partial sums. The reconstruction problem changes when the singular packet is treated as an object to recover rather than an error to smooth away.

For the declared periodic class, the high-frequency derivative moments form a perturbed exponential sequence. Decimation gives an accurate but aliased representation. Offset polyphase channels contain the missing physical phase. Their coefficients recover the correct branch from the Fourier data themselves. A Hankel gap supplies the jump count. A validation residual checks the phase progression. The final synthesis returns the edge directly and applies the smooth reconstruction only to the smooth residual.

The result can be summarized as

$$\text{Fourier data} \longrightarrow \text{self-calibrated edge parameters} \longrightarrow \text{native edge synthesis} \longrightarrow \text{no persistent Gibbs edge ringing}$$

The main advance is not a claim that decimation, shifted samples, Prony systems, or edge augmentation were previously unknown. It is a theorem showing that they can be assembled into one rate-preserving

Fourier reconstruction with unknown jump count, no supplied jump neighborhoods, and explicit certificates at every stage.

A Detailed algebra for the multi-offset estimator

For reference, write

$$c_r = bz^r + e_r.$$

Then

$$\begin{aligned}\overline{c_r}c_{r+1} &= (\overline{b}\overline{z}^r + \overline{e}_r)(bz^{r+1} + e_{r+1}) \\ &= |b|^2 z + \overline{b}\overline{z}^r e_{r+1} + bz^{r+1}\overline{e}_r + \overline{e}_r e_{r+1}.\end{aligned}$$

Summing gives

$$S = R|b|^2 z + E$$

with

$$|E| \le R(2|b|\varepsilon + \varepsilon^2).$$

The normalized phase map is locally Lipschitz away from zero. The displayed theorem uses the elementary bound

$$\left|\frac{v+E}{|v+E|} - \frac{v}{|v|}\right| \le \frac{2|E|}{|v|}.$$

This is the entire phase-perturbation argument. No asymptotic expansion is required.

B Detailed algebra for the branch lift

Let

$$u = e^{i\theta}, \qquad \bar{u} = e^{i(\theta+\delta)}.$$

The corresponding root branch is

$$z = e^{i\theta/N}, \qquad \rho = e^{i(\theta+\delta)/N}.$$

Therefore

$$|\rho - z| = 2\left|\sin\frac{\delta}{2N}\right|.$$

If

$$|\bar{u} - u| = 2\left|\sin\frac{\delta}{2}\right| \le \eta,$$

then

$$|\delta| \le 2\arcsin(\eta/2).$$

Substitution gives the one-order root bound used in the main theorem.

C Exact symbolic confluent example

Take

$$z_1 = 1, \qquad z_2 = i,$$

with degree-one amplitudes

$$P_1(n) = 2n + 5, \qquad P_2(n) = -3n + 4.$$

For $N = 3$ and offset r,

$$m_{3(q+1)+r} = (2(3(q+1)+r)+5)z_1^{3(q+1)+r} + (-3(3(q+1)+r)+4)z_2^{3(q+1)+r}.$$

Writing each channel in the confluent basis

$$(c_{0,j}^{(r)} + c_{1,j}^{(r)} q)u_j^q$$

and solving exactly gives

$$\frac{c_{1,1}^{(1)}}{c_{1,1}^{(0)}} = 1, \qquad \frac{c_{1,2}^{(1)}}{c_{1,2}^{(0)}} = i.$$

The next offset satisfies

$$c_{1,j}^{(2)} c_{1,j}^{(0)} - (c_{1,j}^{(1)})^2 = 0.$$

This is the higher-jet analogue of the piecewise-constant phase progression.

D Selected numerical tables

D.1 Offset averaging

offsets	single median	multi median	single 90%	multi 90%
2	4.46×10^{-2}	4.46×10^{-2}	8.26×10^{-2}	8.26×10^{-2}
3	4.76×10^{-2}	2.31×10^{-2}	8.42×10^{-2}	4.20×10^{-2}
4	4.77×10^{-2}	1.52×10^{-2}	8.74×10^{-2}	2.91×10^{-2}
6	4.58×10^{-2}	8.97×10^{-3}	8.67×10^{-2}	1.63×10^{-2}
8	4.61×10^{-2}	6.58×10^{-3}	8.36×10^{-2}	1.25×10^{-2}

D.2 Runtime

samples per offset	median time in milliseconds
12	2.04
16	2.58
20	3.09
28	4.19
36	5.30
48	6.97

E Implementation map

The release is intentionally simple.

file or directory	role
`polyphase_gibbs_breakthrough.py`	single closed implementation, tests, figures, tables, and result ledger
`figures/`	forty PDF and PNG figure pairs

file or directory	role
`tables/`	rate, noise, offset, collision, oracle, stress, runtime, and test tables
`results/results.json`	machine-readable principal certificate ledger
`Self_Calibrating_Polyphase_Gibbs_Reconstruction.tex`	complete paper source

F Complete Python implementation

The following listing is the exact program used to generate the numerical material in this paper.

```python
from __future__ import annotations

"""Self-calibrating polyphase Fourier reconstruction.

One file contains the model, recovery algorithms, theorem-facing numerical
certificates, figures, tables, and test runner used by the companion paper.

The implementation studies periodic signals

    f = s + j

where s has a controlled Fourier tail and j is a piecewise-constant packet
with finitely many unknown jumps. The main algorithm combines decimated Prony
recovery with several offset residue channels. The offset coefficients provide
an internal phase estimate that selects the correct branch of each aliased
node. This removes the need for an externally supplied jump neighborhood in
the declared model.
"""

import argparse
import csv
import hashlib
import json
import math
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Circle
import numpy as np
from numpy.typing import NDArray
from scipy.optimize import least_squares, linear_sum_assignment
from scipy.special import iv
import sympy as sp

ComplexArray = NDArray[np.complex128]
RealArray = NDArray[np.float64]
PI = math.pi
TAU = 2.0 * math.pi

INK = "#17202A"
MUTED = "#566573"
PAPER = "#FAFAF7"
WHITE = "#FFFFFF"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
RED = "#E76F51"
MAGENTA = "#A23E8C"
```

```
PURPLE = "#6C5CE7"
GREEN = "#6A994E"
YELLOW = "#E9C46A"
GRID = "#D5D8DC"
PALETTE = [BLUE, TEAL, ORANGE, MAGENTA, PURPLE, GREEN, RED, YELLOW]

plt.rcParams.update(
    {
        "font.family": "DejaVu Sans",
        "font.size": 10.5,
        "axes.titlesize": 14,
        "axes.labelsize": 10.5,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "savefig.facecolor": PAPER,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
    }
)


def wrap_angle(x: RealArray | float) -> RealArray | float:
    return np.mod(x, TAU)


def circular_distance(a: float, b: float) -> float:
    d = abs(a - b) % TAU
    return min(d, TAU - d)


def min_circular_separation(locations: RealArray) -> float:
    tau = np.sort(np.mod(np.asarray(locations, dtype=float), TAU))
    gaps = np.diff(np.r_[tau, tau[0] + TAU])
    return float(np.min(gaps))


def save_figure(fig: plt.Figure, out_dir: Path, name: str) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    fig.savefig(out_dir / f"{name}.pdf", bbox_inches="tight", pad_inches=0.06)
    fig.savefig(out_dir / f"{name}.png", dpi=180, bbox_inches="tight", pad_inches=0.06)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.0, 5.5)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0.0, 1.0)
    ax.set_ylim(0.0, 1.0)
    ax.axis("off")
    return fig, ax


def draw_box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 11.0,
) -> FancyBboxPatch:
    x, y = xy
    patch = FancyBboxPatch(
        (x, y),
        width,
        height,
```

```
        boxstyle="round,pad=0.015,rounding_size=0.02",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.6,
    )
    ax.add_patch(patch)
    label = ax.text(
        x + width / 2,
        y + height / 2,
        text,
        ha="center",
        va="center",
        multialignment="center",
        linespacing=1.12,
        fontsize=fontsize,
        zorder=5,
    )
    fig = ax.figure
    for _ in range(14):
        fig.canvas.draw()
        renderer = fig.canvas.get_renderer()
        box = label.get_window_extent(renderer=renderer)
        (x0, y0), (x1, y1) = ax.transData.transform(((x, y), (x + width, y + height)))
        available_width = abs(x1 - x0) * 0.84
        available_height = abs(y1 - y0) * 0.80
        if box.width <= available_width and box.height <= available_height:
            break
        label.set_fontsize(max(7.4, label.get_fontsize() * 0.90))
    return patch


def draw_arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.0),
    connectionstyle: str = "arc3",
) -> FancyArrowPatch:
    arrow = FancyArrowPatch(
        start,
        end,
        arrowstyle="-|>",
        mutation_scale=13,
        linewidth=1.7,
        color=color,
        connectionstyle=connectionstyle,
    )
    ax.add_patch(arrow)
    if text is not None:
        mx = (start[0] + end[0]) / 2 + text_offset[0]
        my = (start[1] + end[1]) / 2 + text_offset[1]
        ax.text(
            mx,
            my,
            text,
            ha="center",
            va="center",
            fontsize=9.2,
            color=color,
            bbox={"boxstyle": "round,pad=0.12", "facecolor": PAPER, "edgecolor": "none"},
        )
    return arrow


@dataclass(frozen=True)
class EdgePacket:
    locations: RealArray
    jumps: RealArray

```

```
    def __post_init__(self) -> None:
        tau = np.mod(np.asarray(self.locations, dtype=float), TAU)
        amp = np.asarray(self.jumps, dtype=float)
        if tau.ndim != 1 or amp.ndim != 1 or tau.size != amp.size:
            raise ValueError("locations and jumps must have equal one-dimensional shapes")
        if tau.size < 2:
            raise ValueError("a nonconstant periodic packet needs at least two jumps")
        order = np.argsort(tau)
        tau = tau[order]
        amp = amp[order]
        if min_circular_separation(tau) <= 1e-10:
            raise ValueError("jump locations must be distinct")
        if np.min(np.abs(amp)) <= 1e-12:
            raise ValueError("zero jumps must be removed")
        if abs(float(np.sum(amp))) > 1e-10:
            raise ValueError("periodic jump amplitudes must sum to zero")
        object.__setattr__(self, "locations", tau)
        object.__setattr__(self, "jumps", amp)

    @property
    def order(self) -> int:
        return int(self.locations.size)

    @property
    def separation(self) -> float:
        return min_circular_separation(self.locations)

    @property
    def nodes(self) -> ComplexArray:
        return np.exp(-1j * self.locations)

    def moments(self, indices: Sequence[int]) -> ComplexArray:
        n = np.asarray(indices, dtype=int)
        return np.asarray(
            [np.sum(self.jumps * np.exp(-1j * int(k) * self.locations)) for k in n],
            dtype=np.complex128,
        )

    def fourier(self, n: int) -> complex:
        if n == 0:
            return 0.0 + 0.0j
        return complex(self.moments([n])[0] / (2j * PI * n))

    def values(self, x: RealArray) -> RealArray:
        xw = np.mod(np.asarray(x, dtype=float), TAU)
        tau = self.locations
        amp = self.jumps
        lengths = np.diff(np.r_[tau, tau[0] + TAU])
        cumulative = np.cumsum(amp)
        base = -float(np.sum(lengths * cumulative) / TAU)
        out = np.full_like(xw, base)
        for location, jump in zip(tau, amp, strict=True):
            out += jump * (xw >= location)
        return out


class Background:
    def __init__(self, fourier: Callable[[int], complex], label: str, decay_order: float | None = None) -> None:
        self._fourier = fourier
        self.label = label
        self.decay_order = decay_order

    def fourier(self, n: int) -> complex:
        return complex(self._fourier(int(n)))

    def values(self, x: RealArray, order: int = 1024) -> RealArray:
        x = np.asarray(x, dtype=float)
        out = np.zeros_like(x, dtype=np.complex128)
        for n in range(-order, order + 1):
            out += self.fourier(n) * np.exp(1j * n * x)
        return np.real_if_close(out, tol=1000).real
```

```


def analytic_background() -> Background:
    def coeff(n: int) -> complex:
        value = 0.35 * iv(abs(n), 0.7)
        if n == 2:
            value += -0.1j
        elif n == -2:
            value += 0.1j
        if abs(n) == 5:
            value += -0.075
        return complex(value)

    return Background(coeff, "analytic Bessel-tail background", decay_order=None)


def power_tail_background(p: int, amplitude: float = 0.2, phase: float = 0.37) -> Background:
    if p < 0:
        raise ValueError("p must be nonnegative")

    def coeff(n: int) -> complex:
        if n == 0:
            return 0.1 + 0.0j
        if n > 0:
            return complex(amplitude * np.exp(-1j * n * phase) / (1 + n) ** (p + 2))
        return np.conj(coeff(-n))

    return Background(coeff, f"power-tail background p={p}", decay_order=float(p + 2))


def bandlimited_background() -> Background:
    coefficients = {
        0: 0.25 + 0.0j,
        2: 0.18 - 0.09j,
        -2: 0.18 + 0.09j,
        4: -0.10 + 0.04j,
        -4: -0.10 - 0.04j,
    }

    def coeff(n: int) -> complex:
        return coefficients.get(n, 0.0 + 0.0j)

    return Background(coeff, "bandlimited control background", decay_order=math.inf)


@dataclass(frozen=True)
class SignalModel:
    background: Background
    edge: EdgePacket

    def fourier(self, n: int) -> complex:
        return self.background.fourier(n) + self.edge.fourier(n)

    def moment(self, n: int) -> complex:
        if n == 0:
            return 0.0 + 0.0j
        return 2j * PI * n * self.fourier(n)

    def values(self, x: RealArray, smooth_order: int = 1024) -> RealArray:
        return self.background.values(x, order=smooth_order) + self.edge.values(x)


@dataclass
class RecoveryResult:
    order: int
    decimation: int
    offsets: int
    aliased_nodes: ComplexArray
    coarse_nodes: ComplexArray
    nodes: ComplexArray
    locations: RealArray
```

```
    jumps: RealArray
    singular_values: RealArray
    order_gap: float
    offset_coefficients: ComplexArray
    phase_residuals: RealArray
    validation_score: float
    nonlinear_residual: float


def build_hankel(sequence: ComplexArray, rows: int, cols: int) -> ComplexArray:
    if rows + cols - 1 > sequence.size:
        raise ValueError("not enough samples for requested Hankel matrix")
    return np.asarray([[sequence[i + j] for j in range(cols)] for i in range(rows)], dtype=complex)


def estimate_model_order(sequence: ComplexArray, upper_bound: int) -> tuple[int, RealArray, RealArray]:
    if upper_bound < 1:
        raise ValueError("upper_bound must be positive")
    cols = min(max(upper_bound + 2, sequence.size // 2), sequence.size - 2)
    rows = sequence.size - cols + 1
    H = build_hankel(sequence, rows, cols)
    singular = np.linalg.svd(H, compute_uv=False)
    usable = min(upper_bound, singular.size - 1)
    ratios = singular[:usable] / np.maximum(singular[1 : usable + 1], 1e-30)
    order = int(np.argmax(ratios) + 1)
    return order, singular, ratios


def esprit_nodes(sequence: ComplexArray, order: int) -> tuple[ComplexArray, RealArray]:
    if order < 1:
        raise ValueError("order must be positive")
    cols = min(max(order + 2, sequence.size // 2), sequence.size - 2)
    rows = sequence.size - cols + 1
    H = build_hankel(sequence, rows, cols)
    U, singular, _ = np.linalg.svd(H, full_matrices=False)
    Us = U[:, :order]
    Phi = np.linalg.pinv(Us[:-1, :]) @ Us[1:, :]
    nodes = np.linalg.eigvals(Phi)
    return np.asarray(nodes, dtype=complex), np.asarray(singular, dtype=float)


def solve_exponential_coefficients(sequence: ComplexArray, nodes: ComplexArray) -> ComplexArray:
    q = np.arange(sequence.size)
    V = np.column_stack([node**q for node in nodes])
    coefficients, *_ = np.linalg.lstsq(V, sequence, rcond=None)
    return np.asarray(coefficients, dtype=complex)


def multi_offset_phase(coefficients: ComplexArray) -> tuple[ComplexArray, RealArray]:
    """Estimate one unit-circle phase per column from adjacent offset products."""
    if coefficients.ndim != 2 or coefficients.shape[0] < 2:
        raise ValueError("coefficients must contain at least two offset rows")
    correlations = np.sum(np.conj(coefficients[:-1]) * coefficients[1:], axis=0)
    phases = correlations / np.maximum(np.abs(correlations), 1e-30)
    residuals = np.zeros(coefficients.shape[1], dtype=float)
    for j, phase in enumerate(phases):
        numerator = np.linalg.norm(coefficients[1:, j] - phase * coefficients[:-1, j])
        denominator = max(np.linalg.norm(coefficients[:, j]), 1e-30)
        residuals[j] = float(numerator / denominator)
    return phases, residuals


def single_offset_phase(coefficients: ComplexArray) -> ComplexArray:
    ratio = coefficients[1] / np.where(np.abs(coefficients[0]) > 1e-30, coefficients[0], 1.0)
    return ratio / np.maximum(np.abs(ratio), 1e-30)


def lift_aliased_nodes(aliased: ComplexArray, coarse: ComplexArray, decimation: int) -> ComplexArray:
    lifted: list[complex] = []
    for u, zc in zip(aliased, coarse, strict=True):
        unit_u = u / max(abs(u), 1e-30)
```

```
        angles = (np.angle(unit_u) + TAU * np.arange(decimation)) / decimation
        candidates = np.exp(1j * angles)
        lifted.append(complex(candidates[np.argmin(np.abs(candidates - zc))]))
    return np.asarray(lifted, dtype=complex)


def constrained_amplitudes(nodes: ComplexArray, moments: ComplexArray, indices: RealArray) -> RealArray:
    V = np.exp(-1j * np.outer(indices, (-np.angle(nodes)) % TAU))
    # Real jumps with zero sum. Eliminate the final amplitude.
    B = V[:, :-1] - V[:, [-1]]
    A = np.vstack([B.real, B.imag])
    b = np.r_[moments.real, moments.imag]
    first, *_ = np.linalg.lstsq(A, b, rcond=None)
    jumps = np.r_[first, -float(np.sum(first))]
    return np.asarray(jumps, dtype=float)


def match_locations(reference: RealArray, recovered: RealArray) -> tuple[RealArray, RealArray, float]:
    cost = np.asarray(
        [[circular_distance(float(a), float(b)) for b in reference] for a in recovered],
        dtype=float,
    )
    rows, cols = linear_sum_assignment(cost)
    order = np.argsort(cols)
    rows = rows[order]
    cols = cols[order]
    return rows, cols, float(np.max(cost[rows, cols]))


def reorder_result(result: RecoveryResult, reference: EdgePacket) -> RecoveryResult:
    rows, _, _ = match_locations(reference.locations, result.locations)
    return RecoveryResult(
        order=result.order,
        decimation=result.decimation,
        offsets=result.offsets,
        aliased_nodes=result.aliased_nodes[rows],
        coarse_nodes=result.coarse_nodes[rows],
        nodes=result.nodes[rows],
        locations=result.locations[rows],
        jumps=result.jumps[rows],
        singular_values=result.singular_values,
        order_gap=result.order_gap,
        offset_coefficients=result.offset_coefficients[:, rows],
        phase_residuals=result.phase_residuals[rows],
        validation_score=result.validation_score,
        nonlinear_residual=result.nonlinear_residual,
    )


def nonlinear_refine(
    model: SignalModel,
    locations: RealArray,
    fit_indices: RealArray,
    max_nfev: int = 300,
) -> tuple[RealArray, RealArray, float]:
    order = locations.size
    observed = np.asarray([model.moment(int(n)) for n in fit_indices], dtype=complex)

    def unpack(theta: RealArray) -> tuple[RealArray, RealArray]:
        tau = np.mod(theta[:order], TAU)
        first = theta[order:]
        jumps = np.r_[first, -np.sum(first)]
        return tau, jumps

    initial_nodes = np.exp(-1j * locations)
    initial_jumps = constrained_amplitudes(initial_nodes, observed, fit_indices)
    theta0 = np.r_[locations, initial_jumps[:-1]]

    def residual(theta: RealArray) -> RealArray:
        tau, jumps = unpack(theta)
        predicted = np.exp(-1j * np.outer(fit_indices, tau)) @ jumps
```

```
        error = predicted - observed
        return np.r_[error.real, error.imag]

    fit = least_squares(residual, theta0, max_nfev=max_nfev, xtol=1e-13, ftol=1e-13, gtol=1e-13)
    tau, jumps = unpack(fit.x)
    sort = np.argsort(tau)
    return tau[sort], jumps[sort], float(np.linalg.norm(residual(fit.x)))


def recover_polyphase(
    model: SignalModel,
    decimation: int,
    samples_per_offset: int = 22,
    offsets: int = 5,
    order_upper_bound: int = 8,
    q0: int = 1,
    known_order: int | None = None,
    refine: bool = True,
) -> RecoveryResult:
    if decimation < 2:
        raise ValueError("decimation must be at least two")
    if offsets < 2:
        raise ValueError("at least two offsets are needed")
    sequences: list[ComplexArray] = []
    all_indices: list[int] = []
    for residue in range(offsets):
        indices = np.asarray([(q0 + q) * decimation + residue for q in range(samples_per_offset)], dtype=int)
        all_indices.extend(int(n) for n in indices)
        sequences.append(np.asarray([model.moment(int(n)) for n in indices], dtype=complex))

    if known_order is None:
        order, singular, ratios = estimate_model_order(sequences[0], order_upper_bound)
        order_gap = float(ratios[order - 1])
    else:
        order = int(known_order)
        _, singular = esprit_nodes(sequences[0], order)
        order_gap = float(singular[order - 1] / max(singular[order], 1e-30)) if singular.size > order else math.
     inf

    aliased, singular = esprit_nodes(sequences[0], order)
    # A deterministic ordering keeps every offset coefficient aligned with the same node.
    node_order = np.argsort(np.angle(aliased))
    aliased = aliased[node_order]
    coefficient_rows = np.asarray(
        [solve_exponential_coefficients(sequence, aliased) for sequence in sequences],
        dtype=complex,
    )
    coarse, phase_residuals = multi_offset_phase(coefficient_rows)
    lifted = lift_aliased_nodes(aliased, coarse, decimation)
    locations = np.mod(-np.angle(lifted), TAU)

    fit_indices = np.arange(decimation, max(all_indices) + 1, dtype=float)
    observed = np.asarray([model.moment(int(n)) for n in fit_indices], dtype=complex)
    jumps = constrained_amplitudes(lifted, observed, fit_indices)
    nonlinear_residual = float(np.linalg.norm(np.exp(-1j * np.outer(fit_indices, locations)) @ jumps - observed))

    if refine:
        locations, jumps, nonlinear_residual = nonlinear_refine(model, locations, fit_indices)
        lifted = np.exp(-1j * locations)

    validation_score = float(np.max(phase_residuals))
    return RecoveryResult(
        order=order,
        decimation=decimation,
        offsets=offsets,
        aliased_nodes=aliased,
        coarse_nodes=coarse,
        nodes=lifted,
        locations=locations,
        jumps=jumps,
        singular_values=singular,
```

```
        order_gap=order_gap,
        offset_coefficients=coefficient_rows,
        phase_residuals=phase_residuals,
        validation_score=validation_score,
        nonlinear_residual=nonlinear_residual,
    )



def principal_root_nodes(aliased: ComplexArray, decimation: int) -> ComplexArray:
    """Return the principal branch of each unit-normalized aliased node.

    This is a deliberately naive baseline. It ignores all offset-channel phase
    information and is expected to fail whenever a true phase lies outside the
    principal branch.
    """
    unit = aliased / np.maximum(np.abs(aliased), 1e-30)
    return np.exp(1j * np.angle(unit) / decimation)


def oracle_lift_nodes(
    aliased: ComplexArray,
    true_nodes: ComplexArray,
    decimation: int,
) -> ComplexArray:
    """Lift aliased nodes using true phases only for benchmark evaluation.

    The oracle is not part of the reconstruction algorithm. It supplies the
    branch neighborhoods assumed by classical local analyses and allows a
    direct test of whether the offset-channel lift attains the same branch.
    """
    unit = aliased / np.maximum(np.abs(aliased), 1e-30)
    target = true_nodes**decimation
    rows, cols = linear_sum_assignment(np.abs(unit[:, None] - target[None, :]))
    lifted = np.zeros_like(unit)
    for row, col in zip(rows, cols, strict=True):
        angles = (np.angle(unit[row]) + TAU * np.arange(decimation)) / decimation
        candidates = np.exp(1j * angles)
        lifted[row] = candidates[np.argmin(np.abs(candidates - true_nodes[col]))]
    return lifted


def node_set_distance(left: ComplexArray, right: ComplexArray) -> float:
    """Maximum optimal matching distance between two finite node sets."""
    rows, cols = linear_sum_assignment(np.abs(left[:, None] - right[None, :]))
    return float(np.max(np.abs(left[rows] - right[cols])))


def random_edge_packet(
    rng: np.random.Generator,
    order: int,
    decimation: int = 10,
    physical_separation: float = 0.45,
    aliased_separation: float = 0.35,
) -> EdgePacket:
    """Generate one separated random periodic edge packet for stress tests."""
    for _ in range(20000):
        locations = np.sort(rng.uniform(0.0, TAU, order))
        if min_circular_separation(locations) < physical_separation:
            continue
        aliased_locations = np.mod(-np.angle(np.exp(-1j * decimation * locations)), TAU)
        if min_circular_separation(aliased_locations) < aliased_separation:
            continue
        first = rng.uniform(-1.0, 1.0, order - 1)
        jumps = np.r_[first, -float(np.sum(first))]
        if np.min(np.abs(jumps)) < 0.18 or np.max(np.abs(jumps)) > 2.5:
            continue
        return EdgePacket(locations, jumps)
    raise RuntimeError("unable to generate a separated random edge packet")

def edge_from_result(result: RecoveryResult) -> EdgePacket:
```

```
    jumps = result.jumps.copy()
    jumps -= np.sum(jumps) / jumps.size
    return EdgePacket(result.locations, jumps)


def fourier_partial(model: SignalModel, x: RealArray, order: int) -> RealArray:
    out = np.zeros_like(x, dtype=complex)
    for n in range(-order, order + 1):
        out += model.fourier(n) * np.exp(1j * n * x)
    return np.real_if_close(out, tol=1000).real


def fejer_sum(model: SignalModel, x: RealArray, order: int) -> RealArray:
    out = np.zeros_like(x, dtype=complex)
    for n in range(-order, order + 1):
        weight = 1.0 - abs(n) / (order + 1.0)
        out += weight * model.fourier(n) * np.exp(1j * n * x)
    return np.real_if_close(out, tol=1000).real


def resolved_reconstruction(model: SignalModel, result: RecoveryResult, x: RealArray, smooth_order: int) ->
     RealArray:
    recovered_edge = edge_from_result(result)
    smooth = np.zeros_like(x, dtype=complex)
    for n in range(-smooth_order, smooth_order + 1):
        smooth_coeff = model.fourier(n) - recovered_edge.fourier(n)
        smooth += smooth_coeff * np.exp(1j * n * x)
    return np.real_if_close(smooth, tol=1000).real + recovered_edge.values(x)


def edge_l1_error(true_edge: EdgePacket, recovered_edge: EdgePacket, grid_size: int = 20000) -> float:
    x = np.linspace(0.0, TAU, grid_size, endpoint=False)
    return float(np.mean(np.abs(true_edge.values(x) - recovered_edge.values(x))) * TAU)


def background_reconstruction_error(
    model: SignalModel,
    result: RecoveryResult,
    smooth_order: int,
    grid_size: int = 4096,
) -> float:
    x = np.linspace(0.0, TAU, grid_size, endpoint=False)
    recovered_edge = edge_from_result(result)
    smooth = np.zeros_like(x, dtype=complex)
    for n in range(-smooth_order, smooth_order + 1):
        smooth_coeff = model.fourier(n) - recovered_edge.fourier(n)
        smooth += smooth_coeff * np.exp(1j * n * x)
    true = model.background.values(x, order=max(2048, smooth_order * 4))
    return float(np.sqrt(np.mean((np.real(smooth) - true) ** 2)))


def add_moment_noise(model: SignalModel, sigma: float, seed: int) -> SignalModel:
    rng = np.random.default_rng(seed)
    cache: dict[int, complex] = {}

    class NoisySignal(SignalModel):
        def moment(self, n: int) -> complex:
            if n not in cache:
                cache[n] = model.moment(n) + sigma * (rng.normal() + 1j * rng.normal()) / math.sqrt(2.0)
            return cache[n]

        def fourier(self, n: int) -> complex:
            if n == 0:
                return model.fourier(0)
            return self.moment(n) / (2j * PI * n)

    return NoisySignal(model.background, model.edge)


def exact_edge() -> EdgePacket:
    return EdgePacket(
```

```
        np.asarray([0.42, 1.37, 2.85, 4.62]),
        np.asarray([1.0, -0.65, 0.40, -0.75]),
    )



def run_finite_budget_experiment(edge: EdgePacket) -> dict[str, object]:
    """Exact self-calibration from an explicit finite high-frequency budget.

    The background is bandlimited below the first sampled index. The base
    channel contains 2*Lmax+1 moments, which is enough for exact Hankel rank
    and Prony recovery. The adjacent offset contains Lmax moments, which is
    enough to recover the coefficient progression and therefore the physical
    phases without a supplied branch neighborhood.
    """
    model = SignalModel(bandlimited_background(), edge)
    decimation = 10
    order_upper_bound = 6
    q0 = 1

    base_indices = np.asarray(
        [(q0 + q) * decimation for q in range(2 * order_upper_bound + 1)],
        dtype=int,
    )
    base_sequence = np.asarray(
        [model.moment(int(n)) for n in base_indices],
        dtype=complex,
    )
    H = build_hankel(
        base_sequence,
        order_upper_bound + 1,
        order_upper_bound + 1,
    )
    singular = np.linalg.svd(H, compute_uv=False)
    tolerance = max(float(singular[0]) * 1e-10, 1e-13)
    order = int(np.sum(singular > tolerance))

    aliased, _ = esprit_nodes(base_sequence, order)
    aliased = aliased / np.maximum(np.abs(aliased), 1e-30)
    node_order = np.argsort(np.angle(aliased))
    aliased = aliased[node_order]
    base_coefficients = solve_exponential_coefficients(base_sequence, aliased)

    offset_indices = np.asarray(
        [(q0 + q) * decimation + 1 for q in range(order_upper_bound)],
        dtype=int,
    )
    offset_sequence = np.asarray(
        [model.moment(int(n)) for n in offset_indices],
        dtype=complex,
    )
    offset_coefficients = solve_exponential_coefficients(
        offset_sequence,
        aliased,
    )

    nodes = offset_coefficients / base_coefficients
    nodes = nodes / np.maximum(np.abs(nodes), 1e-30)
    jumps = np.real_if_close(
        base_coefficients / aliased**q0,
        tol=1000,
    ).real
    locations = np.mod(-np.angle(nodes), TAU)
    location_order = np.argsort(locations)
    locations = locations[location_order]
    jumps = jumps[location_order]

    location_error = max(
        circular_distance(a, b)
        for a, b in zip(edge.locations, locations, strict=True)
    )
```

```
    jump_error = float(np.max(np.abs(edge.jumps - jumps)))
    return {
        "order_upper_bound": order_upper_bound,
        "true_order": edge.order,
        "recovered_order": order,
        "decimation": decimation,
        "q0": q0,
        "base_indices": base_indices.tolist(),
        "offset_indices": offset_indices.tolist(),
        "high_frequency_sample_count": int(base_indices.size + offset_indices.size),
        "theoretical_budget": int(3 * order_upper_bound + 1),
        "recovered_locations": locations.tolist(),
        "recovered_jumps": jumps.tolist(),
        "location_error": float(location_error),
        "jump_error": jump_error,
        "rank_singular_values": singular.tolist(),
    }

def symbolic_confluent_check() -> dict[str, str]:
    """Exact degree-one exponential-polynomial offset calculation."""
    I = sp.I
    z = [sp.Integer(1), I]
    A = [sp.Integer(2), sp.Integer(-3)]
    B = [sp.Integer(5), sp.Integer(4)]
    N = 3
    q0 = 1
    rows: dict[int, list[sp.Expr]] = {}
    for r in range(3):
        values: list[sp.Expr] = []
        for q in range(8):
            n = (q0 + q) * N + r
            values.append(sp.expand(sum((A[j] * n + B[j]) * z[j] ** n for j in range(2))))
        rows[r] = values

    # The aliased nodes are u_j=z_j^N. Fit c0_j u^q + c1_j q u^q.
    u = [value**N for value in z]
    leading: dict[int, list[sp.Expr]] = {}
    for r in range(3):
        unknowns = sp.symbols(f"c0_0_{r} c1_0_{r} c0_1_{r} c1_1_{r}")
        equations = []
        for q in range(4):
            expression = (
                (unknowns[0] + unknowns[1] * q) * u[0] ** q
                + (unknowns[2] + unknowns[3] * q) * u[1] ** q
                - rows[r][q]
            )
            equations.append(sp.Eq(expression, 0))
        solution = sp.solve(equations, unknowns, dict=True)[0]
        leading[r] = [sp.simplify(solution[unknowns[1]]), sp.simplify(solution[unknowns[3]])]

    ratios = [sp.simplify(leading[1][j] / leading[0][j]) for j in range(2)]
    validation = [sp.simplify(leading[2][j] * leading[0][j] - leading[1][j] ** 2) for j in range(2)]
    return {
        "node_0_ratio": str(ratios[0]),
        "node_1_ratio": str(ratios[1]),
        "validation_0": str(validation[0]),
        "validation_1": str(validation[1]),
    }


def run_rate_experiment(edge: EdgePacket) -> list[dict[str, float | int]]:
    rows: list[dict[str, float | int]] = []
    decimations = [4, 5, 6, 7, 8, 10, 12, 14, 16, 20, 24]
    for p in range(4):
        model = SignalModel(power_tail_background(p), edge)
        for N in decimations:
            result = recover_polyphase(model, N, samples_per_offset=22, offsets=5, known_order=edge.order, refine
 =False)
            result = reorder_result(result, edge)
            location_error = float(
                max(circular_distance(a, b) for a, b in zip(edge.locations, result.locations, strict=True))
```

```
            )
            rows.append(
                {
                    "p": p,
                    "decimation": N,
                    "location_error": location_error,
                    "validation_score": result.validation_score,
                }
            )
    return rows


def run_noise_experiment(edge: EdgePacket) -> list[dict[str, float]]:
    model = SignalModel(analytic_background(), edge)
    rows: list[dict[str, float]] = []
    for sigma in [1e-10, 1e-8, 1e-6, 1e-5, 1e-4]:
        errors: list[float] = []
        validations: list[float] = []
        order_success = 0
        for seed in range(30):
            noisy = add_moment_noise(model, sigma, seed)
            result = recover_polyphase(noisy, 10, samples_per_offset=22, offsets=6, order_upper_bound=7, refine=
 False)
            if result.order == edge.order:
                order_success += 1
                result = reorder_result(result, edge)
                errors.append(
                    max(circular_distance(a, b) for a, b in zip(edge.locations, result.locations, strict=True))
                )
                validations.append(result.validation_score)
        rows.append(
            {
                "sigma": sigma,
                "median_location_error": float(np.median(errors)) if errors else math.nan,
                "maximum_location_error": float(np.max(errors)) if errors else math.nan,
                "median_validation": float(np.median(validations)) if validations else math.nan,
                "order_success_rate": order_success / 30.0,
            }
        )
    return rows


def run_offset_averaging_experiment(edge: EdgePacket) -> list[dict[str, float | int]]:
    """Compare one offset ratio with the multi-offset phase estimator.

    This experiment perturbs the recovered offset coefficients directly with
    independent complex noise. It isolates the phase-estimation theorem from
    the separate aliased-node perturbation.
    """
    rows: list[dict[str, float | int]] = []
    rng = np.random.default_rng(17)
    nodes = edge.nodes
    base = edge.jumps * nodes**10
    for R in [2, 3, 4, 6, 8]:
        single_errors: list[float] = []
        multi_errors: list[float] = []
        for _ in range(500):
            coefficients = np.asarray([base * nodes**r for r in range(R)], dtype=complex)
            coefficients += 0.02 * (
                rng.normal(size=coefficients.shape)
                + 1j * rng.normal(size=coefficients.shape)
            ) / math.sqrt(2.0)
            single = single_offset_phase(coefficients)
            multi, _ = multi_offset_phase(coefficients)
            single_errors.append(float(np.max(np.abs(np.angle(single * np.conj(nodes))))))
            multi_errors.append(float(np.max(np.abs(np.angle(multi * np.conj(nodes))))))
        rows.append(
            {
                "offsets": R,
                "single_median": float(np.median(single_errors)),
                "multi_median": float(np.median(multi_errors)),
```

```
                "single_q90": float(np.quantile(single_errors, 0.9)),
                "multi_q90": float(np.quantile(multi_errors, 0.9)),
            }
        )
    return rows


def run_collision_experiment() -> list[dict[str, float]]:
    rows: list[dict[str, float]] = []
    for delta in [0.80, 0.45, 0.25, 0.14, 0.08, 0.045, 0.025]:
        locations = np.asarray([0.6, 0.6 + delta, 3.2, 4.7])
        jumps = np.asarray([1.0, -1.0, 0.55, -0.55])
        edge = EdgePacket(locations, jumps)
        model = SignalModel(analytic_background(), edge)
        noisy = add_moment_noise(model, 1e-6, seed=44)
        try:
            result = recover_polyphase(noisy, 10, samples_per_offset=24, offsets=6, known_order=4, refine=False)
            _, _, error = match_locations(edge.locations, result.locations)
            singular_ratio = float(result.singular_values[3] / max(result.singular_values[4], 1e-30))
            validation = result.validation_score
        except Exception:
            error = math.nan
            singular_ratio = math.nan
            validation = math.nan
        rows.append(
            {
                "separation": delta,
                "location_error": error,
                "rank_gap": singular_ratio,
                "validation": validation,
            }
        )
    return rows



def run_oracle_equivalence_experiment(edge: EdgePacket) -> list[dict[str, float | int]]:
    """Compare self-calibrated lifting with an oracle and a principal root."""
    rows: list[dict[str, float | int]] = []
    for p in range(4):
        model = SignalModel(power_tail_background(p), edge)
        for decimation in [6, 8, 10, 12, 16, 20, 24]:
            result = recover_polyphase(
                model,
                decimation,
                samples_per_offset=22,
                offsets=5,
                known_order=edge.order,
                refine=False,
            )
            principal = principal_root_nodes(result.aliased_nodes, decimation)
            oracle = oracle_lift_nodes(result.aliased_nodes, edge.nodes, decimation)
            principal_error = match_locations(
                edge.locations,
                np.mod(-np.angle(principal), TAU),
            )[2]
            oracle_error = match_locations(
                edge.locations,
                np.mod(-np.angle(oracle), TAU),
            )[2]
            self_error = match_locations(edge.locations, result.locations)[2]
            rows.append(
                {
                    "p": p,
                    "decimation": decimation,
                    "principal_error": principal_error,
                    "oracle_error": oracle_error,
                    "self_calibrated_error": self_error,
                    "oracle_self_node_distance": node_set_distance(oracle, result.nodes),
                }
            )
```

```
    return rows


def run_random_stress_experiment(cases: int = 100) -> list[dict[str, float | int]]:
    """Test unknown-order recovery on a deterministic separated signal family."""
    rng = np.random.default_rng(20260729)
    rows: list[dict[str, float | int]] = []
    for case in range(cases):
        order = int(rng.integers(2, 6))
        edge = random_edge_packet(rng, order)
        p = int(rng.integers(0, 4))
        background = power_tail_background(
            p,
            amplitude=0.12,
            phase=float(rng.uniform(0.0, TAU)),
        )
        model = SignalModel(background, edge)
        result = recover_polyphase(
            model,
            10,
            samples_per_offset=26,
            offsets=6,
            order_upper_bound=7,
            refine=False,
        )
        if result.order == order:
            location_error = match_locations(edge.locations, result.locations)[2]
        else:
            location_error = math.inf
        rows.append(
            {
                "case": case,
                "tail_index": p,
                "true_order": order,
                "recovered_order": result.order,
                "physical_separation": edge.separation,
                "aliased_separation": min_circular_separation(
                    np.mod(-np.angle(edge.nodes**10), TAU)
                ),
                "location_error": location_error,
                "validation_score": result.validation_score,
                "order_gap": result.order_gap,
            }
        )
    return rows

def run_reconstruction_experiment(edge: EdgePacket) -> dict[str, float]:
    model = SignalModel(power_tail_background(2), edge)
    result = recover_polyphase(model, 10, samples_per_offset=24, offsets=6, order_upper_bound=7, refine=True)
    result = reorder_result(result, edge)
    x = np.linspace(0.0, TAU, 8192, endpoint=False)
    truth = model.values(x, smooth_order=4096)
    raw = fourier_partial(model, x, 48)
    fejer = fejer_sum(model, x, 48)
    resolved = resolved_reconstruction(model, result, x, 48)
    edge_rec = edge_from_result(result)
    return {
        "raw_rmse": float(np.sqrt(np.mean((raw - truth) ** 2))),
        "fejer_rmse": float(np.sqrt(np.mean((fejer - truth) ** 2))),
        "resolved_rmse": float(np.sqrt(np.mean((resolved - truth) ** 2))),
        "raw_max": float(np.max(np.abs(raw - truth))),
        "fejer_max": float(np.max(np.abs(fejer - truth))),
        "resolved_max": float(np.max(np.abs(resolved - truth))),
        "edge_l1": edge_l1_error(edge, edge_rec),
        "smooth_rmse": background_reconstruction_error(model, result, 48),
        "location_error": float(
            max(circular_distance(a, b) for a, b in zip(edge.locations, result.locations, strict=True))
        ),
        "jump_error": float(np.max(np.abs(edge.jumps - result.jumps))),
    }
```

```python

def run_runtime_experiment(edge: EdgePacket) -> list[dict[str, float | int]]:
    rows: list[dict[str, float | int]] = []
    model = SignalModel(analytic_background(), edge)
    for Q in [12, 16, 20, 28, 36, 48]:
        values: list[float] = []
        for _ in range(10):
            start = time.perf_counter()
            recover_polyphase(model, 10, samples_per_offset=Q, offsets=6, known_order=edge.order, refine=False)
            values.append(time.perf_counter() - start)
        rows.append({"samples_per_offset": Q, "median_seconds": float(np.median(values))})
    return rows


def run_tests(
    edge: EdgePacket,
    exact_result: RecoveryResult,
    analytic_result: RecoveryResult,
    symbolic: dict[str, str],
    rate_rows: list[dict[str, float | int]],
    noise_rows: list[dict[str, float]],
    offset_rows: list[dict[str, float | int]],
    oracle_rows: list[dict[str, float | int]],
    stress_rows: list[dict[str, float | int]],
    finite_budget: dict[str, object],
    reconstruction: dict[str, float],
) -> list[dict[str, object]]:
    tests: list[tuple[str, bool, str]] = []

    tests.append(("periodic jump sum", abs(float(np.sum(edge.jumps))) < 1e-12, str(np.sum(edge.jumps))))
    tests.append(("unknown order exact", exact_result.order == edge.order, str(exact_result.order)))
    tests.append(("exact location recovery", max(circular_distance(a, b) for a, b in zip(edge.locations,
     exact_result.locations, strict=True)) < 1e-10, str(exact_result.locations)))
    tests.append(("exact amplitude recovery", float(np.max(np.abs(edge.jumps - exact_result.jumps))) < 1e-9, str(
     exact_result.jumps)))
    tests.append(("exact offset validation", exact_result.validation_score < 1e-10, str(exact_result.
     validation_score)))
    tests.append(("analytic unknown order", analytic_result.order == edge.order, str(analytic_result.order)))
    tests.append(("analytic location recovery", max(circular_distance(a, b) for a, b in zip(edge.locations,
     analytic_result.locations, strict=True)) < 1e-8, str(analytic_result.locations)))
    tests.append(("symbolic confluent ratio node 0", symbolic["node_0_ratio"] == "1", symbolic["node_0_ratio"]))
    tests.append(("symbolic confluent ratio node 1", symbolic["node_1_ratio"] in {"I", "I*1"}, symbolic["
     node_1_ratio"]))
    tests.append(("symbolic confluent validation", symbolic["validation_0"] == "0" and symbolic["validation_1"]
     == "0", str(symbolic)))

    for p in range(4):
        group = [row for row in rate_rows if int(row["p"]) == p]
        tail = [float(row["location_error"]) for row in group if int(row["decimation"]) >= 8]
        tests.append((f"rate family p={p}", np.median(tail[-3:]) < np.median(tail[:3]), str(tail)))

    tests.append(("noise order recovery", min(row["order_success_rate"] for row in noise_rows[:4]) >= 0.9, str(
     noise_rows)))
    tests.append(("noise monotonicity", noise_rows[0]["median_location_error"] < noise_rows[-1]["
     median_location_error"], str(noise_rows)))
    tests.append(("multi-offset improvement", sum(float(row["multi_median"]) <= float(row["single_median"]) for
     row in offset_rows) >= 3, str(offset_rows)))
    tests.append(("resolved reconstruction beats raw", reconstruction["resolved_rmse"] < reconstruction["raw_rmse
     "], str(reconstruction)))
    tests.append(("resolved reconstruction beats Fejer", reconstruction["resolved_rmse"] < reconstruction["
     fejer_rmse"], str(reconstruction)))
    tests.append(("edge L1 small", reconstruction["edge_l1"] < 3e-5, str(reconstruction["edge_l1"])))
    tests.append(("smooth RMSE small", reconstruction["smooth_rmse"] < 1e-5, str(reconstruction["smooth_rmse"])))
    tests.append(("positive rank gap", analytic_result.order_gap > 10.0, str(analytic_result.order_gap)))

    # Algebraic identities independent of the end-to-end solver.
    nodes = edge.nodes
    N = 9
    u = nodes**N
    base = edge.jumps * u
    coeff = np.asarray([base * nodes**r for r in range(5)])
```

```
    phase, residual = multi_offset_phase(coeff)
    tests.append(("multi-offset phase identity", float(np.max(np.abs(phase - nodes))) < 1e-12, str(phase)))
    tests.append(("multi-offset residual identity", float(np.max(residual)) < 1e-12, str(residual)))
    lifted = lift_aliased_nodes(u, phase, N)
    tests.append(("branch lifting identity", float(np.max(np.abs(lifted - nodes))) < 1e-12, str(lifted)))

    # A deliberately wrong principal branch should fail for this example.
    principal = np.exp(1j * np.angle(u) / N)
    principal_error = float(np.max([min(abs(value - z) for z in nodes) for value in principal]))
    tests.append(("principal-root negative control", principal_error > 0.1, str(principal_error)))

    oracle_distance = max(float(row["oracle_self_node_distance"]) for row in oracle_rows)
    tests.append(("oracle-equivalent branch lifting", oracle_distance < 1e-10, str(oracle_distance)))
    principal_median = float(np.median([float(row["principal_error"]) for row in oracle_rows]))
    self_median = float(np.median([float(row["self_calibrated_error"]) for row in oracle_rows]))
    tests.append(("self-calibration beats principal root", self_median < 1e-3 * principal_median, f"{self_median}
      vs {principal_median}"))
    stress_order_rate = float(np.mean([int(row["true_order"]) == int(row["recovered_order"]) for row in
     stress_rows]))
    stress_location_rate = float(np.mean([float(row["location_error"]) < 1e-3 for row in stress_rows]))
    tests.append(("random-family order recovery", stress_order_rate >= 0.98, str(stress_order_rate)))
    tests.append(("random-family branch recovery", stress_location_rate >= 0.98, str(stress_location_rate)))
    finite_budget_pass = (
        int(finite_budget["recovered_order"]) == edge.order
        and int(finite_budget["high_frequency_sample_count"]) == int(finite_budget["theoretical_budget"])
        and float(finite_budget["location_error"]) < 1e-10
        and float(finite_budget["jump_error"]) < 1e-10
    )
    tests.append(("explicit finite sample budget", finite_budget_pass, str(finite_budget)))

    return [
        {"name": name, "passed": bool(passed), "detail": detail}
        for name, passed, detail in tests
    ]


def generate_figures(
    base: Path,
    edge: EdgePacket,
    exact_result: RecoveryResult,
    analytic_result: RecoveryResult,
    rate_rows: list[dict[str, float | int]],
    noise_rows: list[dict[str, float]],
    offset_rows: list[dict[str, float | int]],
    collision_rows: list[dict[str, float]],
    runtime_rows: list[dict[str, float | int]],
    oracle_rows: list[dict[str, float | int]],
    stress_rows: list[dict[str, float | int]],
    finite_budget: dict[str, object],
    reconstruction: dict[str, float],
) -> None:
    out = base / "figures"
    out.mkdir(parents=True, exist_ok=True)

    # 01 architecture
    fig, ax = diagram_axes((13.0, 5.6))
    ax.text(0.5, 0.91, "Self-calibrating polyphase Fourier reconstruction", ha="center", fontsize=22, weight="
     bold")
    stages = [
        ("Fourier data", BLUE),
        ("derivative moments", TEAL),
        ("decimated Hankel rank", GREEN),
        ("aliased nodes", ORANGE),
        ("offset phases", MAGENTA),
        ("branch lifting", PURPLE),
        ("edge-native synthesis", RED),
    ]
    xs = np.linspace(0.02, 0.84, len(stages))
    for i, ((label, color), x0) in enumerate(zip(stages, xs, strict=True)):
        draw_box(ax, (x0, 0.43), 0.13, 0.18, label, face=color + "20", edge=color, fontsize=10.5)
        if i < len(stages) - 1:
```

```
            draw_arrow(ax, (x0 + 0.13, 0.52), (xs[i + 1] - 0.005, 0.52))
    ax.text(0.5, 0.18, "Unknown jump count, no externally supplied jump neighborhoods, and a validation residual from unused offsets", ha="center", fontsize=12, color=MUTED)
    save_figure(fig, out, "01_architecture")

    # 02 model signal
    model = SignalModel(analytic_background(), edge)
    x = np.linspace(0.0, TAU, 2048, endpoint=False)
    fig, axes = plt.subplots(3, 1, figsize=(12.5, 7.3), sharex=True)
    axes[0].plot(x, model.background.values(x, 2048), color=BLUE, lw=2)
    axes[0].set_title("Non-bandlimited smooth background", loc="left", weight="bold")
    axes[1].plot(x, edge.values(x), color=ORANGE, lw=2)
    for tau in edge.locations:
        axes[1].axvline(tau, color=RED, lw=0.8, alpha=0.55)
    axes[1].set_title("Unknown piecewise-constant edge packet", loc="left", weight="bold")
    axes[2].plot(x, model.values(x, 2048), color=INK, lw=2)
    axes[2].set_title("Observed signal", loc="left", weight="bold")
    axes[2].set_xlabel("periodic coordinate")
    for a in axes:
        a.grid(alpha=0.16)
        a.set_ylabel("amplitude")
    fig.tight_layout()
    save_figure(fig, out, "02_signal_model")

    # 03 Fourier tail
    ns = np.arange(1, 101)
    smooth_mag = np.asarray([abs(model.background.fourier(int(n))) for n in ns])
    edge_mag = np.asarray([abs(edge.fourier(int(n))) for n in ns])
    fig, ax = plt.subplots(figsize=(10.5, 5.5))
    ax.semilogy(ns, smooth_mag, label="smooth background", color=BLUE, lw=2)
    ax.semilogy(ns, edge_mag, label="edge packet", color=ORANGE, lw=2)
    ax.set_title("The jump controls the high-frequency tail", weight="bold")
    ax.set_xlabel("Fourier index")
    ax.set_ylabel("coefficient magnitude")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, out, "03_fourier_tail")

    # 04 derivative moments
    indices = np.arange(1, 100)
    moments = np.asarray([model.moment(int(n)) for n in indices])
    true_moments = edge.moments(indices)
    fig, axes = plt.subplots(2, 1, figsize=(12.0, 6.4), sharex=True)
    axes[0].plot(indices, moments.real, color=BLUE, lw=1.1, label="observed real")
    axes[0].plot(indices, true_moments.real, color=INK, lw=0.9, ls="--", label="edge-only real")
    axes[0].legend(ncol=2)
    axes[0].set_ylabel("real part")
    axes[1].plot(indices, np.abs(moments - true_moments), color=RED, lw=1.4)
    axes[1].set_yscale("log")
    axes[1].set_ylabel("smooth-tail defect")
    axes[1].set_xlabel("moment index")
    for a in axes:
        a.grid(alpha=0.16)
    fig.suptitle("Differentiation turns jumps into an exponential moment sequence", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, .93])
    save_figure(fig, out, "04_derivative_moments")

    # 05 polyphase lattice
    fig, ax = diagram_axes((12.5, 6.0))
    ax.text(0.5, 0.93, "A high-frequency band is reorganized into offset polyphase channels", ha="center", fontsize=20, weight="bold")
    N = 10
    for r in range(6):
        y = 0.78 - r * 0.115
        ax.text(0.06, y, f"r={r}", ha="right", va="center", weight="bold", color=PALETTE[r])
        for q in range(8):
            n = (q + 1) * N + r
            ax.add_patch(Circle((0.11 + q * 0.105, y), 0.023, facecolor=PALETTE[r] + "33", edgecolor=PALETTE[r], lw=1.4))
            ax.text(0.11 + q * 0.105, y - 0.045, str(n), ha="center", fontsize=8)
    ax.text(0.5, 0.08, r"m_{qN+r}=\sum_j (a_jz_j^r)(z_j^N)^q+e_{qN+r}", ha="center", fontsize=16)
```

```
    save_figure(fig, out, "05_polyphase_lattice")

    # 06 rank spectrum
    s = analytic_result.singular_values
    fig, ax = plt.subplots(figsize=(9.5, 5.2))
    ax.semilogy(np.arange(1, min(13, len(s)) + 1), s[:12], marker="o", color=TEAL, lw=2)
    ax.axvline(edge.order + 0.5, color=RED, ls="--", lw=1.2, label=f"estimated order = {edge.order}")
    ax.set_title("Hankel singular values expose the unknown jump count", weight="bold")
    ax.set_xlabel("singular-value index")
    ax.set_ylabel("singular value")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, out, "06_hankel_rank")

    # 07 aliased unit circle
    N = analytic_result.decimation
    fig, axes = plt.subplots(1, 2, figsize=(12.0, 5.2))
    theta = np.linspace(0, TAU, 400)
    for axr in axes:
        axr.plot(np.cos(theta), np.sin(theta), color=GRID, lw=1.2)
        axr.axhline(0, color=GRID, lw=0.7)
        axr.axvline(0, color=GRID, lw=0.7)
        axr.set_aspect("equal")
        axr.set_xlim(-1.15, 1.15)
        axr.set_ylim(-1.15, 1.15)
    axes[0].scatter(edge.nodes.real, edge.nodes.imag, c=PALETTE[:edge.order], s=85, edgecolor=INK)
    axes[0].set_title("true edge phases $z_j$", weight="bold")
    u = edge.nodes**N
    axes[1].scatter(u.real, u.imag, c=PALETTE[:edge.order], s=85, edgecolor=INK)
    axes[1].set_title(rf"aliased phases $u_j=z_j^{{{N}}}$", weight="bold")
    fig.suptitle("Decimation improves separation but creates branch ambiguity", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig, out, "07_alias_geometry")

    # 08 branch candidates
    fig, ax = plt.subplots(figsize=(7.2, 7.2))
    ax.plot(np.cos(theta), np.sin(theta), color=GRID)
    u0 = analytic_result.aliased_nodes[0] / abs(analytic_result.aliased_nodes[0])
    candidates = np.exp(1j * (np.angle(u0) + TAU * np.arange(N)) / N)
    ax.scatter(candidates.real, candidates.imag, s=48, color=BLUE, label="$N$ candidate roots")
    zc = analytic_result.coarse_nodes[0]
    zhat = analytic_result.nodes[0]
    ax.scatter([zc.real], [zc.imag], marker="x", s=110, color=MAGENTA, linewidth=2.4, label="offset phase
     estimate")
    ax.scatter([zhat.real], [zhat.imag], marker="*", s=160, color=ORANGE, edgecolor=INK, label="selected branch")
    ax.set_aspect("equal")
    ax.set_xlim(-1.15,1.15)
    ax.set_ylim(-1.15,1.15)
    ax.set_title("Offset phases select the correct root internally", weight="bold")
    ax.legend(loc="upper right")
    ax.grid(alpha=.12)
    save_figure(fig, out, "08_branch_selection")

    # 09 offset coefficient spirals
    coeff = analytic_result.offset_coefficients
    fig, axes = plt.subplots(2, 2, figsize=(11.5, 8.5))
    for j, axr in enumerate(axes.ravel()):
        values = coeff[:, j]
        axr.plot(values.real, values.imag, marker="o", color=PALETTE[j], lw=1.8)
        for r, value in enumerate(values):
            axr.text(value.real, value.imag, str(r), fontsize=8)
        axr.set_title(f"node {j+1}, phase residual {analytic_result.phase_residuals[j]:.2e}", weight="bold",
     fontsize=10.5)
        axr.axhline(0,color=GRID,lw=.7)
        axr.axvline(0,color=GRID,lw=.7)
        axr.set_aspect("equal", adjustable="datalim")
    fig.suptitle("Across offsets, each coefficient follows one geometric phase progression", fontsize=16.5,
     weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig, out, "09_offset_coefficients")

```

```
    # 10 validation bars
    fig, ax = plt.subplots(figsize=(9.5, 5.0))
    ax.bar(np.arange(edge.order), analytic_result.phase_residuals, color=PALETTE[:edge.order], edgecolor=INK)
    ax.set_yscale("log")
    ax.set_xticks(np.arange(edge.order), [f"edge {j+1}" for j in range(edge.order)])
    ax.set_ylabel("normalized offset recurrence residual")
    ax.set_title("Unused offsets provide an a posteriori phase certificate", weight="bold")
    ax.grid(alpha=.18, axis="y")
    save_figure(fig, out, "10_validation_residual")

    # 11 exact recovery table visual
    fig, ax = plt.subplots(figsize=(11.0, 4.8))
    ax.axis("off")
    columns = ["edge", "true location", "recovered location", "true jump", "recovered jump"]
    rows = []
    for j in range(edge.order):
        rows.append([
            str(j+1),
            f"{edge.locations[j]:.12f}",
            f"{analytic_result.locations[j]:.12f}",
            f"{edge.jumps[j]:.8f}",
            f"{analytic_result.jumps[j]:.8f}",
        ])
    table = ax.table(cellText=rows, colLabels=columns, loc="center", cellLoc="center")
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.scale(1.0, 1.65)
    ax.set_title("Recovered edge parameters", weight="bold", pad=18)
    save_figure(fig, out, "11_recovered_parameters")

    # 12 rate curves
    fig, ax = plt.subplots(figsize=(10.5, 6.0))
    for p, color in zip(range(4), PALETTE[:4], strict=True):
        group = [row for row in rate_rows if int(row["p"]) == p]
        Nvals = np.asarray([row["decimation"] for row in group], dtype=float)
        errors = np.asarray([row["location_error"] for row in group], dtype=float)
        ax.loglog(Nvals, errors, marker="o", lw=1.9, color=color, label=f"tail exponent p={p}")
    ax.set_title("Jump-location error improves with smooth-background decay", weight="bold")
    ax.set_xlabel("decimation scale N")
    ax.set_ylabel("maximum circular location error")
    ax.grid(alpha=.2, which="both")
    ax.legend()
    save_figure(fig, out, "12_rate_curves")

    # 13 fitted slopes
    slopes=[]
    for p in range(4):
        group=[row for row in rate_rows if int(row["p"])==p and int(row["decimation"])>=8]
        xx=np.log(np.asarray([row["decimation"] for row in group],float))
        yy=np.log(np.maximum(np.asarray([row["location_error"] for row in group],float),1e-16))
        slopes.append(float(np.polyfit(xx,yy,1)[0]))
    fig, ax=plt.subplots(figsize=(9.5,5.2))
    ax.bar(np.arange(4),[-s for s in slopes],color=PALETTE[:4],edgecolor=INK,label="observed magnitude")
    ax.plot(np.arange(4),np.arange(2,6),marker="o",color=INK,lw=1.5,label="reference p+2")
    ax.set_xticks(np.arange(4),[f"p={p}" for p in range(4)])
    ax.set_ylabel("empirical decay exponent")
    ax.set_title("Observed localization rates track the predicted one-order lifting",weight="bold")
    ax.legend();ax.grid(alpha=.16,axis="y")
    save_figure(fig,out,"13_rate_slopes")

    # 14 noise
    fig, ax=plt.subplots(figsize=(10.0,5.5))
    sig=np.asarray([row["sigma"] for row in noise_rows])
    med=np.asarray([row["median_location_error"] for row in noise_rows])
    mx=np.asarray([row["maximum_location_error"] for row in noise_rows])
    ax.loglog(sig,med,marker="o",color=TEAL,lw=2,label="median")
    ax.loglog(sig,mx,marker="o",color=ORANGE,lw=1.5,label="maximum")
    ax.set_xlabel("moment noise standard deviation")
    ax.set_ylabel("location error")
    ax.set_title("Noise propagates continuously until the branch margin is threatened",weight="bold")
    ax.grid(alpha=.2,which="both");ax.legend()
```

```
    save_figure(fig,out,"14_noise_error")

    # 15 order success
    fig, ax=plt.subplots(figsize=(9.5,5.0))
    ax.semilogx(sig,[row["order_success_rate"] for row in noise_rows],marker="o",color=BLUE,lw=2)
    ax.set_ylim(-.03,1.05)
    ax.set_xlabel("moment noise standard deviation")
    ax.set_ylabel("correct order frequency")
    ax.set_title("Unknown jump count is recovered from the Hankel gap",weight="bold")
    ax.grid(alpha=.18)
    save_figure(fig,out,"15_order_success")

    # 16 offset averaging
    fig, ax=plt.subplots(figsize=(10.0,5.5))
    R=np.asarray([row["offsets"] for row in offset_rows])
    single=np.asarray([row["single_median"] for row in offset_rows])
    multi=np.asarray([row["multi_median"] for row in offset_rows])
    ax.semilogy(R,single,marker="o",color=ORANGE,lw=2,label="single ratio")
    ax.semilogy(R,multi,marker="o",color=TEAL,lw=2,label="multi-offset phase fit")
    ax.set_xlabel("number of offset channels")
    ax.set_ylabel("median location error")
    ax.set_title("Several offsets stabilize the internal branch selector",weight="bold")
    ax.grid(alpha=.18);ax.legend()
    save_figure(fig,out,"16_offset_averaging")

    # 17 collision boundary
    fig, axes=plt.subplots(1,2,figsize=(12.0,4.9))
    sep=np.asarray([row["separation"] for row in collision_rows])
    err=np.asarray([row["location_error"] for row in collision_rows])
    gap=np.asarray([row["rank_gap"] for row in collision_rows])
    axes[0].loglog(sep,err,marker="o",color=RED,lw=2)
    axes[0].invert_xaxis();axes[0].set_xlabel("minimum separation");axes[0].set_ylabel("location error")
    axes[0].set_title("near-collision error",weight="bold");axes[0].grid(alpha=.18)
    axes[1].loglog(sep,gap,marker="o",color=PURPLE,lw=2)
    axes[1].invert_xaxis();axes[1].set_xlabel("minimum separation");axes[1].set_ylabel("rank gap")
    axes[1].set_title("Hankel gap collapse",weight="bold");axes[1].grid(alpha=.18)
    fig.suptitle("The method exposes rather than hides the separation barrier",fontsize=16.5,weight="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig,out,"17_collision_boundary")

    # 18 raw/fejer/resolved
    model2=SignalModel(power_tail_background(2),edge)
    result2=recover_polyphase(model2,10,samples_per_offset=24,offsets=6,order_upper_bound=7,refine=True)
    result2=reorder_result(result2,edge)
    xx=np.linspace(0,TAU,4096,endpoint=False)
    truth=model2.values(xx,4096)
    raw=fourier_partial(model2,xx,48)
    fej=fejer_sum(model2,xx,48)
    res=resolved_reconstruction(model2,result2,xx,48)
    fig,axes=plt.subplots(3,1,figsize=(12.3,8.2),sharex=True)
    for axr,curve,title,color in [
        (axes[0],raw,"raw Fourier partial sum",BLUE),
        (axes[1],fej,"Fejer averaging",TEAL),
        (axes[2],res,"polyphase edge recovery plus native synthesis",MAGENTA),
    ]:
        axr.plot(xx,truth,color=INK,lw=2,label="truth")
        axr.plot(xx,curve,color=color,lw=1.2,label=title)
        for tau in edge.locations: axr.axvline(tau,color=RED,lw=.65,alpha=.45)
        axr.legend(loc="upper right",ncol=2,fontsize=8.8)
        axr.grid(alpha=.15)
        axr.set_ylabel("amplitude")
    axes[-1].set_xlabel("periodic coordinate")
    fig.suptitle("The recovered edge is synthesized directly instead of through a global partial sum",fontsize
     =16.5,weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig,out,"18_reconstruction_comparison")

    # 19 errors
    fig,ax=plt.subplots(figsize=(11.0,5.3))
    ax.plot(xx,np.abs(raw-truth),color=BLUE,lw=1,label="raw Fourier")
    ax.plot(xx,np.abs(fej-truth),color=TEAL,lw=1,label="Fejer")
```

```
    ax.plot(xx,np.abs(res-truth),color=MAGENTA,lw=1,label="resolved")
    ax.set_yscale("log")
    ax.set_xlabel("periodic coordinate");ax.set_ylabel("absolute error")
    ax.set_title("Persistent oscillatory edge error is replaced by parameter and smooth-tail error",weight="bold")
    ax.legend();ax.grid(alpha=.18)
    save_figure(fig,out,"19_reconstruction_errors")

    # 20 metric bars
    labels=["raw RMSE","Fejer RMSE","resolved RMSE","edge L1","smooth RMSE"]
    vals=[reconstruction["raw_rmse"],reconstruction["fejer_rmse"],reconstruction["resolved_rmse"],reconstruction["edge_l1"],reconstruction["smooth_rmse"]]
    fig,ax=plt.subplots(figsize=(10.2,5.4))
    ax.bar(np.arange(len(vals)),vals,color=[BLUE,TEAL,MAGENTA,ORANGE,GREEN],edgecolor=INK)
    ax.set_yscale("log");ax.set_xticks(np.arange(len(vals)),labels,rotation=18,ha="right")
    ax.set_ylabel("error");ax.set_title("End-to-end reconstruction metrics",weight="bold");ax.grid(alpha=.18,axis="y")
    save_figure(fig,out,"20_metric_summary")

    # 21 negative principal branch
    u=edge.nodes**10
    principal=np.exp(1j*np.angle(u)/10)
    fig,ax=plt.subplots(figsize=(7.2,7.2))
    ax.plot(np.cos(theta),np.sin(theta),color=GRID)
    ax.scatter(edge.nodes.real,edge.nodes.imag,s=90,c=PALETTE[:edge.order],edgecolor=INK,label="true nodes")
    ax.scatter(principal.real,principal.imag,s=85,marker="x",color=RED,linewidth=2,label="principal roots")
    ax.set_aspect("equal");ax.set_xlim(-1.15,1.15);ax.set_ylim(-1.15,1.15)
    ax.set_title("Taking principal roots is not a valid de-aliasing rule",weight="bold")
    ax.legend();ax.grid(alpha=.12)
    save_figure(fig,out,"21_principal_root_failure")

    # 22 branch margin theorem picture
    fig,ax=diagram_axes((12.0,5.5))
    ax.text(.5,.91,"Branch selection is certified by a geometric margin",ha="center",fontsize=21,weight="bold")
    draw_box(ax,(.08,.45),.20,.20,r"aliased estimate\n$\widehat u\approx z^N$",face=BLUE+"20",edge=BLUE,fontsize=14)
    draw_box(ax,(.40,.45),.20,.20,r"offset estimate\n$\widetilde z\approx z$",face=TEAL+"20",edge=TEAL,fontsize=14)
    draw_box(ax,(.72,.45),.20,.20,r"nearest candidate\n$\widehat z$",face=MAGENTA+"20",edge=MAGENTA,fontsize=14)
    draw_arrow(ax,(.28,.55),(.40,.55),text=r"form $N$ roots",text_offset=(0,.06))
    draw_arrow(ax,(.60,.55),(.72,.55),text="choose within margin",text_offset=(0,.06))
    ax.text(.5,.20,r"coarse phase error + root perturbation $<\sin(\pi/N)$",ha="center",fontsize=15)
    save_figure(fig,out,"22_branch_margin")

    # 23 theorem map
    fig,ax=diagram_axes((13.0,6.0))
    ax.text(.5,.94,"The proof is modular",ha="center",fontsize=22,weight="bold")
    boxes=[
        (.04,.64,.20,.17,"Hankel gap\nrecovers $L$",BLUE),
        (.30,.64,.20,.17,"decimated Prony\nrecovers $u=z^N$",TEAL),
        (.56,.64,.20,.17,"offset phase fit\nrecovers coarse $z$",ORANGE),
        (.82,.64,.14,.17,"branch lift",MAGENTA),
        (.19,.28,.22,.17,"rate transfer\nadds one power of $N$",PURPLE),
        (.50,.28,.22,.17,"edge-native synthesis\nremoves Gibbs ringing",RED),
    ]
    for x0,y0,w,h,label,color in boxes: draw_box(ax,(x0,y0),w,h,label,face=color+"20",edge=color,fontsize=12.5)
    draw_arrow(ax,(.24,.725),(.30,.725));draw_arrow(ax,(.50,.725),(.56,.725));draw_arrow(ax,(.76,.725),(.82,.725))
    draw_arrow(ax,(.66,.64),(.41,.45),connectionstyle="arc3,rad=.12")
    draw_arrow(ax,(.89,.64),(.61,.45),connectionstyle="arc3,rad=-.12")
    ax.text(.5,.10,"Known exponential-analysis tools are used as inputs. The new theorem-facing object is their rate-preserving integration with piecewise-smooth Fourier reconstruction.",ha="center",fontsize=10.8,color=MUTED)
    save_figure(fig,out,"23_theorem_map")

    # 24 literature comparison
    labels=["Gegenbauer","Eckhoff/Batenkov","FRI/Prony","VEXPA","this synthesis"]
    properties=["unknown L","no external localization","non-bandlimited smooth tail","branch certificate","native edge synthesis"]
    matrix=np.asarray([
        [0,0,1,0,1],
```

```
        [0,0,1,0,1],
        [0,1,0,0,1],
        [1,1,0,1,0],
        [1,1,1,1,1],
    ],dtype=float)
    fig,ax=plt.subplots(figsize=(10.8,5.8))
    image=ax.imshow(matrix,aspect="auto",cmap=matplotlib.colors.ListedColormap(["#EEF1F2",TEAL]),vmin=0,vmax=1)
    ax.set_xticks(np.arange(len(properties)),properties,rotation=22,ha="right")
    ax.set_yticks(np.arange(len(labels)),labels)
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]): ax.text(j,i,"yes" if matrix[i,j] else "-",ha="center",va="center",color=WHITE if matrix[i,j] else MUTED,weight="bold")
    ax.set_title("The contribution is a synthesis of capabilities, not a claim that every ingredient is new",weight="bold")
    fig.tight_layout()
    save_figure(fig,out,"24_literature_position")

    # 25 stability barrier diagram
    fig,ax=diagram_axes((12.0,5.6))
    ax.text(.5,.91,"Why the general Fourier stability barrier is not contradicted",ha="center",fontsize=20,weight="bold")
    draw_box(ax,(.07,.48),.34,.22,"arbitrary analytic or nonperiodic function\nfirst $m$ Fourier coefficients\nstable universal reconstruction",face=RED+"18",edge=RED,fontsize=13)
    draw_box(ax,(.59,.48),.34,.22,"finite-rate edge packet\nFourier-tail decay class\nnonlinear parametric recovery",face=TEAL+"18",edge=TEAL,fontsize=13)
    ax.text(.24,.28,"root-exponential stability barrier",ha="center",color=RED,weight="bold")
    ax.text(.76,.28,"algebraic rates under model assumptions",ha="center",color=TEAL,weight="bold")
    draw_arrow(ax,(.41,.59),(.59,.59),text="different problem class",text_offset=(0,.06))
    save_figure(fig,out,"25_stability_boundary")

    # 26 multi-decimation consensus
    fig,ax=plt.subplots(figsize=(8.0,7.0))
    ax.plot(np.cos(theta),np.sin(theta),color=GRID)
    for Nval,color in [(8,BLUE),(10,TEAL),(13,ORANGE)]:
        result=recover_polyphase(model,Nval,samples_per_offset=20,offsets=5,known_order=edge.order,refine=False)
        ax.scatter(result.nodes.real,result.nodes.imag,s=55,facecolors="none",edgecolors=color,label=f"N={Nval}")
    ax.scatter(edge.nodes.real,edge.nodes.imag,s=90,c=PALETTE[:edge.order],edgecolor=INK,marker="*",label="truth")
    ax.set_aspect("equal");ax.set_xlim(-1.15,1.15);ax.set_ylim(-1.15,1.15);ax.grid(alpha=.12)
    ax.set_title("Independent decimations provide a consensus check",weight="bold");ax.legend()
    save_figure(fig,out,"26_multiscale_consensus")

    # 27 residual heatmap across N/R
    Ns=[6,8,10,12,16]
    Rs=[2,3,4,5,6]
    heat=np.zeros((len(Rs),len(Ns)))
    for i,Rval in enumerate(Rs):
        for j,Nval in enumerate(Ns):
            rr=recover_polyphase(model,Nval,samples_per_offset=20,offsets=Rval,known_order=edge.order,refine=False)
            heat[i,j]=rr.validation_score
    fig,ax=plt.subplots(figsize=(9.5,5.7));im=ax.imshow(np.log10(np.maximum(heat,1e-18)),aspect="auto",cmap="viridis")
    ax.set_xticks(np.arange(len(Ns)),Ns);ax.set_yticks(np.arange(len(Rs)),Rs)
    ax.set_xlabel("decimation N");ax.set_ylabel("offset channels R")
    ax.set_title("Validation residual guides parameter choice",weight="bold")
    cbar=fig.colorbar(im,ax=ax,fraction=.046);cbar.set_label("log10 validation residual")
    save_figure(fig,out,"27_validation_heatmap")

    # 28 amplitude recovery
    fig,ax=plt.subplots(figsize=(9.5,5.2))
    positions=np.arange(edge.order);width=.35
    ax.bar(positions-width/2,edge.jumps,width,label="true",color=INK)
    ax.bar(positions+width/2,analytic_result.jumps,width,label="recovered",color=TEAL)
    ax.axhline(0,color=GRID,lw=.8);ax.set_xticks(positions,[f"edge {j+1}" for j in positions])
    ax.set_ylabel("jump amplitude");ax.set_title("Jump amplitudes are recovered after phase lifting",weight="bold")
    ax.legend();ax.grid(alpha=.15,axis="y")
    save_figure(fig,out,"28_amplitude_recovery")

    # 29 recovered smooth packet
```

```
    recovered_edge=edge_from_result(analytic_result)
    smooth_rec=np.zeros_like(x,dtype=complex)
    for n in range(-40,41): smooth_rec += (model.fourier(n)-recovered_edge.fourier(n))*np.exp(1j*n*x)
    fig,ax=plt.subplots(figsize=(11.5,5.2))
    ax.plot(x,model.background.values(x,2048),color=INK,lw=2,label="true smooth packet")
    ax.plot(x,np.real(smooth_rec),color=BLUE,lw=1.2,ls="--",label="recovered from residual Fourier data")
    ax.set_xlabel("periodic coordinate");ax.set_ylabel("amplitude")
    ax.set_title("After edge removal, the remaining Fourier data are smooth",weight="bold")
    ax.legend();ax.grid(alpha=.16)
    save_figure(fig,out,"29_smooth_packet_recovery")

    # 30 edge packet recovery
    fig,ax=plt.subplots(figsize=(11.5,5.2))
    ax.step(x,edge.values(x),where="post",color=INK,lw=2,label="true edge packet")
    ax.step(x,recovered_edge.values(x),where="post",color=ORANGE,lw=1.2,ls="--",label="recovered edge packet")
    ax.set_xlabel("periodic coordinate");ax.set_ylabel("amplitude")
    ax.set_title("The singular packet is reconstructed in its native coordinate",weight="bold")
    ax.legend();ax.grid(alpha=.16)
    save_figure(fig,out,"30_edge_packet_recovery")

    # 31 runtime
    fig,ax=plt.subplots(figsize=(9.5,5.2))
    Q=np.asarray([row["samples_per_offset"] for row in runtime_rows])
    times=np.asarray([row["median_seconds"] for row in runtime_rows])
    ax.plot(Q,times*1000,marker="o",color=PURPLE,lw=2)
    ax.set_xlabel("samples per offset channel");ax.set_ylabel("median runtime (ms)")
    ax.set_title("The finite solver remains small compared with dense global reconstruction",weight="bold")
    ax.grid(alpha=.18)
    save_figure(fig,out,"31_runtime")

    # 32 algorithm pseudocode graph
    fig,ax=diagram_axes((12.5,6.2))
    ax.text(.5,.94,"Executable algorithm",ha="center",fontsize=22,weight="bold")
    steps=[
        (.06,.68,"1\nform $m_n$",BLUE),(.27,.68,"2\nsplit offsets",TEAL),(.48,.68,"3\nestimate rank",GREEN)
     ,(.69,.68,"4\nrecover $u_j$",ORANGE),
        (.17,.30,"5\nfit offset coefficients",MAGENTA),(.43,.30,"6\nphase lift",PURPLE),(.69,.30,"7\nrefine and
     synthesize",RED),
    ]
    for x0,y0,label,color in steps: draw_box(ax,(x0,y0),.17,.16,label,face=color+"20",edge=color,fontsize=12)
    for a,b in [((.23,.76),(.27,.76)),((.44,.76),(.48,.76)),((.65,.76),(.69,.76)),((.78,.68),(.72,.46))
     ,((.69,.38),(.60,.38)),((.43,.38),(.34,.38))]: draw_arrow(ax,a,b)
    ax.text(.5,.10,"Every stage emits a residual or gap that can be checked before the next stage is trusted.",ha
     ="center",fontsize=11.5,color=MUTED)
    save_figure(fig,out,"32_algorithm")

    # 33 certificate dashboard
    metrics={
        "order":float(analytic_result.order),
        "order gap":analytic_result.order_gap,
        "location error":reconstruction["location_error"],
        "jump error":reconstruction["jump_error"],
        "validation":analytic_result.validation_score,
        "resolved RMSE":reconstruction["resolved_rmse"],
    }
    fig,ax=plt.subplots(figsize=(11.0,5.8));ax.axis("off")
    ax.text(.5,.94,"Certificate dashboard",ha="center",fontsize=21,weight="bold",transform=ax.transAxes)
    y=.82
    for i,(name,value) in enumerate(metrics.items()):
        face=WHITE if i%2==0 else "#EEF1F2"
        ax.add_patch(FancyBboxPatch((.10,y-.042),.80,.065,boxstyle="round,pad=.005,rounding_size=.008",facecolor=
     face,edgecolor="none",transform=ax.transAxes))
        ax.text(.13,y,name,ha="left",va="center",fontsize=12,transform=ax.transAxes)
        ax.text(.87,y,f"{value:.6e}",ha="right",va="center",fontsize=12,family="monospace",transform=ax.transAxes
     ,color=TEAL if value<1e-4 or name=="order" else ORANGE)
        y-=.105
    save_figure(fig,out,"33_certificate_dashboard")

    # 34 scope diagram
    fig,ax=diagram_axes((12.0,5.8))
    ax.text(.5,.92,"What is proved and what remains outside the theorem",ha="center",fontsize=20,weight="bold")
```

```
    draw_box(ax,(.06,.46),.40,.26,"proved in the declared class\nunknown finite jump count\nno external localization\nrate-preserving de-aliasing\nGibbs-free native synthesis",face=TEAL+"18",edge=TEAL,fontsize=13)
    draw_box(ax,(.54,.46),.40,.26,"not claimed\narbitrary edge geometry\nseparation-free stability\nunbounded jump count\nmodel-free exponential convergence",face=ORANGE+"18",edge=ORANGE,fontsize=13)
    ax.text(.5,.18,"The advance is a sharper theorem for a structured class, not a denial of the general impossibility results.",ha="center",fontsize=11.5,color=MUTED)
    save_figure(fig,out,"34_scope_boundary")

    # 35 final synthesis
    fig,ax=diagram_axes((13.0,6.0))
    ax.text(.5,.94,"The field-facing result",ha="center",fontsize=23,weight="bold")
    draw_box(ax,(.07,.58),.24,.20,"decimated algebraic recovery\noptimal conditioning",face=BLUE+"18",edge=BLUE,fontsize=13)
    draw_box(ax,(.38,.58),.24,.20,"offset polyphase lifting\ninternal de-aliasing",face=TEAL+"18",edge=TEAL,fontsize=13)
    draw_box(ax,(.69,.58),.24,.20,"projector synthesis\nno Gibbs edge ringing",face=MAGENTA+"18",edge=MAGENTA,fontsize=13)
    draw_arrow(ax,(.31,.68),(.38,.68));draw_arrow(ax,(.62,.68),(.69,.68))
    ax.text(.5,.33,"unknown jump count + no supplied jump neighborhoods + one-order rate lift + explicit certificates",ha="center",fontsize=15,weight="bold")
    ax.text(.5,.15,"The shifted-channel identity is established exponential-analysis machinery. The contribution is the theorem that carries it into self-calibrating piecewise-smooth Fourier reconstruction and preserves the localization rate.",ha="center",fontsize=10.8,color=MUTED,wrap=True)
    save_figure(fig,out,"35_final_synthesis")

    # 36 oracle equivalence
    fig, ax = plt.subplots(figsize=(10.5, 6.0))
    group = [row for row in oracle_rows if int(row["p"]) == 2]
    Nvals = np.asarray([int(row["decimation"]) for row in group])
    principal = np.asarray([float(row["principal_error"]) for row in group])
    oracle = np.asarray([float(row["oracle_error"]) for row in group])
    self_error = np.asarray([float(row["self_calibrated_error"]) for row in group])
    ax.semilogy(Nvals, principal, marker="o", lw=2, color=RED, label="principal root")
    ax.semilogy(Nvals, oracle, marker="o", lw=2.6, color=INK, label="oracle branch")
    ax.semilogy(Nvals, self_error, marker="x", ms=8, lw=1.4, color=TEAL, label="self-calibrated branch")
    ax.set_xlabel("decimation scale N")
    ax.set_ylabel("maximum circular location error")
    ax.set_title("The offset phase recovers the same branch as the oracle", weight="bold")
    ax.grid(alpha=.18, which="both")
    ax.legend()
    save_figure(fig, out, "36_oracle_equivalence")

    # 37 oracle-self distance heatmap
    pvals = sorted({int(row["p"]) for row in oracle_rows})
    nvals = sorted({int(row["decimation"]) for row in oracle_rows})
    matrix = np.zeros((len(pvals), len(nvals)))
    for row in oracle_rows:
        i = pvals.index(int(row["p"]))
        j = nvals.index(int(row["decimation"]))
        matrix[i, j] = max(float(row["oracle_self_node_distance"]), 1e-18)
    fig, ax = plt.subplots(figsize=(11.0, 4.8))
    image = ax.imshow(np.log10(matrix), aspect="auto", cmap="magma", vmin=-18, vmax=-8)
    ax.set_xticks(np.arange(len(nvals)), nvals)
    ax.set_yticks(np.arange(len(pvals)), [f"p={p}" for p in pvals])
    ax.set_xlabel("decimation scale N")
    ax.set_title("Oracle and self-calibrated node sets agree to numerical precision", weight="bold")
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            ax.text(j, i, f"{matrix[i,j]:.1e}", ha="center", va="center", fontsize=8.5, color=WHITE if np.log10(matrix[i,j]) < -12 else INK)
    fig.colorbar(image, ax=ax, fraction=.035, pad=.025, label="log10 node-set distance")
    fig.tight_layout()
    save_figure(fig, out, "37_oracle_distance")

    # 38 random stress error distribution
    finite_errors = np.asarray([float(row["location_error"]) for row in stress_rows if math.isfinite(float(row["location_error"]))])
    fig, ax = plt.subplots(figsize=(10.0, 5.5))
    ax.hist(np.log10(np.maximum(finite_errors, 1e-18)), bins=24, color=TEAL, edgecolor=INK, alpha=.88)
    ax.axvline(-3, color=RED, ls="--", lw=1.5, label="10^-3 success threshold")
```

```
    ax.set_xlabel("log10 maximum circular location error")
    ax.set_ylabel("number of cases")
    ax.set_title("One hundred separated random signals with unknown jump count", weight="bold")
    ax.legend()
    ax.grid(alpha=.15, axis="y")
    save_figure(fig, out, "38_random_stress_errors")

    # 39 random order confusion matrix
    orders = [2, 3, 4, 5]
    confusion = np.zeros((len(orders), len(orders)), dtype=int)
    for row in stress_rows:
        t = int(row["true_order"])
        r = int(row["recovered_order"])
        if t in orders and r in orders:
            confusion[orders.index(t), orders.index(r)] += 1
    fig, ax = plt.subplots(figsize=(7.2, 6.1))
    image = ax.imshow(confusion, cmap="viridis")
    ax.set_xticks(np.arange(len(orders)), orders)
    ax.set_yticks(np.arange(len(orders)), orders)
    ax.set_xlabel("recovered jump count")
    ax.set_ylabel("true jump count")
    ax.set_title("Unknown-order recovery across the random stress family", weight="bold")
    for i in range(len(orders)):
        for j in range(len(orders)):
            value = confusion[i, j]
            color = WHITE if value > max(1, confusion.max() / 2) else INK
            ax.text(j, i, str(value), ha="center", va="center", fontsize=13, weight="bold", color=color)
    fig.colorbar(image, ax=ax, fraction=.045, pad=.035, label="cases")
    fig.tight_layout()
    save_figure(fig, out, "39_random_order_confusion")



    # 40 explicit finite sample budget
    base_indices = np.asarray(finite_budget["base_indices"], dtype=int)
    offset_indices = np.asarray(finite_budget["offset_indices"], dtype=int)
    fig, ax = plt.subplots(figsize=(12.0, 4.8))
    ax.scatter(base_indices, np.zeros_like(base_indices), s=70, color=BLUE, label=r"base channel $r=0$")
    ax.scatter(offset_indices, np.ones_like(offset_indices), s=70, color=ORANGE, label=r"adjacent channel $r=1$")
    for n in base_indices:
        ax.vlines(n, -0.08, 0.08, color=BLUE, alpha=0.45)
    for n in offset_indices:
        ax.vlines(n, 0.92, 1.08, color=ORANGE, alpha=0.45)
    ax.set_yticks([0, 1], [r"$2L_{\max}+1$ moments", r"$L_{\max}$ moments"])
    ax.set_xlabel("Fourier derivative-moment index")
    ax.set_title(
        "Exact self-calibration uses an explicit high-frequency budget of "
        + str(finite_budget["high_frequency_sample_count"])
        + " coefficients",
        weight="bold",
    )
    ax.grid(alpha=0.16, axis="x")
    ax.legend(loc="upper center", ncol=2, bbox_to_anchor=(0.5, -0.16))
    fig.tight_layout(rect=[0, 0.08, 1, 1])
    save_figure(fig, out, "40_finite_sample_budget")

def write_csv(path: Path, rows: Sequence[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        path.write_text("", encoding="utf-8")
        return
    fields = list(rows[0].keys())
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
```

```
        for block in iter(lambda: handle.read(1 << 20), b""):
            digest.update(block)
    return digest.hexdigest()


def run_all(base: Path) -> dict[str, object]:
    base.mkdir(parents=True, exist_ok=True)
    (base / "results").mkdir(exist_ok=True)
    (base / "tables").mkdir(exist_ok=True)

    edge = exact_edge()
    exact_model = SignalModel(bandlimited_background(), edge)
    analytic_model = SignalModel(analytic_background(), edge)

    exact = reorder_result(
        recover_polyphase(exact_model, 10, samples_per_offset=22, offsets=6, order_upper_bound=7, refine=True),
        edge,
    )
    analytic = reorder_result(
        recover_polyphase(analytic_model, 10, samples_per_offset=22, offsets=6, order_upper_bound=7, refine=True)
    ,
        edge,
    )
    symbolic = symbolic_confluent_check()
    rate_rows = run_rate_experiment(edge)
    noise_rows = run_noise_experiment(edge)
    offset_rows = run_offset_averaging_experiment(edge)
    collision_rows = run_collision_experiment()
    runtime_rows = run_runtime_experiment(edge)
    oracle_rows = run_oracle_equivalence_experiment(edge)
    stress_rows = run_random_stress_experiment(100)
    finite_budget = run_finite_budget_experiment(edge)
    reconstruction = run_reconstruction_experiment(edge)

    tests = run_tests(edge, exact, analytic, symbolic, rate_rows, noise_rows, offset_rows, oracle_rows,
     stress_rows, finite_budget, reconstruction)
    failed = [row for row in tests if not row["passed"]]
    if failed:
        raise AssertionError(f"failed tests: {failed}")

    generate_figures(
        base,
        edge,
        exact,
        analytic,
        rate_rows,
        noise_rows,
        offset_rows,
        collision_rows,
        runtime_rows,
        oracle_rows,
        stress_rows,
        finite_budget,
        reconstruction,
    )

    write_csv(base / "tables" / "rate_experiment.csv", rate_rows)
    write_csv(base / "tables" / "noise_experiment.csv", noise_rows)
    write_csv(base / "tables" / "offset_averaging.csv", offset_rows)
    write_csv(base / "tables" / "collision_experiment.csv", collision_rows)
    write_csv(base / "tables" / "runtime.csv", runtime_rows)
    write_csv(base / "tables" / "oracle_equivalence.csv", oracle_rows)
    write_csv(base / "tables" / "random_stress.csv", stress_rows)
    write_csv(base / "tables" / "finite_budget.csv", [finite_budget])
    write_csv(base / "tables" / "tests.csv", tests)

    result = {
        "title": "Self-Calibrating Polyphase Fourier Reconstruction",
        "jump_count": edge.order,
        "true_locations": edge.locations.tolist(),
        "true_jumps": edge.jumps.tolist(),
```

```
        "exact_recovered_locations": exact.locations.tolist(),
        "exact_recovered_jumps": exact.jumps.tolist(),
        "analytic_recovered_locations": analytic.locations.tolist(),
        "analytic_recovered_jumps": analytic.jumps.tolist(),
        "analytic_order_gap": analytic.order_gap,
        "analytic_validation_score": analytic.validation_score,
        "symbolic_confluent_check": symbolic,
        "reconstruction": reconstruction,
        "oracle_equivalence_max_node_distance": max(float(row["oracle_self_node_distance"]) for row in oracle_rows),
        "random_stress_order_success": float(np.mean([int(row["true_order"]) == int(row["recovered_order"]) for row in stress_rows])),
        "random_stress_location_success": float(np.mean([float(row["location_error"]) < 1e-3 for row in stress_rows])),
        "random_stress_median_location_error": float(np.median([float(row["location_error"]) for row in stress_rows])),
        "random_stress_maximum_location_error": float(np.max([float(row["location_error"]) for row in stress_rows])),
        "finite_budget": finite_budget,
        "test_count": len(tests),
        "tests_passed": len(tests),
        "figure_pairs": 40,
    }
    (base / "results" / "results.json").write_text(json.dumps(result, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    return result


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", type=Path, default=Path(__file__).resolve().parent)
    args = parser.parse_args()
    result = run_all(args.out)
    print(json.dumps(result, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

References

[1] B. Adcock, A. C. Hansen, and A. Shadrin, *A stability barrier for reconstructions from Fourier samples*, SIAM Journal on Numerical Analysis 52 (2014), 125–139.

[2] D. Batenkov, *Complete algebraic reconstruction of piecewise-smooth functions from Fourier data*, Mathematics of Computation 84 (2015), 2329–2350.

[3] D. Batenkov and Y. Yomdin, *Algebraic Fourier reconstruction of piecewise smooth functions*, Mathematics of Computation 81 (2012), 277–318.

[4] D. Batenkov, *Accurate solution of near-colliding Prony systems via decimation and homotopy continuation*, Theoretical Computer Science 681 (2017), 27–40.

[5] M. Briani, A. Cuyt, F. Knaepkens, and W.-s. Lee, *VEXPA: validated exponential analysis through regular sub-sampling*, Signal Processing 147 (2018), 207–220.

[6] M. Vetterli, P. Marziliano, and T. Blu, *Sampling signals with finite rate of innovation*, IEEE Transactions on Signal Processing 50 (2002), 1417–1428.

[7] A. Gelb and Z. Jackiewicz, *Determining analyticity for parameter optimization of the Gegenbauer reconstruction method*, SIAM Journal on Scientific Computing 27 (2005), 1014–1031.

[8] D. Gottlieb and C.-W. Shu, *On the Gibbs phenomenon and its resolution*, SIAM Review 39 (1997), 644–668.

[9] D. Batenkov and Y. Yomdin, *Local and global geometry of Prony systems and Fourier reconstruction of piecewise-smooth functions*, Contemporary Mathematics 586 (2013), 57–72.

[10] A. Cuyt, Y. Hou, F. Knaepkens, and W.-s. Lee, *Sparse multidimensional exponential analysis with an application to radar imaging*, SIAM Journal on Scientific Computing 42 (2020), B675–B695.